

claude-windows / 1c6eb3b4-ed47-4cb3-84e8-50b309c6495c

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\1c6eb3b4-ed47-4cb3-84e8-50b309c6495c.jsonl`
- SHA-256: `56bfe189470024c32db370956c49084471fc314e94ef0359d7ddb6da96bdced4`
- Source modified: `2026-06-10T12:13:01+00:00`
- Imported at: `2026-07-05T16:48:17+00:00`
- project: `C--Users-avidu-Projects-badminton-highlight-indexer`
- session_id: `1c6eb3b4-ed47-4cb3-84e8-50b309c6495c`
```

Transcript

1. user (2026-06-10T11:17:29.179Z)

In C:\Users\avidu\Projects\badminton-highlight-indexer: the gemini segmenter (backend/pipeline/segmenters/gemini.py) extracts 90s chunks via stream-copy, so 4K source video produces ~1.3GB chunks that exceed backend/providers/gemini.py::MAX_UPLOAD_MB=500 ? and the provider SILENTLY skips oversize clips (documented audit gotcha), yielding "0 play segments / success". Observed live 2026-06-10 on KushagraYashAviFirstVideo.MP4 (4K): all 7 chunks 832-1299MB, all skipped, 0 segments.

Fix: (1) in the chunk-extraction step, when the source's bitrate/size implies chunks over ~450MB (or always, for AI handoff), pass reencode=True + scale_width (e.g. 1280 ? matches gemini_refine's --clip-scale-width default) to backend/utils/video.py::extract_video_chunk; re-encoding also fixes the keyframe-drift gotcha for chunk boundaries. Consider a config knob (indexing.ai_chunk_scale_width, default 1280; typed in backend/config/models.py::IndexingConfig ? note nested config needs the field DECLARED, extra keys are dropped). (2) Make the provider's oversize skip LOUD: logger.error + surface in the per-video observability report (a new spec.yaml rule, e.g. oversize_clip_skipped ? remember spec.yaml path strings couple to backend/reporting/harvest.py detail names). (3) Tests: mocked chunk-extraction asserts reencode/scale args for a 4K source; provider oversize path logs and is reflected in the report. Run python run_all_tests.py (offline). Branch off master, one PR, descriptive message. Conventions in CLAUDE.md.

2. assistant (2026-06-10T11:17:41.768Z)

I'll pick up this fix. Let me first read the session handoff and the relevant code.

3. user (2026-06-10T11:17:43.307Z)

<system-reminder>[Truncated: PARTIAL view ? showing lines 1-471 of 1232 total (55518 tokens, cap 25000). Call Read with offset=472 limit=471 for the next page, or Grep to find a specific section. Do NOT answer from this page alone if the answer may be further in the file.]</system-reminder>

```
1      ---
2      name: current-state
3      description: "Live micro-checkpoint state for Claude ? updated every checkpoint (not
4      metadata:
5          node_type: memory
6          type: project
7          originSessionId: b13673cc-f087-43b1-9527-35dc0b67b301
8      ---
9
10     # Current state ? updated continuously (see [[working-agreement-checkpoints]])
11
12     **Checkpoint:** 2026-06-10 (M0 GREENLIT SEQUENCE EXECUTED ? session wrap). Owner
ratified ADR-008 ("split driven by
```

```

13     productionization, not isolation") + greenlit M0.4+M0.1. **All 5 steps shipped, every PR
merged on verified-green CI:**
14     - **#73 ADR-008** (DECISIONS.md): training in-repo behind an executable wall; artifact-
bundle boundary; ship-gate
15     product-side; pre-committed split trigger ? `sports-temporal-models` at Gen-2/second-
consumer.
16     - **#74 canonical GT loader** `backend/eval/gt_loader.py::load_gt_intervals` ? closes
the two-loader fork (accepts BOTH
17     `start,end` and `start_time,end_time`); legacy loaders delegate; round-trip contract
test; LOVO 0.626 re-verified.
18     - **#75+#76 featurizer promotion + import wall**: `backend/features/rally_seq.py`
(FEATURE_SCHEMA v1, fps semantics
19     explicit) imported by train AND serve; `training/` skeleton (5 wall rules,
requirements-train.txt); smoke tests:
20     runtime no-training-in-backend.main, static sweep, featurizer purity (torch/cv2
blocked; importorskip numpy on the
21     minimal CI lane ? #76 hotfixed a red lane after #75 merged early; merges now gated on
verified SUCCESS rollout).
22     - **collector #19 ? n=6 OWNED CORPUS IN THE SoR**: all 6 golden matches via `import-
local --normalize` (262 rallies,
23     0 dropped, Proprietary, explicit fps); commercial export now 6 real matches + demo
entry.
24     - **#77 Gen-0 harness** `python -m training.gen0.harness --manifest ? --floor
eval_baselines/heuristic_lovo_n6.json
25     --version 0.1.0`: train?honest LOVO?ship-gate?artifact bundle (gitignored weights.json
+ manifest.json w/ schema pin,
26     calibration, lineage gt_hashes, attestations incl. C3, gate verdict). **First real
bundle built: LOVO 0.626, floor
27     PASSED (0.480), paired sign test 5/6 wins p=0.109 = honestly NOT significant,
ship=False** (blockers: target
28     undefined, C3, significance). Committed floor baseline
`eval_baselines/heuristic_lovo_n6.json`. Suite **325 green**.
29     - ? **Gemini baseline STILL BLOCKED after owner provided a NEW key (2026-06-10 later):**
key written to gitignored
30     `.env`, validated via `models.list` (55 models incl. `gemini-flash-latest`; **config
migrated off deprecating
31     gemini-2.5-flash ? `gemini-flash-latest` alias, PR #78 merged**; gemini_refine already
defaulted to it). But inference
32     = 429 RESOURCE_EXHAUSTED "prepayment credits depleted" ? **the AI-Studio PROJECT
behind the key has no credits** (key
33     auth ? billing). Owner: add credits at https://ai.studio/projects (+ consider rotating
the chat-pasted key). Planned
34     battery on unblock (in task 17): wrapper Mahadevpura ? score; wrapper Kushagra (multi-
court MOAT test); two-stage
35     gemini_refine; compile a real highlight reel (academy-demo artifact). Pipeline handled
both failed runs gracefully
36     (validators pass, reports emitted, artifacts cleaned); shared-genai-client thread-
safety finding observed live.
37     - **C13 field plan (owner executing):** recruit interviewer; camera idea RATIFIED w/ 3
constraints (consent-at-capture/
38     adults-only for training; demo via Gemini-or-human-polish, NOT gated on owned model;
interviews?cameras sequencing).
39     - ?? NEXT: owner tops up Gemini billing (wrapper baseline) . owner sets the ship-gate
F1@tIoU target . grow corpus
40     (camera pilot videos ? import-local ? re-run harness) . repo rename . M0.1 event-index
contract in collector.
41
42     **Checkpoint:** 2026-06-10 (Roora PB+TT pilot review ? REPORT DELIVERED to owner; owner
emails Roora himself) .
43     ? Owner cross-checked the rally tables in VLC ? **CONFIRMED: every boundary off ?0.5s
(up to ~2s)** vs the ±0.3s bar
44     (Brief §4). **Root cause (my forensics): all 24 boundaries sit EXACTLY on the 30-frame
sampling grid** (~0.5s at
45     59.94fps) ? first/last box never on the true contact/dead frame ? structurally cannot
meet ±0.3s. Also decoded their

```

46 bogus `start/end time` attrs = ****frame=900** (decimal minutes @ assumed 15fps, ~4x off wall-clock)**. Report findings:

47 boundary accuracy (CRITICAL), skip-rallies-in-progress-at-file-start/end convention (PB rally#1 starts at frame 0 ?

48 shouldn't have been labeled), dead-time+warmup reminders for Phase-B, schema cut to exactly

49 `rally\_number,shot\_count,ending\_reason,last\_shot\_outcome,note` (drop their time attrs + scores), field quality

50 (PB outcome='0' off-vocab; TT 5/6 'other' + missing required notes; TT#5 intra-rally other|winner). Per owner:

51 NO redelivery demand in the report (he words the email himself; thread = Jamshad@roora on avidullu@live.com Outlook).

52 ****Artifacts:**** PDF `Annotation Setup\6 June 2026\Feedback\Roora Pilot Review - Pickleball and Table Tennis

53 (2026-06-10).pdf` + md record at `Annotation Setup\Roora Pilot Review ? Pickleball & Table Tennis (2026-06-10).md`

54 (generator: `%TEMP%\roora\make\_review\_pdf.py`). ? Local guideline docs are ****DRAFT v2**** ? owner mentioned "v7";

55 no v7 found on disk (possibly in the email thread; report cites v2). Spawned chip `task\_c55f0be6`: collector

56 `rally\_cvat.py` ATTR_MAP drops `shot\_count` (present in Roora XML, empty in our CSVs) ? OUR bug, kept out of the

57 vendor report. ?? owner sends email; Roora's badminton long-video + redelivery decisions pending owner.

58

59 ****Checkpoint:**** 2026-06-10 (REPO-TOPOLOGY decision panel + C13 plan ? AWAITING OWNER RATIFICATION). Owner leaning

60 M0.4+M0.1 greenlight; asked: model in-repo vs separate repo consumed as a segmenter (owner prefers isolation). Ran a

61 4-agent judge panel (`wf\_471a9cdd-d1d`; full output `?\tasks\wnksv545g.output`). ****Verdict** (my rec): "artifact boundary

62 NOW, repo split at Gen-2" ? M0.4 lands as a top-level `training/` tree in the indexer w/ executable isolation

63 (import-smoke asserts backend.main loads no training module; own requirements-train.txt + 3rd CI lane), featurizer

64 promoted to `backend/features/rally\_seq.py` (single source, train==serve), artifact bundle = gitignored weights

65 (private Releases) + committed manifest (feature-schema ver, normalization, calibration, corpus ref+gt_hash,

66 consent/license attestation incl. C3), ship-gate stays product-side (regression.py extension); ****pre-committed split**

67 trigger****** to a named repo (`sports-temporal-models`) when Gen-2/cloud-training or a 2nd artifact consumer arrives.

68 Decisive evidence: Gen-0's input = the indexer's OWN featurization (mid-pipeline seam ? WASB/Gemini video-boundary);

69 indexer not pip-installable ? split today = copy-paste = the observed drift class; obsreport "precedent" never actually

70 exercised (pin is a commented line, repo unpublished, CI skips). ****FIRST** contract to freeze: golden-label/corpus

71 contract****** ? indexer carries TWO incompatible GT loaders (annotations.py `start,end` vs calibrate_wasb `start_time,

72 end_time` ? fed to 7 modules) vs collector export `start,end`; consolidate + round-trip test BEFORE any training epoch.

73 Panel extras: n=6 corpus's durable home = loose `Annotation Setup\` folder (neither repo!) ? M0.1 first deliverable =

74 hash-addressed in collector; training fixtures synthetic-only until consent ledger exists; run the Gemini-wrapper

75 baseline before/with M0.4. ****C13 field plan:**** owner recruiting an interviewer; my rec = bake in the camera idea

76 (paid recordings = consented owned multi-court corpus + demo), demo served via Gemini path or human-polish (owned model

77 honest 0.626 ? >0.8 yet; the comparison doubles as the wrapper baseline). ?? awaiting owner ratification of topology +

78 M0.4/M0.1 scope; then: ADR + GT-loader consolidation + featurizer promotion + harness scaffold.

```

79
80     **Checkpoint:** 2026-06-10 (CORPUS RECLASS EXECUTED + NEXT_STEPS refreshed ? session
wrap). Owner approved + I ran the
81     corpus correction: `reclassify-licenses --apply` (23 ShuttleSet broadcast rows ?
annotation_license=MIT, video=Unknown,
82     is_commercial=False; **zero annotations deleted**, 1,917 rallies intact) + `curate
export` (commercial manifest now
83     owned-data-only: 1 video/2 segments; stale ShuttleSet CSVs removed from the gitignored
eval_export; eval-only re-export
84     available via `--include-staging --include-noncommercial`). **Merged: collector #17**
(DB + .gitignore backup rule);
85     pre-change backup at `data/sports_metadata.backup-2026-06-10.db` (gitignored). Quality
signals UNAFFECTED (n=6 corpus
86     never flows through the collector DB; learned 0.626 re-verified earlier). **Indexer #72
merged:** NEXT_STEPS.md
87     refreshed (sweep banner, 0.626 vs 0.480, M0.3 partially-done with corrected
distill_local pointer, NEW prioritized PMF-
88     validation item, durable artifact paths). **Owner is now reading NEXT_STEPS to chalk out
next steps** ? open owner
89     decisions stand: greenlight M0.x (rec M0.4+M0.1), confirm collector=SoR, ship-gate
F1@tIoU, repo rename, C13 interviews
90     / Gemini-wrapper baseline. Both repos clean on master, 0 open PRs.
91
92     **Checkpoint:** 2026-06-10 (HIGH-RISK REMEDIATION COMPLETE ? owner approved "everything
A7K, open+merge PRs after CI
93     green, re-run quality evals, GPU available"). **ALL 11 PRs MERGED + CI green; both repos
clean on master.**
94     - **Indexer (badminton-highlight-indexer):** #63 compiler-syntax fix+CI (prior), **#64**
detector keying by video_id,
95     **#65** re-ingest destroy-after-confirm (add_video UPSERT + watermark/prune), **#66**
nested-config wasb typed,
96     **#67** eval gate + metric footguns (iou=0, midrank AUC, fps-normalized speed,
baseline?`eval_baselines/` exit-3),
97     **#68** API path validation, **#69** reliability (timeouts 1800s, SQLite
WAL+busy_timeout, chunk-step guard, CORS),
98     **#70** test coverage (TestClient endpoints + compiler + WasbRunner), **#71** docs
truth-sync. Suite 256?308 green**.
99     - **Collector (sports-data-collector):** **#14** licensing gate (video vs annotation
license; `reclassify-licenses`
100     found 23 conflated ShuttleSet rows) + **added collector CI**, **#15** import-local
golden-label guard (+`--force`,
101     `--normalize`, duplicate-rally guard), **#16** validate-delivery fails on
empty/corrupt/zip CVAT. Suite 124?136 green**.
102     - ? **QUALITY RE-VALIDATION (owner asked): metric fixes STRENGTHENED the conclusion.**
After midrank-AUC +
103     fps-normalized speed + occlusion-gap guard, **learned LOVO 0.583 ? 0.626** (per-video
AUC unchanged 0.83?0.92 =
104     never tie-inflated; fps-fix mainly helped the 30fps short fold 0.30?0.56). Heuristic
LOVO **0.480** unchanged (scored
105     at IoU 0.5; my iou>0 guard is a no-op there). **Learned beats heuristic by +0.146**
(was +0.10).
106     - **Eval artifacts RESCUED from `%TEMP%` ? durable `C:\Users\avidu\Projects\Annotation
Setup\Collect\Trajectories\`**
107     (+ `\\roora\`); manifest `output/human_lovo_manifest.json` repointed there + reverified
0.626.
108     - **Deferred WITH INTENT (in BACKLOG): repo-rename; Gen-2 cost benchmark; ship-gate
F1@tIoU target; packaging
109     (pyproject); app-factory/lifespan + lazy torch-cv2 imports (with async slice #16); WSL
temp/frame/cache + Gemini
110     remote-file GC (retention policy); shlex-quote WSL config values (breaks `~` expansion
? non-trivial); deeper
111     validator tests + 3-hybrid contract test (partial coverage shipped).**
112     - ? **Process note:** during PR-F I briefly `git checkout --`'d the owner's uncommitted
`data/sports_metadata.db`
113     change (200704b), then RECOVERED it from dangling stash `d3c2e77` and restored it to

```

the working tree (no loss).

114 Going forward stash-and-pop that DB for collector PRs; never `checkout --` it. The corpus-DB reclassify (`curate`
115 reclassify-licenses --apply` + `curate export`) is left for the OWNER to run deliberately (binary SoR data).

116 - Plan doc (reference): `scratch/HIGH\_RISK\_REMEDIATION\_PLAN.md` (untracked). Audit JSON: `?\tasks\wf38bovts.output`
117 (batch 1) + `?\tasks\wa4d2980l.output` (full 8 auditors).

118

119 ****Checkpoint:**** 2026-06-10 (compiler SyntaxError FIX ? ****PR #63 MERGED****, squash `master 13d4573`; owner approved merge-on-green) .

120 ? ****master UNBROKEN** again ? fixed the ? SyntaxError from 7fbb0b0** (= the audit's fix chip `task\_2b7a19d4` work):

121 restored the f-string quotes on the 7 mangled lines of
`backend/pipeline/stitcher/compiler.py`
122 (100/104/156/160/181/191/213; message texts verified against the pre-conversion file at `7fbb0b0`; line 191 =
123 plain string, no placeholders) + fixed line 168 `f"\nSUMMARY]" ? `f"\n[SUMMARY]`. ****NEW** regression guard
124 `tests/test\_import\_smoke.py`******: (a) py_compile sweep over every file in backend/ (imports nothing ? runs on
125 cv2-free machines), (b) subprocess `import backend.main` with missing cv2/torch/numpy stubbed. Negative check:
126 the broken compiler.py fails the sweep (`SyntaxError: invalid decimal literal`, line 100).

127 ? ****CI NOW EXISTS** ? the audit's no-CI finding is CLOSED** (`github/workflows/ci.yml`, runs on every PR + master
128 push): job 1 ****import-smoke**** = smoke tests on a GPU-stack-free runner (minimal dep set verified empirically in a
129 fresh venv: pytest/fastapi/uvicorn/pydantic/python-dotenv/****google-genai**** ? needed at import time by
130 backend.providers); job 2 ****full-suite**** = requirements.txt with ****CPU-only torch wheels****
131 (`--index-url download.pytorch.org/whl/cpu`) + libgl for opencv ? `python run\_all\_tests.py` (obsreport absent ?
132 reporting tests skip by design). ****Both jobs GREEN** on the first run** (27s / 1m29s). Verified post-merge:
133 `import backend.main` OK on master, suite 256 green (254+2).

134 ?? Audit's ****coverage-inversion**** finding (wasb_hybrid/validators untested) remains open ? CI only guards the
135 import/syntax class + whatever the suite covers.

136

137 ****Checkpoint:**** 2026-06-10 (DEEP AUDIT session ? ultracode 18-agent audit `wf\_2a64bc88-2a8` over indexer+collector:
138 engineering/testing/reliability/docs-consistency/PMF; 3 of 8 auditors completed+verified, 5 rate-limited).

139 - ? ****CRITICAL** ? master is BROKEN since 7fbb0b0 (2026-06-07 print?logger sweep):
140 `backend/pipeline/stitcher/compiler.py` has ****7 unquoted f-string lines** (100/104/156/160/181/191/213)****** ? SyntaxError ?
141 `import backend.main` fails ? FastAPI server + `/api/compile` dead. Triple-verified + reproduced first-hand
142 (`py\_compile` exit 1). The ****254-green suite never noticed**** (zero tests import backend.main / compiler / wasb_hybrid /
143 motion / any concrete validator; ****no CI exists at all****). Spawned fix chip `task\_2b7a19d4` (fix + import/compileall
144 smoke test). Coverage is INVERTED vs production risk (tracknet twin 8 tests vs live wasb_hybrid 0; TrackNetRunner 21
145 vs WasbRunner 1). Suite itself healthy where it exists (254 green / 19.9s / faithful mocks / low flake risk); "fully
146 mocked runs anywhere" is false on cv2-free machines (bare `import cv2` at collection time, only obsreport importorskip).

147 - ****Doc drift** (verified, one-directional ? newest decisions not back-propagated):****** (1) licensing "Apache-2.0 only" still
148 in CLAUDE.md:33 / PLATFORM:269,301 / COMMERCIALIZATION C14 vs ratified ****MIT/Apache/BSD**** (a literal CLAUDE.md reader

149 rejects the plan's own MIT ActionFormer/ASFormer heads); (2) **Gemma-4** still
 "approved" in those same docs vs
 150 AMBER/bench-only in the plan; (3) **M0.3** purge pointer **WRONG** ?
 `training.label\_candidates` is a pure GT-agnostic fn;
 151 the live Gemini-silver training violation = `backend/eval/distill\_local.py` (+
 calibrate_local thresholds) + gitignored
 152 `output/local\_rally\_model.json`; (4) **C3** escape hatch broken ? "train-own weights
 (Phase-4 roadmap)" cites a phase
 153 that contains no such plan and ADR-002 says corpus labels are temporal-only (CAN'T
 train a detector) ? own-weights is
 154 an unscoped new workstream (coordinate labels or detector swap); (5)
 OWNED_MODEL_IMPLEMENTATION_PLAN + NEXT_STEPS are
 155 linked from NO entry point (CLAUDE.md "read first" / docs/README index) and NEXT_STEPS
 is branch-only; (6) README
 156 "yolo_hybrid by default" vs config.json `gemini` vs CODE_MAP "wasb = LIVE default";
 (7) ROADMAP still says audio
 157 "gated on golden labels" (trigger now satisfied!) vs plan "parked, not in reserve";
 (8) Roora multisport CSVs live ONLY
 158 in `%TEMP%` (OS-clearable) + n=6 manifest gitignored ? headline
 TT-0.909/pickleball-0.308 evidence is one cleanup from
 159 loss; (9) TT-on-WASB idea silently walks into the C3 ship-gate ("multiplies per
 sport"); (10) ship-gate F1@tIoU still
 160 undefined while 0.44/0.54/0.480 floors circulate; repo-rename "ASAP" tracked in no
 tracker (BACKLOG lacks it).
 161 - **PMF** (web-grounded, sources in audit output): **pricing/cost benchmarks VERIFIED**
 (SwingVision \$179.99/yr ~20k paying
 162 subs rev \$2.75M +12% YoY; PB Vision \$99/yr + **\$8/hr-of-video partner API**; Gemini
 COGS ~\$0.30?0.60/hr ? fat margin);
 163 but **"India badminton whitespace" FALSIFIED** ? Bangalore is the most contested
 badminton-AI city: Machaxi (\$1.5M
 164 Rainmatter+Prakash Padukone), VISIST.AI (70+ Bengaluru academies, Padukone School),
 Game Theory (Gopichand-backed smart
 165 courts w/ auto-highlights), Stupa (\$3.3M, BWF-approved, TT?badminton) ? NONE in our
 2026-06-07 competitive table.
 166 Surviving wedge (narrow, real): **software-only rally-precise indexing of EXISTING**
messy multi-court BYO footage
 167 (market solves it only with per-court hardware) sold per-hour as engine/API, possibly
 THROUGH those players
 168 (PB Vision×PodPlay template). Fastest falsification: C13 interviews (still undone) +
Gemini-wrapper baseline vs the
 169 n=6 LOVO corpus (if wrapper ? learned 0.583, owned-model is premature) + 1 paid
 academy pilot.
 170 - **AUDIT NOW COMPLETE** (8/8 auditors, resumed `wa4d2980l` after Max-20x refresh; 40
 agents). **New high/critical findings**
 171 beyond the first batch:
 172 - ? **CRITICAL** (collector) ? **LICENSING GUARDRAIL VIOLATION**: **`src/cli/ingest.py:12`**
 defaults **--license MIT**; the
 173 ShuttleSet **annotation** license is stamped onto **YouTube broadcast video** (e.g.
 shuttleset_21 = a YONEX Thailand
 174 Open broadcast, **is_commercial=true, license_type=MIT**). **24 videos / 1915 segments**
 currently exported as
 175 "commercially-safe" on unsound grounds ? directly violates the "training data must
 be commercial-clean" guardrail.
 176 - **HIGH** (indexer, all verified): (a) **nested config sections** silently drop unknown
 keys ? only `AppConfig` has
 177 `extra='allow'`, so the ENTIRE `indexing.wasb` block
 (weights/conda_env/velocity_threshold) + `yolo\_prefetch\_workers`
 178 + `debug\_duration` are unconfigurable via config.json; meanwhile
 `yolo\_parallel\_chunks/yolo\_chunk\_workers` are read
 179 nowhere. (b) **detector artifacts** keyed by file BASENAME not video_id ? two
 `match.mp4` files reuse stale frames ?
 180 a trajectory + candidate_segments computed from the **WRONG** video (silent training-
 data poisoning). (c) **re-ingest** is
 181 **destroy-before-confirm** ? **add_video('processed'/'failed')** **INSERT-OR-REPLACE**
 cascade-deletes prior good segments

```

182     BEFORE the new run; a failed/empty re-run leaves zero. (d) **stitching padding
config not wired? API reels always
183     end_padding=1.0 despite config 0.5. (e) **`/api/process` arbitrary server-file read
+ `/api/compile` path-traversal/
184     absolute write? via output_filename. (f) **run_regression "CI gate" not in CI +
silently passes? on missing baseline;
185     baseline omits iou/detector/GT-hash. (g) **metrics bug?: `match_intervals` at
iou_threshold=0.0 matches
186     non-overlapping intervals (P=R=F1=1.0 for garbage); **rally_seq_proto AUC has no tie
handling? (order-dependent;
187     all-zero dead-time rows guarantee ties) + speed features fps-inconsistent within one
vector.
188     - **HIGH (collector):? silent DELETE-then-INSERT replace of golden labels (no
warn/backup, filename-derived video_id);
189     validate-delivery **PASSES on a zero-rally CVAT export?; not yet SoR (no
provenance/maturity/labeled_window cols,
190     import-local hardcodes CURATED); `int(float())` rally_number crashes mid-import
leaving a committed video row w/
191     rolled-back annotations (normalize_annotations guard is opt-in, NOT wired into
import-local).
192     - **MEDIUM batch:? WSL `timeout_sec=0` = hang-forever (both runners + provider
PROCESSING poll); `uvicorn backend.main:app`
193     leaves globals None (init only in main()); SQLite no WAL/no busy_timeout + swallow-
on-lock; chunk_overlap?duration =
194     infinite loop; WSL temp/frame/cache + Gemini remote-file leaks; CORS
`*`+credentials; sport-profile relevance config
195     dead under typed config (bites first non-badminton sport ? the current branch!).
196     - **VERDICT NUANCE:? verifier DOWNGRADED 2 "high"?not-real (Result-contract gap + DB-
swallow are DOCUMENTED CODE_MAP
197     gotchas/deliberate-incremental, medium at most; DB "no busy_timeout" claim factually
wrong ? sqlite3 default
198     timeout=5s). Eng-pipeline/eval/collector strengths confirmed: registry seams,
DetectorRunner ABC, WASB crash-cache,
199     honest fit-on-self labeling, leakage-free LOVO, frame-true CVAT conversion all
genuinely good.
200     - ? ? **SPOOF-CHECK PASSED ? quality iterations are REAL (re-ran the evals first-hand on
this box, pure numpy/scipy):?
201     all 6 videos + 6 human label CSVs + 6 trajectories exist; traj row-counts match docs
exactly. **`rally_seq_proto` LOVO
202     reproduced: mean 0.583? (AUC 0.877/0.851/0.916/0.832/0.730/0.902 vs doc
0.879/0.853/0.915/0.833/0.726/0.902).
203     **`calibrate_local` heuristic LOVO reproduced: 0.480 ± 0.242? (per-fold Adarsh
0.751/Kushagra 0.157/doubles 0.737/
204     gBaaddy 0.309/testlarge 0.267/mahadevpura 0.657 ? exact). **Contrast metric recomputed
independently from raw CSVs:?
205     3.7/1.4/3.7/1.9/1.4/2.2× vs doc 3.9/1.4/3.6/1.9/1.4/2.2. Opus 4.8 did NOT spoof. ? one
nit: gBaaddy CSV has 48 rallies
206     (doc says 47). Caveat: trajectories were WASB-GPU-generated (can't regenerate here)
but sizes/frame-counts/visibility all sane.
207     - **HIGH-RISK REMEDIATION PLAN written for owner approval:?
`scratch/HIGH_RISK_REMEDIATION_PLAN.md` (12 PRs, sequenced;
208     PR-1 compiler fix already DONE via #63). Awaiting owner scope + push/PR decision
before execution. Full audit JSON:
209     `%LOCALAPPDATA%\Temp\claude\...\tasks\wa4d2980l.output` (8 auditors) +
`wf38bovts.output` (first batch).
210     Full audit JSON: `%LOCALAPPDATA%\Temp\claude\C--Users-avidu-Projects-badminton-
highlight-indexer\45ea3a91-654d-4972-9cd3-65f35db54caf\tasks\wf38bovts.output`.
211
212     **Checkpoint:? 2026-06-09 (Roora MULTISPORT recon ? for FRESH next session) . ??
**Pickleball + Tabletennis golden labels
213     arrived; INGESTION = SOLVED, but the "merged model" experiment is BLOCKED on per-sport
trajectory detectors (validates the
214     roadmap blocker).? Owner left a recon-only ask (deliberately did NOT drain context /
spin heavy jobs ? owner wants me fresh).
215     - **Deliverables:? `Annotation Setup/6 June 2026/{pickleball,tabletennis}/*.zip` (3

```

CVAT export FORMATS each: COCO

```
216     `instances_default.json`, **CVAT-XML `annotations.xml`** ? use this, Pascal-VOC).
Videos: `?/6 June 2026/Annotation Videos/
217     Video 2 - Pickleball.mp4` (4K, 59.94fps, 97s, 5737fr) + `Video 3 - Tabletennis.mp4`
(4K, 59.94fps, 58s, 3490fr). Short clips.
218     - **What Roora annotated:** label class "rally" as **per-frame BOUNDING BOXES on sampled
frames (~every 30fr)** ? box geometry
219     is just a CARRIER; the boxes carry the FULL rally schema as CVAT **attributes**
(rally_number, start/end time, shot_count,
220     score_before/after(A-B), ending_reason, last_shot_outcome [+note for TT]). 456/513
attr instances. Boxed frames cluster into
221     rallies (runs of ~30-spaced frames = a rally; big gaps = dead time).
222     - **INGESTION = YES, cleanly:** the collector's `src/parsers/cvat/rally_cvat.py`
(`curate convert-cvat`) is PURPOSE-BUILT for
223     exactly this ? ignores box geometry, **recomputes rally interval from the frame
range** (`sec=frame/fps` at true fps, ignoring
224     vendor start/end times which are unreliable), groups boxes by rally_number, maps
attrs?canonical. Minor: verify the attr-name
225     mapping handles spaces/"(A-B)" (`ATTR_MAP`). Collector already has pickleball +
tabletennis schemas.
226     - ? **"MERGED LEARNT MODEL" across sports is BLOCKED:** `rally_seq_proto` features come
from a WASB **shuttle** trajectory =
227     BADMINTON-ONLY; pickleball (wiffle ball) + TT (tiny fast ball on a table) are out-of-
distribution ? no clean trajectory yet.
228     This EMPIRICALLY validates the roadmap's binding multisport blocker (clean per-sport
MIT/Apache ball detectors not identified).
229     - **PROGRESS (owner asked to complete 1/2/3 before badminton):**
230     - ? **{(1) convert-cvat DONE ? INGESTION PROVEN.** `python -m src.cli.curate convert-
cvat --input <annotations.xml> --out
231     <csv> --fps 59.94` ? **pickleball 6 rallies + tabletennis 6 rallies**, sane
intervals, attrs mapped. CSVs at
232     `%TEMP%\roora\{pickleball,tabletennis}_rallies.csv` (+ extracted XMLs
`%TEMP%\roora\{pb,tt}\annotations.xml`). ? Roora data
233     nits (non-blocking): pickleball `last_shot_outcome='0'` (off-vocab in/n_a/net/out);
TT `ending_reason` mostly 'other' with the
234     real outcome in `notes` ("out of table"/"net") ? ending_reason taxonomy mapping
could improve. Both clips SHORT (97s/58s).
235     - ? **{(2) WASB-transfer test DONE (`%TEMP%\{pickleball,tabletennis}_traj.csv`; log
`~/clips/multisport_run.log`):** WASB
236     (badminton shuttle detector) detects SOMETHING on both ? **pickleball 28% /
tabletennis 39% visibility** (NOT near-zero;
237     comparable to badminton 40?67%). So it's not blind.
238     - ? **{(3) DONE ? SURPRISE RESULT (directional, n=6 each, TINY so noisy):**
`eval_partial_labels` on (WASB-transfer traj + Roora
239     6-rally CSV, fps 59.94, fw 1920): **TABLE TENNIS best F1 0.909** (tuned+gate2:
P1.0/R0.833, 5/6 matched, boundary err 0.49s ?
240     **WASB transfers to TT remarkably well!); **PICKLEBALL best F1 0.308** (poor ?
wiffle ball too out-of-distribution).
241     ? **NUANCES the roadmap blocker: per-sport detector transfer is SPORT-DEPENDENT ? TT
may be servable with WASB short-term;
242     pickleball needs a proper detector.** ? BIG caveat: 6 rallies each = could be partly
luck ? owner sending more videos to
243     confirm. A "merged" prototype LOVO across badminton+pickleball+TT is the natural
next experiment once n grows per sport.
244     Roora LABELS ingest cleanly + are corpus-gold regardless. **Great honest surprise to
greet owner with.**
245
246     **Checkpoint:** 2026-06-09 (PR #61 MERGED ? roadmap SET, session wrap) . ? **Owner
agreed + merged #61 (`master 4d04a73`)).
247     ALL CLEAN: master, 0 uncommitted, 0 open PRs, branches pruned.** The owned-model
**ROADMAP is now set + agreed** ?
248     authoritative doc = **`docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md`** (full verified
decisions in the "(PR review)" checkpoint
249     just below: ms-swift + Qwen3-VL, TAL-head-first, conversational-QA seam-now-model-later,
multisport one-head/base, Python
```


250 everywhere, MIT/Apache/BSD ratified, "envision now build later"). [[session-handoff]]
rewritten to this phase.

251 - ?? ****NEXT SESSION** (awaiting owner greenlight ? NO build until then):** greenlight a
Phase-0 milestone (rec ****M0.1**

252 corpus+event-index** ? ****M0.4 Gen-0 TAL harness****, both \$0/local); confirm ****collector**
= system-of-record** ; define the

253 ****ship-gate F1@tIOU target**** (floor = beat heuristic LOVO 0.480); ****rename repo off**
"badminton**"; M0.3 purge Gemini

254 training labels + WASB C3 (own-weights/clean swap). Corpus = n=6
(`output/human\_lovo\_manifest.json`); learned prototype

255 is the Gen-0 seed. Committed next PLATFORM slice stays the async job model (single-
tenant). Owner flagged context ~85% ? this is the wrap checkpoint.

256

257 ****Checkpoint:**** 2026-06-09 (PR review) · ? ****OWNER REVIEWED PR #61 ? REFRAMED ROADMAP**
(important checkpoint).** Owner left

258 ~12 inline comments (mostly state-updates + recommendations wanted). ****Key roadmap**
shifts to internalize:**

259 - ****Immediate goal REFRAMED:**** not "on-device model" but a ****TRAINING FRAMEWORK +**
HARNESS to produce a VERY HIGH

260 precision/recall model that extracts HIGHLIGHTS + RALLIES from a clip.** Quality
first. On-device = LATER (compression

261 for power users to save upload cost), definitely on roadmap but not now.

262 - ****MULTISPORT FROM THE START**** ? all net-separated racquet sports ? eventually all
sports; a SINGLE sports-agnostic model.

263 Collector + indexer already sport-generic by design. ? ****ACTION:** rename the repo off
"badminton" ASAP** (owner emphatic).

264 - ****CONVERSATIONAL VIDEO-QA = north-star feature:**** upload video ? ask contextual
questions ("longest rally", "make a clip

265 of all service faults", "rally >20 shots"). Needs per-video STATE/bookkeeping ? design
the framework for it. Owner asks:

266 embark now or punt + revisit after quality? (My lean: design the bookkeeping/event-
store layer now, punt the heavy

267 conversational model ? pending the strategy workflow.)

268 - ****LICENSING GUARDRAIL** (owner: "never corrupted"): ****EVERY** dependency ? code, data, AND
weights ? must be usable in

269 PRIVATE PROPRIETARY COMMERCIAL software. ****MIT / Apache-2.0 / BSD only.**** Ratify (was
"Apache-2.0 only"). Enforce religiously.

270 - Owner wants a ****coherent recommendation: local Gemma 4 vs ms-swift starting point****
(multisport, single, conversational,

271 commercial). NB: ms-swift = training FRAMEWORK, Gemma 4 = BASE model ? orthogonal;
clarify in the reply.

272 - Owner: ****upgrade WASB C3 (own-weights/provenance) to a first-class action item now.****
M0.2 (grow n) = practically DONE

273 (n=6); owner will add 1 more varied multi-court-club video soon ? Phase-0 ready to
start.

274 - ****LANGUAGE QUESTION** (owner callout, no preference):** is Python right for the upcoming
work, or brainstorm alternatives?

275 - ? ****STRATEGY WORKFLOW** done (`wf\_f9aa1d1c-c83`, 10 agents, verified+critiqued) ?
VERIFIED DECISIONS (now in the rewritten

276 `docs/OWNED\_MODEL\_IMPLEMENTATION\_PLAN.md`, the authoritative roadmap; pushed to PR #61
+ a consolidated PR comment reply):**

277 - ****Framework = ms-swift (Apache-2.0)**** (only forge w/ native video+audio+timestamp-
grounding+video-GRPO); unsloth = 8GB-local

278 SFT/compression tool (avoid its AGPL Studio). ****Base = Qwen3-VL (Apache-2.0,**
timestamp emission)** primary; ****Gemma-4 STAYS**

279 **AMBER/bench-only**** (critic corrected the workflow's "clear AMBER" ? posture
unconfirmed + 60s/no-timestamp dispute);

280 InternVL3 (MIT) fallback. ms-swift?Gemma4 (framework vs base ? clarified for owner).

281 - ****START WITH THE TAL HEAD, NOT THE VLM**** (VLMS miss strict temporal boundaries;
boundary accuracy = the product; head

282 de-risked AUC 0.83?0.92, \$0 local, doubles as teacher+baseline). Head now ? VLM
later (two-stage: VLM proposes, head refines).

283 - ****Conversational-QA: build the SEAM now** (per-video event store + structured text-to-
SQL + ffmpeg clip-cut), PUNT the NL

284 model** until quality clears the bar. Queries ("longest rally"/"service faults"/">20

shots") = structured-numeric ? SQL, not

285 a VLM. Reserve fault/event taxonomy in M0.1. ? real DB delta (new cols + a
`highlights` table; play_segments lacks them today).

286 - **Multisport: one shared head + one base; scale DATA + per-sport clean trajectory
detectors (the binding blocker ? none

287 identified for tennis/TT/pickleball/padel yet); WASB C3 multiplies. Single-model
temporal generalization = OPEN HYPOTHESIS

288 (RacketVision evidence REFUTED = ball-tracking not temporal). Repo rename off
"badminton" = action item.**

289 - **Language: Python everywhere** (training Python-locked; platform I/O-bound, no hot
path ? reject Go/Rust split); on-device =

290 deferred EXPORTED runtime shell (ONNX/CoreML/GGUF), doesn't dictate dev language.
Constraint NOW: **train export-clean ops**.

291 - **Honest gaps:** cost-to-Gen-2 UNQUANTIFIED (biggest risk); ship-gate "very high
P/R" target undefined (floor = beat heuristic

292 LOVO 0.480 ? set explicit F1@tIoU); per-sport detectors TBD; base PRETRAINING
provenance unauditable ? owner risk-acceptance;

293 **Gemini-silver TRAINING labels = LIVE violation ? hard-blocker purge M0.3.**

294 - **Owner principle recorded: "envision now, build later"*** ? design every deferred
seam (event store, export boundary, sport-

295 agnostic head) up front; punt the feature, never the seam.

296 - ?? **NEXT:** owner reviews the reframed plan + decisions (greenlight a Phase-0
milestone ? rec M0.1 corpus+event-index then M0.4

297 harness; confirm collector=system-of-record). Then start the greenlit milestone. Repo
rename. Define the ship-gate F1@tIoU target.

298

299 **Checkpoint:** 2026-06-09 . ? **n=6: MahadevpuraSingles added while owner reviews PR
#61.** Owner moved ALL golden videos

300 + CSVs to `Annotation Setup/Collect/Golden Labelled/` (manifest label paths updated;
trajectories unaffected in `%TEMP%`).

301 New: **MahadevpuraSingles** (singles, 1080p, 29.97fps, 31 rallies; traj
`%TEMP%\mahadevpura\_traj.csv`, script

302 `run\_wasb\_mahadevpura.sh`). **Borderline multi-court (contrast 2.2x)** ? heuristic best
0.667, learned held-out **AUC

303 0.902/F1 0.712** (clean generalization to an unseen match). **n=6 LOVO: heuristic 0.480
± 0.242 vs learned 0.583** (learned

304 holds ~+0.10 since n=3). Contrast metric now predicts heuristic F1 MONOTONICALLY across
5 matches (Mahadevpura's 2.2x?0.67

305 lands on the curve). **Decoupling headline: learned held-out AUC 0.83?0.92 regardless of
contrast; heuristic swings 0.16?0.75

306 with it.** Committed to PR #61 (n=6 doc update). ? GOTCHA: don't combine inner `&` with
run_in_background (orphans the job);

307 calibrate_local buffers output to end + is slow (~17?20min). "Ingested" = added to EVAL
corpus (manifest); collector

308 import-local (system-of-record) still = M0.1, plan-only pending greenlight. ?? owner
reviewing PR #61 (4 plan decisions pending).

309

310 **Checkpoint:** 2026-06-08 (later+4, SESSION WRAP) . ? **n=5 corpus + MULTI-COURT root
cause + heuristic discovery + SINGLE PR #61 for offline review.**

311 Owner tagged gBaaddy (47, 1080p, multi-court) + TestLargeVideo (6, short) ? corpus
n=5 (`output/human\_lovo\_manifest.json`).

312 - **HEURISTIC DISCOVERY (user asked "discover more heuristics like audio"):** tested 2
label-free trajectory pre-filters

313 for multi-court (measured on all 5): foreground spatial-density gate = **net ?0.04**
(helps multi-court +0.03, hurts clean

314 ?0.12); track-continuity/jump filter = **no-op**. ? simple single-cue trajectory
heuristics CAN'T isolate the foreground

315 court. **AUDIO-foreground idea PROTOTYPED (no more videos needed) ? WEAK ?:** on the 2
multi-court videos + clean

316 control, audio onset/RMS energy AUC only **0.53?0.65** for in-rally vs dead-time; HF
bands (8?22kHz @48kHz) raise

317 CONTRAST (Kushagra 1.1?1.7x, gBaaddy 1.3?1.9x, confirming distance-attenuation
physics) but AUC stays ~0.6. Fails:

318 dead-time isn't quiet ? foreground court loud during dead-time too
(footwork/talk/pickup) + gym reverb ? audio loudness

319 ? court occupancy, not rally (same wall as E1 ~2.2x). **Audio alone ? multi-court
 gate.** Remaining heuristic lever =
 320 real court-region detection (player-cluster/court-line, ROADMAP #2); robust path = the
 learned visual model (AUC 0.85).
 321 All tested + corrected in PR #61 doc + a PR comment.
 322 - **COURT-REGION DETECTION SMOKE-TESTED ? LOW HEADROOM ?:** only 12?14% of dead-time
 detections on the multi-court videos
 323 fall OUTSIDE the foreground rally region (86?88% INSIDE) ? a perfect court gate lifts
 contrast only ~1.6?2.2x (still
 324 failing). The confusion is the foreground court's OWN between-point activity, NOT
 separable background courts. So court
 325 detection (heuristic OR own model) won't help. **3 heuristics now tested + FAILED
 (spatial gate, audio, court
 326 separability) ? all because it's a TEMPORAL rally-vs-non-rally problem in the same
 space; only the learned model solves
 327 it (AUC 0.83). NO cheap heuristic shortcut ? the learned segmenter is the path.** (In
 PR #61 doc + PR comment.)
 328 - ? **SINGLE PR #61** (<https://github.com/avidullu/badminton-highlight-indexer/pull/61>)
 carries EVERYTHING for offline
 329 review: rally_seq_proto + tests, owned-model STUDY + PLAN (4 decisions pending),
 multivideo-lovo learnings (n=5 + multi-court
 330 + heuristic discovery), CODE_MAP. Title/desc updated. Plan doc NOW committed (was
 held). Temp experiment scripts deleted.
 331 (#60 + collector #13 merged earlier; #61 = only open PR.)
 332 - **? DEFINING FINDING (owner's insight, data-confirmed): heuristic fails on MULTI-COURT
 footage.** gBaaddy (1080p like
 333 Adarsh) heuristic best F1 **0.277** ; Kushagra 0.163 ? both have BACKGROUND GAMES. WASB
 tracks every shuttle incl. other
 334 courts ? high dead-time visibility (gBaaddy 41%, Kushagra 57% vs clean Adarsh 18% /
 doubles 22%). ***"Contrast" metric**
 335 (in-rally+dead-time vis) predicts heuristic success: ?3.6x works, ?1.9x fails. Academy
 = multi-court = TARGET ENV ? these
 336 are the most representative videos. **Lever = foreground-court isolation** (spatial
 gate to prominent court; needs real
 337 court-region detection, naive centroid muddled by foreground-shuttle-resting-low).
 NEAR-TERM, no new model.
 338 - **Learned prototype LOVO (n=5):** per-frame AUC 0.83?0.92 on every match; **recovers
 BOTH multi-court videos**
 339 (Kushagra 0.561, gBaaddy 0.574 vs heuristic 0.16/0.28). Kushagra climbs
 0.049?0.417?0.561 with n. Mean 0.568 ? dipped
 340 from 0.615 (n=3) ONLY due to the 6-rally TestLarge noise fold; **excl. it = 0.639**.
 Per-video gap on hard videos is the signal.
 341 - **Heuristic LOVO n=5 = 0.463 ± 0.247** (per-fold: Adarsh 0.751, doubles 0.758,
 Kushagra 0.157, gBaaddy 0.295, testlarge
 342 0.353). **FELL from n=3's 0.544** as multi-court videos entered ? the heuristic can't
 represent the target env, so a more
 343 representative corpus LOWERS its honest score. vs **learned 0.568** ? gap INVERTED
 (heuristic led at n=2 +0.066; learned
 344 leads at n=5 **+0.105** and widening). (b8jpxbryo was just slow, ~17min; completed
 fine.)
 345 - **SESSION WRAP shipped ? PR #61** (branch feat/owned-model-study-and-prototype): n=5 +
 multi-court finding appended to
 346 `docs/archives/quality-iterations/2026-06-multivideo-lovo/`; **session-handoff.md
 REWRITTEN** (quality-iteration phase);
 347 **CODE_MAP updated** (eval_partial_labels + rally_seq_proto + multi-court gotcha).
 Owned model still PLAN-ONLY
 348 (`docs/OWNED\_MODEL\_IMPLEMENTATION\_PLAN.md` DRAFT, 4 decisions pending).
 349 - ?? **NEXT (tomorrow): owner tags more varied videos** ? re-run LOVO+prototype.
 Foreground-court isolation experiment
 350 = #1 lever. Owner reviews plan + greenlights a Phase-0 milestone (rec M0.1 corpus
 spine). Commit plan doc on approval.
 351
 352 **Checkpoint:** 2026-06-08 (later+3) . ? **n=3 CROSSOVER ? learned model OVERTAKES the
 heuristic (the data-backed green
 353 light).** User tagged a 3rd distinct match **LargeTestVideo (DOUBLES, 5.3K?1920,

```

29.97fps, 49 rallies)** ? corpus now
354     n=3 (2 singles + 1 doubles, 3 cameras/frame-rates). Trajectory
`~/clips/largetestvideo_traj.csv` (script
355     `run_wasb_largetestvideo.sh`). Manifest `output/human_lovo_manifest.json` now n=3.
356     - **Single-video LargeTest (doubles):** heuristic best F1 **0.750** (tuned, R0.918),
best-fit 0.827 ? **doubles is
357     heuristic-FRIENDLY; Kushagra remains the LONE heuristic failure** (not a play-type
issue).
358     - **Honest LOVO, heuristic vs learned, n=2?n=3:** heuristic mean held-out **0.443?0.544
± 0.278** (Adarsh 0.725,
359     **Kushagra 0.151 capped at any n**, doubles 0.755). Learned prototype mean
**0.377?0.615** (Adarsh F1 0.701/AUC 0.878,
360     **Kushagra 0.049?0.417**/AUC 0.850, **doubles 0.729/AUC 0.914** trained only on the 2
singles). **? The learned model
361     OVERTAKES the heuristic at n=3 (0.615 vs 0.544); at n=2 it trailed (0.377 vs 0.443).**
Crossover predicted by the plan.
362     - **The story (honest):** learned path SCALES with data (+0.24 for one match,
calibration gap closing ? Kushagra
363     0.049?0.417) and GENERALIZES across play type (doubles held-out AUC 0.914); heuristic
is HARD-CAPPED on footage it
364     can't represent (Kushagra ~0.15 at any n). Honest "bar to beat" is now ~0.54 (n=3),
not the fit-on-self 0.73. ?
365     Reusing every labeled video for training validated; bottleneck = data scale +
calibration (n?5), not method.
366     - **Docs updated ? pushed to indexer PR #61** (branch feat/owned-model-study-and-
prototype): n=3 findings + final quality
367     metrics + conclusions in `docs/archives/quality-iterations/2026-06-multivideo-lovo/`;
study + index updated. (PR #60
368     MERGED earlier; #13 MERGED.) ? Prototype speed features NOT yet fps-normalized (30 vs
60fps) yet doubles still
369     generalized (AUC 0.914) ? known refinement.
370     - **OWNED-MODEL: PLAN-ONLY per owner** ? `docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md`
written (DRAFT, uncommitted, for
371     review). User chose "just plan, don't code yet." 4 decisions pending (which Phase-0
milestone to greenlight [rec M0.1
372     corpus spine], WASB C3 stance, ratify Apache-2.0/MIT, confirm collector = corpus
system-of-record). NO owned-model
373     code until greenlight. Momentum = tag more matches (n?5) + I score each.
374     - ?? **NEXT:** owner reviews plan + greenlights a Phase-0 milestone; tag more matches ?
re-run LOVO+prototype (expect
375     learned mean to keep rising). gBaaddy/TestLargeVideo still untagged test videos
(trajectories ready). Commit the plan
376     doc on approval.
377
378     **Checkpoint:** 2026-06-08 (later+2) . ? **Multi-video eval corpus + owned-model
research underway.** User fully tagged
379     Adarsh-and-Avi (40?96 rallies**) + provided **Kushagra** (32 rallies, 4K) for the
TRAINING mix, and **gBaaddy**
380     (new, 1080p, untagged) + TestLargeVideo as held-out TEST videos. Generated full-video
WASB trajectories (single 8GB
381     GPU, serialized via wait-loops): `~/clips/{adarsh_avi_full,kushagra,gbaaddy}_traj.csv`
(+ %TEMP% copies). Scripts:
382     `~/clips/run_wasb_{avi_full,kushagra,gbaaddy}.sh`.
383     - **Adarsh-full eval (96 rallies, IoU?0.5, window [0,1405.8s]):** baseline F1 0.632,
**tuned+gate2 0.727** (P0.80/R0.67,
384     best out-of-box), sweep **ceiling 0.750** (vel0.02/merge1.0/min_dur2.0/gate off, fit-
on-self). vs the opening-10-min
385     (40 rallies): best 0.667 / ceiling 0.742. **Best-config REGION is identical across the
slice and full match**
386     (merge_gap 1.0, min_dur 2.0; velocity irrelevant; gate-off=recall, gate-on=precision)
? early sign tuning may
387     transfer; LOVO is the real test. Per-rally: `output/adarsh_avi_full_vs_human.csv`.
388     - ? **KEY FINDING ? Kushagra: WASB tracks shuttle BETTER than Adarsh (97% of GT rallies
have ?? net-crossings, med 5,
389     310 pts/rally vs Adarsh 76%) BUT no velocity-windowing config can segment it (per-

```

video best-fit F1 only 0.163; baseline
390 0.000; mean boundary err ~5s).** Signal is PRESENT; the unsupervised heuristic FAILS ?
likely because Kushagra's shuttle
391 is visible 67.5% (vs Adarsh 40%), i.e. tracked through inter-rally activity ?
boundaries bleed. Root cause is the
392 heuristic class, NOT WASB recall or labels or tuning.
393 - **First cross-video LOVO (`calibrate\_local`, Adarsh-full 96 + Kushagra 32, human
labels, manifest
394 `output/human\_lovo\_manifest.json`):** baseline mean held-out F1 **0.316** ? calibrated
0.443 ± 0.279 (overfit gap
395 +0.027, tiny). Per-fold: held-out **adarsh 0.722** (cfg tuned on Kushagra ? Adarsh's
own 0.777 ? config TRANSFERS great),
396 held-out **kushagra 0.163** (cfg tuned on Adarsh; = Kushagra's own ceiling).
Asymmetry is the story: tuning transfers
397 fine WHERE the heuristic can solve the video; Kushagra is fundamentally unsegmentable
by velocity-windowing (ceiling
398 0.16) regardless of config. The ±0.279 std = method variance, not config. ?
**definitive, data-backed case for the
399 learned OWNED model** (trajectory carries the signal; heuristic can't exploit it).
Kushagra labels+traj = valuable
400 TRAINING data even though the current pipeline can't use them.
401 - ? LOVO honesty: 1 entry per DISTINCT MATCH (group Adarsh-main + Adarsh-last-game as
one unit; calibrate_local has no
402 grouping ? keep matches as separate entries only). gBaaddy traj still inferring (test
video).
403 - ? **PROTOTYPE built (user asked) ? `backend/eval/rally\_seq\_proto.py` (+3 tests):**
learned per-frame TAS-style
404 segmenter over 6 hand-built WASB-trajectory features (vis-rate, speed, **net-crossing
rate**, spreads) ? pure-Python
405 LogisticRegression, honest LOVO. **Held-out per-frame AUC 0.878 (Adarsh) / 0.841
(Kushagra).** Kushagra: LOVO-threshold
406 F1 0.049 (threshold doesn't transfer at n=2) but **oracle-threshold F1 0.578 = 3.5×
the heuristic's 0.163 ceiling.**
407 ? PROVES the rally signal is LEARNABLE where the heuristic fails; gap = threshold
CALIBRATION not representation; answers
408 the study's "can low-dim WASB features feed a TAS head?" = YES. Seed of Gen-0.
409 - ? **OWNED-MODEL STUDY done (11-agent workflow `wf\_49aab888-e10`, 35 sources) ?
`docs/archives/research/OWNED\_MODEL\_TRAINING\_STUDY.md`.**
410 3 generations off ONE corpus: Gen-0 ASFormer(MIT)/ActionFormer over WASB features ?
Gen-1 cue fusion ? Gen-2 e2e VLM
411 (Qwen3-VL-4B / Qwen2.5-VL-7B / Gemma-4, ALL Apache-2.0; Gemma-3 REJECTED).
**Platforms: Modal primary (best written
412 no-access), RunPod-Secure fallback, Vertex on bench (India residency but tuning Gemini
? owned weights), 2070S =
413 Gen-0+inference only.** Label-reuse = 3 transcoders, split-by-match. Critic caveats:
**WASB checkpoint provenance C3
414 propagates** (MIT code ? checkpoint provenance), Gemma-4 stay AMBER until raw license
check, **purge Gemini-trained
415 distilled labels** to comply with eval-only policy. DESIGN-ONLY: no impl until P0-P4 +
healthy corpus + owner sign-off.
416 - ? **DOC CHECKPOINT shipped ? indexer PR #61** (branch feat/owned-model-study-and-
prototype): prototype + study +
417 `docs/archives/quality-iterations/2026-06-multivideo-lovo/` learnings + index README.
Full suite **254 green**.
418 - ? **gBaaddy test trajectory inferring** (started 13:13, ~16 min) ? ready for when user
tags gBaaddy.
419 - ?? **NEXT:** review/merge PR #61. User tags a held-out TEST match (gBaaddy/TestLarge)
? score calibrated config +
420 prototype on truly-unseen footage. **Highest-ROI: grow n?2?5 distinct matches** (small
Roora batch) ? honest LOVO +
421 per-video threshold calibration ? re-run prototype. Then (owner sign-off, post-
scaffold) corpus format + ASFormer Gen-0 A/B.
422
423 **Checkpoint:** 2026-06-08 (later+1) . ? **3 work items shipped (user said "go ahead
with all three").**

```

424 1. **Tuning sweep** ? added `--sweep` to `backend/eval/eval_partial_labels.py` (+test);
indexer branch `feat/eval-human-labels`
425 (commits 2c950f6 harness, 58d10cd sweep; NOT pushed/PR'd ? offer to PR). On Adarsh-
and-Avi: **F1 0.667?0.742**
426 (velocity 0.03 / merge_gap 1.0 / **min_dur 2.0** / gate OFF; P0.67/R0.83). Dominant
lever = min_window_duration;
427 gate over-prunes this camera; velocity ~irrelevant. ? fit-on-self (one video,
optimistic) ? real number = LOVO.
428 2. **Collector P0 normalizer + positioning doc ? PR #13**
(https://github.com/avidullu/sports-data-collector/pull/13,
429 branch `feat/manual-ingestion-normalizer`, pushed).
`src/cli/normalize_annotations.py` (+test) = standalone
430 pre-import guard: drops blank/?start/**negative-start** rows (else pydantic
validator/safe_float_required abort the
431 WHOLE import) + rennumbers fractional/dup rally_number 1..N by start (else
int(float()) PK-collides + DELETE-INSERT
432 destroys one). Also mm:ss?sec, pipe ending_reason?primary.
`docs/MANUAL_INGESTION_POSITIONING.md` = positioning +
433 schema map + P1/P2 roadmap. **Validated full chain on the REAL file vs a TEMP db**:
normalize 40?40 ? import-local
434 40 ? export 40-interval indexer CSV. Collector suite **124 green** (was 116).
435 3. **Adversarial review** (1 agent) caught a REAL hole pre-merge: negative start_time
was undefended (pydantic
436 `start_time must be non-negative` ? import abort) ? fixed (drop branch + test, commit
1fe1c24) + corrected 2 doc
437 inaccuracies (abort mechanism = pydantic validator not safe_float_required;
ending_reason is free-text not enum).
438 4. **Iteration doc archived + indexer PR'd** `docs/archives/quality-
iterations/2026-06-human-gt-tuning/` (STATUS
439 COMPLETED; results table, sweep, failure modes, bugs, infra gotchas, reproduce) +
updated the quality-iterations
440 index README + 2026-06-initial seed forward-pointer. **Indexer PR #60 OPEN**
441 (https://github.com/avidullu/badminton-highlight-indexer/pull/60; branch feat/eval-
human-labels =
442 harness 2c950f6 + sweep 58d10cd + docs; 5 files, +446/-1, full suite 251 green).
443 - **GOTCHA learned** import-local does NOT copy the video (records local_path); writes
obs report NEXT TO THE VIDEO
444 (clean those up after validation). DB path from `config/config.yaml` `database_path` ?
override via temp config for
445 side-effect-free validation (no `--config`/`--db` CLI flag exists).
446 - ? **BOTH PRs MERGED ? both repos CLEAN on master, no open PRs, branches deleted**
Indexer **#60** (squash
447 `90fb8b3`: eval_partial_labels harness + sweep + iteration doc). Collector **#13**
(squash `099cf67`: normalize_annotations
448 guard + MANUAL_INGESTION_POSITIONING.md). NOTE: user said "review is merged" while
live API showed both OPEN ? I
449 completed both squash merges + synced.
450 - ?? **NEXT** after user's 60-min tagging ? full-video re-run
(`backend/eval/eval_partial_labels.py`, window auto =
451 last labeled end) + (recommended) tag BREADTH across 3?4 videos for LOVO. P1 collector
follow-ups (labeled-window in
452 manifest, maturity ladder needs_review?confirmed?gold) when prioritized.
453
454 **Checkpoint** 2026-06-08 (later) . ? **FIRST human-GT eval landed + collector manual-
ingestion positioning done**
455 User did ~10 min of VLC rally-annotator tagging ? partial human CSV `Annotation
Setup/Collect/Adarsh and Avi.rallies.csv`
456 (40 rallies, covers 0?605s of a 23m29s video; schema
`rally_number,start_time,end_time,ending_reason,sport`).
457 - **Built `backend/eval/eval_partial_labels.py` (+test, 3 green) ? committed on branch
`feat/eval-human-labels` (commit
458 2c950f6, NOT pushed/PR'd)** Scores WASB rally segments vs a PARTIAL human CSV,
**restricted to the labeled window
459 [0, last_end]** so correct detections in the un-labeled tail aren't counted as FP (THE
key methodology for partial

```

```

460     labels). Reuses calibrate_wasb.make_predict_fn/load_golden_gts + rally_gate + metrics
+ export_wasb_segments.build_comparison.
461     - **Generated WASB trajectory for the labeled window** (first 610s ? 36,564 frames ?
`~/clips/adarsh_avi_traj.csv` +
462     Windows `%TEMP%\adarsh_avi_traj.csv`; WSL script `~/clips/run_wasb_avi.sh`; ~18 min
inference). ? GOTCHA: passing a
463     long multi-statement WSL command via the Bash tool with `run_in_background` MANGLES
`$VAR`/`$()` (empty expansion) ?
464     put the job in a .sh FILE (tr-strip CRLF into WSL) and background `bash file.sh`.
465     - **FIRST trustworthy human-GT numbers (IoU?0.5, 40 rallies in-window):** baseline F1
**0.650** (P/R 0.65/0.65);
466     tuned 0.623 (P0.50/R0.825, over-segments); **tuned+gate2 best F1 0.667**
(P0.79/R0.575). Boundary err ~1s.
467     Per-rally CSV `output/adarsh_avi_vs_human.csv`. **Failure modes:** detector catches
LONG rallies well (IoU 0.7?0.95,
468     tight boundaries) but MISSES short 3?5s rallies (most of the 17 misses) and FRAGMENTS
long ones (golden#1's 34.8s
469     split into 2 sub-fires; most "FPs" are real-rally fragments + one possibly-untagged
early rally @23.5s, NOT
470     hallucinations ? precision is recoverable via merge_gap). gate2 alone hurt recall hard
here (0.65?0.475) ? net-axis
471     est may be off for this camera. Levers: min_window_duration? (short rallies),
merge_gap? (fragmentation).

```

4. user (2026-06-10T11:17:43.211Z)

```

1     import os
2     import time
3     import json
4     import logging
5     import subprocess
6     import concurrent.futures
7     from typing import List, Dict, Any, Tuple
8     from backend.infrastructure.database import Database
9     from backend.pipeline.segmenters.base import BasePlaySegmenter, segmenter_registry
10    from backend.utils.video import get_video_duration, extract_video_chunk
11    from backend.sports import sport_registry
12    from backend.providers import provider_registry
13
14    logger = logging.getLogger(__name__)
15
16    class GeminiPlaySegmenter(BasePlaySegmenter):
17        @property
18        def name(self) -> str:
19            return "gemini"
20
21        @property
22        def description(self) -> str:
23            return "High-precision AI-powered semantic play segment detection using Google
Gemini Multimodal API."
24
25        def process_video(self, video_id: str, video_path: str, player_pool: List[str] =
None, max_frames: int = None) -> List[Dict[str, Any]]:
26            """
27            Processes a video to detect play segments using Google Gemini Multimodal Video
API,
28            populates SQLite, and indexes detailed events.
29            """
30            # Get sport profile config
31            sport_name = self.config.get("sport", "badminton")
32            try:
33                sport_profile = sport_registry.get(sport_name)
34            except KeyError as e:
35                logger.error(str(e))
36            return []

```

```

37
38         # Get AI provider and model config
39         idx_cfg = self.config.get("indexing", {})
40         provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider",
"gemini"))
41         model_name = self.config.get("ai_model", idx_cfg.get("ai_model",
"gemini-2.5-flash"))
42
43         # Get appropriate API key based on the provider
44         api_key_env_var = f"{provider_name.upper()}_API_KEY"
45         api_key = os.environ.get(api_key_env_var)
46         if not api_key:
47             logger.error(
48                 f"{api_key_env_var} environment variable is not set! "
49                 f"To run the {provider_name} segmenter, set your API key. "
50                 f"In PowerShell: $env:{api_key_env_var}=\"your_api_key_here\""
51             )
52             return []
53
54         try:
55             provider = provider_registry.get(provider_name, api_key=api_key,
model_name=model_name)
56         except KeyError as e:
57             logger.error(str(e))
58             return []
59
60         # A: Check for debug_duration to trim the video first
61         debug_duration = self.config.get("indexing", {}).get("debug_duration")
62         video_to_upload = video_path
63         is_trimmed = False
64         temp_trimmed_path = None
65         is_compressed = False
66         temp_compressed_path = None
67
68         if debug_duration:
69             logger.info(f"DEBUG FEATURE: Trimming video to first {debug_duration}
seconds for fast processing...")
70             temp_trimmed_path = os.path.join("output", f"temp_trimmed_{video_id}.mp4")
71             os.makedirs("output", exist_ok=True)
72             if os.path.exists(temp_trimmed_path):
73                 try:
74                     os.remove(temp_trimmed_path)
75                 except Exception:
76                     pass
77
78             cmd = [
79                 "ffmpeg", "-y",
80                 "-ss", "0",
81                 "-t", str(debug_duration),
82                 "-i", video_path,
83                 "-c", "copy",
84                 temp_trimmed_path
85             ]
86             logger.debug(f"Running command: {' '.join(cmd)}")
87             try:
88                 subprocess.run(cmd, check=True, stdout=subprocess.DEVNULL,
stderr=subprocess.DEVNULL)
89                 logger.info(f"Trimmed video successfully created at:
{temp_trimmed_path}")
90                 video_to_upload = temp_trimmed_path
91                 is_trimmed = True
92             except Exception as e:
93                 logger.warning(f"Failed to trim video using ffmpeg: {e}. Falling back to
full video.")
94

```



```

95
96
97     # Helper to clean up local temp files
98     def cleanup_temp_files():
99         if is_trimmed and temp_trimmed_path and os.path.exists(temp_trimmed_path):
100             try:
101                 os.remove(temp_trimmed_path)
102             except Exception:
103                 pass
104
105     # 1. Determine actual video duration
106     video_duration = get_video_duration(video_to_upload)
107     if video_duration > 0:
108         logger.info(f"Detected video duration: {video_duration:.1f}s")
109     else:
110         logger.warning("Could not detect video duration. Prompt will not include
duration context.")
111
112     # 2. Decide whether to use chunked processing
113     idx_cfg = self.config.get("indexing", {})
114     chunk_duration = idx_cfg.get("chunk_duration", 300)    # 5 minutes default
115     chunk_overlap = idx_cfg.get("chunk_overlap", 30)       # 30 seconds overlap
116     use_chunking = video_duration > 0 and video_duration > chunk_duration
117
118     ai_segments = []
119     chunks_dir = os.path.join("output", "chunks")
120
121     if use_chunking:
122         # --- CHUNKED PROCESSING ---
123         step = chunk_duration - chunk_overlap
124         chunk_starts = []
125         t = 0.0
126         while t < video_duration:
127             chunk_starts.append(t)
128             t += step
129         num_chunks = len(chunk_starts)
130         logger.info(
131             f"Video is {video_duration:.1f}s ? splitting into {num_chunks}
overlapping chunks "
132             f"({chunk_duration}s each, {chunk_overlap}s overlap).")
133         )
134
135         os.makedirs(chunks_dir, exist_ok=True)
136
137         def process_single_chunk(ci: int, chunk_start: float) -> Tuple[str, list,
dict]:
138             chunk_end = min(chunk_start + chunk_duration, video_duration)
139             actual_chunk_dur = chunk_end - chunk_start
140             chunk_label = f"Chunk {ci+1}/{num_chunks}"
141             chunk_path = os.path.join(chunks_dir, f"chunk_{video_id}_{ci}.mp4")
142
143             # Extract chunk with ffmpeg
144             logger.info(f"Extracting {chunk_label}: {chunk_start:.1f}s -
{chunk_end:.1f}s ({actual_chunk_dur:.1f}s)")
145             success = extract_video_chunk(video_to_upload, chunk_start,
actual_chunk_dur, chunk_path)
146             if not success:
147                 logger.error(f"Failed to extract {chunk_label}. Skipping.")
148                 return chunk_label, [], {}
149
150             # Measure chunk size
151             try:
152                 chunk_size_mb = os.path.getsize(chunk_path) / (1024 * 1024)
153             except Exception:
154                 chunk_size_mb = 0.0

```

```

155
156         # Get prompt from sport profile
157         prompt = sport_profile.get_prompt(chunk_duration=actual_chunk_dur)
158
159         # Process this chunk through the provider
160         chunk_segments, metrics = provider.analyze_video(
161             chunk_path, prompt, chunk_duration=actual_chunk_dur,
label=chunk_label
162         )
163         metrics["file_size_mb"] = chunk_size_mb
164         metrics["segments_found"] = len(chunk_segments)
165
166         # Offset timestamps back to full-video coordinates
167         for segment in chunk_segments:
168             if "start_time" in segment:
169                 try:
170                     segment["start_time"] = float(segment["start_time"]) +
chunk_start
171
172                     except (ValueError, TypeError):
173                         pass
174                 if "end_time" in segment:
175                     try:
176                         segment["end_time"] = float(segment["end_time"]) +
chunk_start
177
178                     except (ValueError, TypeError):
179                         pass
180
181                 logger.info(f"{chunk_label} returned {len(chunk_segments)} play
segments.")
182
183         # Clean up chunk file
184         try:
185             os.remove(chunk_path)
186         except Exception:
187             pass
188
189         return chunk_label, chunk_segments, metrics
190
191     # Run chunks in parallel. Workers come from indexing.ai_max_workers (same
knob
192     # the hybrids honor; was hardcoded 5). ai_max_workers=1 = serial ? needed to
193     # gently ramp a cold/prepaid Gemini project, whose burst throttling returns
a
194     # misleading "credits depleted" 429 under 5 simultaneous uploads.
195     all_metrics = {}
196     max_workers = max(1, int(self.config.get("indexing",
197     {}).get("ai_max_workers", 5)))
198     logger.info("Starting parallel processing with up to %d concurrent
workers...", max_workers)
199     t_chunking_start = time.time()
200     with concurrent.futures.ThreadPoolExecutor(max_workers=max_workers) as
executor:
201         futures = [executor.submit(process_single_chunk, ci, start) for ci,
start in enumerate(chunk_starts)]
202         for future in concurrent.futures.as_completed(futures):
203             label, segments, metrics = future.result()
204             ai_segments.extend(segments)
205             if metrics:
206                 all_metrics[label] = metrics
207
208     # Print Summary Log
209     summary_lines = [
210         "",
211         "=" * 60,
212         "
                GEMINI CHUNKING SUMMARY",

```

```

210         "=" * 60,
211         f" Total time: {time.time() - t_chunking_start:.1f}s",
212         f" Total chunks: {num_chunks}",
213         f" Total play segments found: {len(ai_segments)}",
214         "-" * 60,
215         f" {'Chunk':<12} | {'Size':<8} | {'Upload':<8} | {'Process':<8} |
{'Gen.':<8} | {'Segments'}",
216         "-" * 60,
217     ]
218     for i in range(num_chunks):
219         lbl = f"Chunk {i+1}/{num_chunks}"
220         m = all_metrics.get(lbl, {})
221         sz = f"{m.get('file_size_mb', 0):.1f}MB" if m else "N/A"
222         upl = f"{m.get('upload_time', 0):.1f}s" if m else "N/A"
223         prc = f"{m.get('process_time', 0):.1f}s" if m else "N/A"
224         gen = f"{m.get('gen_time', 0):.1f}s" if m else "N/A"
225         seg = m.get('segments_found', 0) if m else 0
226         summary_lines.append(f" {lbl:<12} | {sz:<8} | {upl:<8} | {prc:<8} |
{gen:<8} | {seg}")
227     summary_lines.append("=" * 60)
228     logger.info("\n".join(summary_lines))
229
230     # Clean up local temp files
231     cleanup_temp_files()
232     try:
233         os.rmdir(chunks_dir)
234     except Exception:
235         pass
236
237     logger.info(f"All chunks processed. {len(ai_segments)} total play segments
before deduplication.")
238
239     else:
240         # --- SINGLE-FILE PROCESSING (short videos) ---
241         try:
242             file_sz = os.path.getsize(video_to_upload) / (1024 * 1024)
243         except Exception:
244             file_sz = 0.0
245
246         # Get prompt from sport profile
247         prompt = sport_profile.get_prompt(chunk_duration=video_duration)
248         ai_segments, metrics = provider.analyze_video(
249             video_to_upload, prompt, chunk_duration=video_duration,
label="SingleFile"
250         )
251         summary_lines = [
252             "",
253             "=" * 50,
254             "          GEMINI SINGLE FILE SUMMARY",
255             "=" * 50,
256             f" File Size: {file_sz:.1f} MB",
257             f" Upload time: {metrics.get('upload_time', 0):.1f}s",
258             f" API Processing time: {metrics.get('process_time', 0):.1f}s",
259             f" Prompt Generation time: {metrics.get('gen_time', 0):.1f}s",
260             f" Play segments found: {len(ai_segments)}",
261             "=" * 50,
262         ]
263         logger.info("\n".join(summary_lines))
264         cleanup_temp_files()
265
266     if not ai_segments:
267         return []
268
269     # Populate SQLite DB with parsed play segments
270     segments_data = []

```

```

271         logger.info(f"Processing {len(ai_segments)} play segments through
validation...")
272
273     def parse_time(val) -> float:
274         """Converts a timestamp value to total float seconds.
275         Handles float/int pass-through, plain numeric strings, and
276         colon-separated formats (MM:SS, HH:MM:SS). Logs warnings
277         when unexpected formats are encountered."""
278         if isinstance(val, (int, float)):
279             return float(val)
280         if isinstance(val, str):
281             val = val.strip()
282             if ":" in val:
283                 # Gemini returned MM:SS or HH:MM:SS despite being asked for decimal
seconds.
284                 # Convert it, but warn so the issue is visible in logs.
285                 parts = val.split(":")
286                 parts.reverse()
287                 total = 0.0
288                 for i, p in enumerate(parts):
289                     try:
290                         total += float(p) * (60 ** i)
291                     except ValueError:
292                         pass
293                 logger.warning(f"Timestamp '{val}' was in colon-separated format
(MM:SS/HH:MM:SS) instead of decimal seconds. Converted to {total:.2f}s.")
294                 return total
295             try:
296                 return float(val)
297             except ValueError:
298                 logger.warning(f"Could not parse timestamp string '{val}' as a
number. Defaulting to 0.0.")
299                 return 0.0
300                 logger.warning(f"Unexpected timestamp type {type(val).__name__} for value
'{val}'. Defaulting to 0.0.")
301                 return 0.0
302
303     # 7. Parse timestamps and build candidate segment list
304     idx_cfg = self.config.get("indexing", {})
305     min_segment_duration = idx_cfg.get("min_segment_duration", 3.0)
306     max_segment_duration = 90.0 # Singles play segments almost never exceed this
307
308     candidates = []
309     for idx, segment in enumerate(ai_segments):
310         start = parse_time(segment.get("start_time", 0.0))
311         end = parse_time(segment.get("end_time", 0.0))
312         if start >= end:
313             logger.info(f"Skipping segment #{idx+1}: start_time ({start:.2f}s) >=
end_time ({end:.2f}s)")
314             continue
315             candidates.append({
316                 "idx": idx,
317                 "start": start,
318                 "end": end,
319                 "raw": segment
320             })
321
322     # 8. Post-hoc validation pass
323     logger.info(f"Running validation on {len(candidates)} candidate play
segments...")
324     validated = []
325     rejected = []
326
327     for c in candidates:
328         label = f"Segment #{c['idx']+1} [{c['start']:.2f}s - {c['end']:.2f}s]"

```

```

329
330         # A: Reject segments that start beyond the video
331         if video_duration > 0 and c["start"] >= video_duration:
332             rejected.append((label, f"start_time ({c['start']:.2f}s) is beyond video
duration ({video_duration:.1f}s)"))
333             continue
334
335         # B: Clamp end_time to video duration
336         if video_duration > 0 and c["end"] > video_duration:
337             logger.info(f"{label}: end_time ({c['end']:.2f}s) exceeds video duration
({video_duration:.1f}s). Clamping.")
338             c["end"] = video_duration
339
340         dur = c["end"] - c["start"]
341
342         # C: Reject segments shorter than min_segment_duration
343         if dur < min_segment_duration:
344             rejected.append((label, f"duration ({dur:.2f}s) is below minimum
({min_segment_duration}s)"))
345             continue
346
347         # D: Flag suspiciously long segments
348         if dur > max_segment_duration:
349             logger.warning(
350                 f"{label}: duration ({dur:.1f}s) is unusually long
(>{max_segment_duration}s). "
351                 f"This may be a detection error. Keeping but flagging."
352             )
353
354         validated.append(c)
355
356     # E: Resolve overlapping segments (keep the longer one)
357     if len(validated) > 1:
358         # Sort by start time
359         validated.sort(key=lambda r: r["start"])
360         non_overlapping = [validated[0]]
361         for curr in validated[1:]:
362             prev = non_overlapping[-1]
363             # Check if current segment overlaps with the previous one
364             if curr["start"] < prev["end"]:
365                 prev_dur = prev["end"] - prev["start"]
366                 curr_dur = curr["end"] - curr["start"]
367                 kept, dropped = (prev, curr) if prev_dur >= curr_dur else (curr,
prev)
368                 kept_label = f"Segment #{kept['idx']+1} [{kept['start']:.2f}s -
{kept['end']:.2f}s]"
369                 dropped_label = f"Segment #{dropped['idx']+1}
[{dropped['start']:.2f}s - {dropped['end']:.2f}s]"
370                 rejected.append((dropped_label, f"overlaps with {kept_label} ? kept
the longer segment"))
371                 if kept == curr:
372                     non_overlapping[-1] = curr # Replace previous with current
373             else:
374                 non_overlapping.append(curr)
375         validated = non_overlapping
376
377     # Log validation summary
378     if rejected:
379         reject_lines = [f" ? {label}: {reason}" for label, reason in rejected]
380         logger.info(f"{len(rejected)} play segments were REJECTED:\n" +
"\n".join(reject_lines))
381         logger.info(f"{len(validated)} play segments passed validation (out of
{len(ai_segments)} from Gemini).")
382
383     stitch_cfg = self.config.get("stitching", {})

```

```

384         begin_pad = stitch_cfg.get("begin_padding", 0.5)
385         end_pad = stitch_cfg.get("end_padding", 0.5)
386
387         total_highlight_dur = 0.0
388         for c in validated:
389             total_highlight_dur += (c["end"] - c["start"]) + begin_pad + end_pad
390
391         mins = int(total_highlight_dur // 60)
392         secs = int(total_highlight_dur % 60)
393         logger.info(
394             f"Estimated final highlight video duration: {mins:02d}:{secs:02d} "
395             f"({total_highlight_dur:.1f}s total; includes {begin_pad}s begin +
396             {end_pad}s end padding per segment)"
397         )
398
399         # 9. Populate SQLite DB with validated play segments
400         for c in validated:
401             start = c["start"]
402             end = c["end"]
403             segment = c["raw"]
404             dur = end - start
405
406             # Extract shot_count from Gemini or compute estimate if not present
407             shot_count = segment.get("shuttle_exchange_count")
408             if shot_count is None:
409                 shot_count = max(3, int(dur / 0.8)) # 1 shot every 0.8 seconds as
410                 fallback estimate
411             else:
412                 try:
413                     shot_count = int(shot_count)
414                 except Exception:
415                     shot_count = max(3, int(dur / 0.8))
416
417             # Only store metrics we actually know ? no fabricated data
418             metadata = {
419                 "shot_count": shot_count,
420                 "shot_count_confidence": segment.get("shot_count_confidence",
421                 "Unknown"),
422                 "detector": model_name
423             }
424
425             segment_id = self.db.add_play_segment(video_id, start, end, None, None,
426             metadata)
427
428             if segment_id:
429                 segments_data.append({
430                     "id": segment_id,
431                     "start_time": start,
432                     "end_time": end,
433                     "duration": dur,
434                     "metadata": metadata
435                 })
436
437         return segments_data
438
439     # Register the segmenter
440     segmenter_registry.register(GeminiPlaySegmenter)
441
442

```

5. user (2026-06-10T11:17:43.619Z)

```

1     import os
2     import time
3     import json
4     import logging

```

```

5 from typing import Tuple
6 from google import genai
7 from google.genai import types
8 from backend.providers.base import AIVideoProvider
9
10 logger = logging.getLogger(__name__)
11
12 # The Gemini File API caps uploads at 2GB; stay well under it. Callers must chunk so
13 # each uploaded clip is below this. Oversize uploads are refused (not silently
attempted).
14 MAX_UPLOAD_MB = 500
15
16 class GeminiProvider(AIVideoProvider):
17     def __init__(self, api_key: str, model_name: str):
18         super().__init__(api_key, model_name)
19         # Initialize client lazily to avoid connection/auth issues during class
instantiation
20         self._client = None
21
22     @property
23     def client(self):
24         if self._client is None:
25             self._client = genai.Client(api_key=self.api_key)
26         return self._client
27
28     def analyze_video(self, video_path: str, prompt: str,
29                      chunk_duration: float = 0.0,
30                      label: str = "") -> Tuple[list, dict]:
31         log_prefix = f"Gemini {label}" if label else "Gemini"
32
33         metrics = {
34             "upload_time": 0.0,
35             "process_time": 0.0,
36             "gen_time": 0.0
37         }
38
39         # 0. Refuse oversize uploads (Gemini File API limit is 2GB; we cap lower for
safety/speed)
40         try:
41             size_mb = os.path.getsize(video_path) / (1024 * 1024)
42         except OSError:
43             size_mb = 0.0
44         if size_mb > MAX_UPLOAD_MB:
45             logger.error(f"[{log_prefix}] {video_path} is {size_mb:.0f}MB >
{MAX_UPLOAD_MB}MB cap ? "
46                        f"chunk smaller. Skipping upload.")
47             return [], metrics
48
49         # 1. Upload (retry transient network failures, e.g. WinError 10053 / reset
connections)
50         logger.info(f"[{log_prefix}] Uploading {video_path}...")
51         t_upload_start = time.time()
52         video_file = None
53         for attempt in range(3):
54             try:
55                 video_file = self.client.files.upload(file=video_path)
56                 metrics["upload_time"] = time.time() - t_upload_start
57                 logger.info(f"[{log_prefix}] Upload complete in
{metrics['upload_time']:.1f}s. URI: {video_file.uri}")
58                 break
59             except Exception as e:
60                 logger.warning(f"[{log_prefix}] Upload attempt {attempt + 1}/3 failed:
{e}")
61                 if attempt < 2:
62                     time.sleep(2 * (attempt + 1))

```

```

63         if video_file is None:
64             logger.error(f"[{log_prefix}] Upload failed after 3 attempts.")
65             return [], metrics
66
67         # 2. Wait for processing on Google servers (bounded ? a stuck file must not hang
68         # the worker thread / request forever).
69         t_process_start = time.time()
70         process_deadline_s = 1200 # 20 min
71         try:
72             while video_file.state.name == "PROCESSING":
73                 if time.time() - t_process_start > process_deadline_s:
74                     logger.error(f"[{log_prefix}] Processing exceeded
{process_deadline_s}s ? giving up.")
75                     try:
76                         self.client.files.delete(name=video_file.name)
77                     except Exception:
78                         pass
79                     return [], metrics
80                     logger.info(f"[{log_prefix}] Processing on Google servers. Elapsed:
{time.time()-t_process_start:.1f}s. Retrying in 10s...")
81                     time.sleep(10)
82                     video_file = self.client.files.get(name=video_file.name)
83                 if video_file.state.name == "FAILED":
84                     logger.error(f"[{log_prefix}] Video processing failed:
{video_file.error.message}")
85                     try:
86                         self.client.files.delete(name=video_file.name)
87                     except Exception:
88                         pass
89                     return [], metrics
90             except Exception as e:
91                 logger.error(f"[{log_prefix}] Status check failed: {e}")
92                 try:
93                     self.client.files.delete(name=video_file.name)
94                 except Exception:
95                     pass
96                 return [], metrics
97
98         metrics["process_time"] = time.time() - t_process_start
99         logger.info(f"[{log_prefix}] Processing finished in
{metrics['process_time']:.1f}s.")
100
101         # 3. Call Gemini (retry transient 503/UNAVAILABLE model-overload + network
blips)
102         logger.info(f"[{log_prefix}] Submitting multimodal request to
{self.model_name}...")
103         t_gen_start = time.time()
104         raw_text = None
105         for attempt in range(3):
106             try:
107                 response = self.client.models.generate_content(
108                     model=self.model_name,
109                     contents=[video_file, prompt],
110                     config=types.GenerateContentConfig(
111                         response_mime_type="application/json",
112                         temperature=0.0
113                     )
114                 )
115                 raw_text = response.text
116                 metrics["gen_time"] = time.time() - t_gen_start
117                 logger.info(f"[{log_prefix}] Response received in
{metrics['gen_time']:.1f}s:\n{raw_text}")
118                 break
119             except Exception as e:
120                 logger.warning(f"[{log_prefix}] generate attempt {attempt + 1}/3 failed:

```



```

{e}")
121         if attempt < 2:
122             time.sleep(3 * (attempt + 1))
123     try:
124         self.client.files.delete(name=video_file.name)
125     except Exception:
126         pass
127
128     if not raw_text:
129         return [], metrics
130
131     # 4. Parse JSON
132     try:
133         segments = json.loads(raw_text)
134         if not isinstance(segments, list):
135             logger.error(f"[{log_prefix}] Response is not a JSON array.")
136             return [], metrics
137         return segments, metrics
138     except Exception as e:
139         logger.error(f"[{log_prefix}] JSON parse failed: {e}")
140         return [], metrics
141

```

6. assistant (2026-06-10T11:17:52.360Z)

Now let me read the supporting files: video utils, config models, and the reporting spec/harvest.

7. user (2026-06-10T11:17:53.321Z)

```

1     import subprocess
2     import os
3     import shutil
4     import logging
5
6     logger = logging.getLogger(__name__)
7
8     def get_video_duration(video_path: str) -> float:
9         """Returns the exact duration of a video file in seconds using ffprobe.
10         Returns 0.0 on failure."""
11         try:
12             duration_output = subprocess.check_output([
13                 "ffprobe", "-v", "error", "-show_entries", "format=duration",
14                 "-of", "default=noprint_wrappers=1:nokey=1", video_path
15             ]).decode().strip()
16             return float(duration_output)
17         except Exception as e:
18             logger.error(f"Could not detect video duration for {video_path}: {e}")
19             return 0.0
20
21     def extract_video_chunk(video_path: str, start_time: float, duration: float,
22                             output_path: str, reencode: bool = False,
23                             scale_width: int = None) -> bool:
24         """Extracts a chunk from a video file.
25
26         Args:
27             reencode: If True, re-encodes with libx264 (slower but frame-accurate).
28                       If False, uses stream-copy (fast but cuts at nearest keyframe).
29                       Use True for short clips where keyframe drift matters.
30             scale_width: If set (requires reencode), downscale to this width (height auto,
31                           even). Useful for high-res sources headed to an API that analyses at
32                           low res anyway ? shrinks uploads with no analysis loss.
33
34         """
35         cmd = [
36             "ffmpeg", "-y",

```

```

36         "-ss", str(start_time),
37         "-t", str(duration),
38         "-i", video_path,
39     ]
40     if reencode:
41         if scale_width:
42             cmd += ["-vf", f"scale={int(scale_width)}:-2"]
43             cmd += ["-c:v", "libx264", "-preset", "ultrafast", "-crf", "23", "-c:a", "aac"]
44         else:
45             cmd += ["-c", "copy"]
46         cmd.append(output_path)
47         try:
48             subprocess.run(cmd, check=True, stdout=subprocess.DEVNULL,
stderr=subprocess.DEVNULL)
49             return True
50         except Exception as e:
51             logger.error(f"Failed to extract chunk {start_time}s - {start_time+duration}s:
{e}")
52             return False
53

```

8. user (2026-06-10T11:17:54.368Z)

```

64: class IndexingConfig(BaseModel):
65-     default_segmenter: str = "yolo_hybrid"
66-     use_gpu: bool = True
67-     motion_threshold: float = 0.08
68-     min_segment_duration: float = 3.0
69-     dead_time_merge_gap: float = 2.0
70-     chunk_duration: float = 90.0
71-     chunk_overlap: float = 10.0
72-     detector_model: str = "fasterrcnn_resnet50_fpn"
73-     detector_score_threshold: float = 0.5
74-     yolo_velocity_threshold: float = 0.15
75-     yolo_frame_skip: int = 3
76-     yolo_tracker: str = "bytetrack.yaml"
77-     yolo_prefetch_workers: int = 2
78-     min_action_window_duration: float = 2.0
79-     log_yolo_candidates: bool = True
80-     store_yolo_candidates: bool = True
81-     ai_handoff_padding: float = 2.0
82-     ai_max_workers: int = 5
83-     # Optional debug knob: trim every video to the first N seconds before processing.
84-     debug_duration: Optional[float] = None
85-
86-     tracknet: TrackNetConfig = Field(default_factory=TrackNetConfig)
87-     wasb: WasbIndexConfig = Field(default_factory=WasbIndexConfig)
88-
89-     @field_validator("default_segmenter")
90-     @classmethod
91-     def segmenter_must_be_registered(cls, v: str) -> str:
92-         # Registry check happens at runtime in main/segmenter selection.
93-         # Here we just allow any string for forward compatibility.
94-         return v
95-
96-     @model_validator(mode="after")
97-     def _chunk_step_must_be_positive(self) -> "IndexingConfig":
98-         # The chunked segmenters advance by (chunk_duration - chunk_overlap); a non-positive
99-         # step makes `while t < duration: t += step` loop forever, growing memory before any
100-         # GPU work. Reject the bad config at load time instead.
101-         if self.chunk_overlap >= self.chunk_duration:
102-             raise ValueError(
103-                 f"indexing.chunk_overlap ({self.chunk_overlap}) must be < chunk_duration "
104-                 f"({self.chunk_duration}); otherwise chunking never advances."
105-             )

```

```

106-         return self
107-
108-
109-class OutputConfig(BaseModel):
110-     default_highlights_name: str = "highlights.mp4"
111-     db_name: str = "sports_indexer.db"
112-     # Directory where the DB, highlight reels, and processed clips are written.
113-     # The CLI's --output-dir overrides this at startup (see backend\main.py::main).
114-     output_dir: str = "output"
115-
116-
117-class StitchingConfig(BaseModel):
118-     begin_padding: float = 0.5
119-     end_padding: float = 0.5
120-     use_cache: bool = False
121-     min_shot_count: int = 1
122-
123-
124-class SecurityConfig(BaseModel):

```

9. user (2026-06-10T11:17:54.851Z)

```

backend\reporting\__init__.py
backend\reporting\harvest.py
backend\reporting\spec.yaml
backend\reporting\__pycache__\__init__.cpython-313.pyc
backend\reporting\__pycache__\harvest.cpython-313.pyc

```

10. user (2026-06-10T11:18:00.762Z)

```

1      # Observability report spec for the badminton-highlight-indexer.
2      # The report ENGINE (config language, renderers) lives in the shared `obsreport`
3      # library; this file is the indexer-specific layout + footgun rules. Every
4      # actionable rule cites a README/FAQ section so a reader can self-debug.
5      name: indexer
6      title: Badminton Highlight Indexer ? Video Report
7
8      # Customer here = whoever receives the highlight reel.
9      customer_verdict:
10         pass: "? Your highlights are ready."
11         pass_with_warnings: "? Your highlights are ready. (A couple of minor notes above.)"
12         fail: "?? We couldn't finish generating highlights for this video. Please contact
support."
13
14     sections:
15         - { id: run,          title: "Run",          kind: identity,  audience: both }
16         - { id: video,       title: "Video",         kind: video_meta, audience: both }
17         - { id: highlights,  title: "Highlights",     kind: highlights, audience: customer
18     }
19         - { id: cust_notes,  title: "Things to know", kind: flags,      audience: customer
20     }
21         - { id: stages,     title: "Processing stages", kind: stages,    audience:
technical }
22         - { id: findings,   title: "Findings & warnings", kind: flags,      audience:
technical }
23         - { id: verdict,    title: "Verdict",         kind: verdict,   audience: both }
24
25     rules:
26         # The classic silent failure: 0 segments because the AI provider no-op'd on a
27         # missing API key ? NOT because the video has no rallies.
28         - code: silent_provider_noop
29           severity: error
30           when:
31             - { path: "stages.ai_handoff.details.api_key_present", op: eq, value: false }
32             - { path: "stages.segmentation.metrics.segment_count", op: eq, value: 0 }

```

```

31 message: >-
32     0 segments AND no AI provider API key was set. The segmenter (see Processing
33     stages > segmentation > segmenter_actual) hands its candidate windows to the AI
34     provider, which silently did nothing without a key ? so this is a MISSING
35     CREDENTIAL, not 'the video has no rallies'. Fix: put GEMINI_API_KEY=<key> in a
36     .env file in the project root and re-run (see README#Q7).
37     faq_ref: "README#Q7"
38     customer_message: >-
39         We couldn't generate highlights for this video due to a configuration issue
40         on our side. Please contact support.
41
42 # API key WAS present but an AI segmenter still produced nothing ? likely a
43 # provider error / quota / WSL healthcheck issue rather than a truly empty video.
44 - code: ai_empty_vs_no_rallies
45     severity: warn
46     when:
47         - { path: "stages.ai_handoff.details.ai_based", op: eq, value: true }
48         - { path: "stages.ai_handoff.details.api_key_present", op: eq, value: true }
49         - { path: "stages.segmentation.metrics.segment_count", op: eq, value: 0 }
50     message: >-
51         An AI-based segmenter returned 0 segments even though the API key is set.
52         Empty can mean a provider error, exhausted quota, or (for WASB/TrackNet) a
53         failed WSL healthcheck ? these all return [] silently. Re-run with -u for
54         unbuffered logs and check the provider/WSL output before concluding 'no rallies'.
55     faq_ref: "README#Q7"
56
57 # fps was assumed rather than probed -> velocity thresholds + timing may be wrong.
58 - code: fps_fallback_used
59     severity: warn
60     when:
61         - { path: "video_meta.fps_source", op: eq, value: fallback }
62     message: >-
63         fps was NOT probed from the file ? a fallback default (shown as the fps value)
64         was used. Because rally timing and motion thresholds scale with fps, boundaries
65         and motion sensitivity may be miscalibrated. This usually means validation didn't
66         run (it probes the file) or ffprobe couldn't read r_frame_rate. Confirm the real
67         fps with ffprobe and re-run.
68     faq_ref: "README#Q14"
69
70 # Audio stream missing or disabled. Per VIDEO_RECORDING_GUIDELINES (and ADR-007)
71 # the shuttle "thwack" is a first-class rally cue for recall; recording with audio
72 # on (mic unobstructed, no aggressive noise suppression) is now a hard requirement
73 # for good results with the shuttle-tracking substrates.
74 - code: audio_disabled_or_missing
75     severity: warn
76     when:
77         - { path: "video_meta.audio_present", op: eq, value: false }
78     message: >-
79         No audio stream was detected in the input (or it could not be read). The shuttle
80         "thwack" impact sound is a strong cue that improves rally recall (especially for
81         the WASB/TrackNet detector paths). Record with audio enabled, keep the camera
82         mic unobstructed, and avoid heavy wind/noise-reduction filters that flatten the
83         transient. See VIDEO_RECORDING_GUIDELINES.md (the "Audio (REQUIRED)" row) and
84         AUDIO_RALLY_DETECTION_PLAN.md.
85     faq_ref: "VIDEO_RECORDING_GUIDELINES"
86     customer_message: >-
87         For best highlight results, record with audio on and the mic unobstructed.
88         The shuttle impact sound helps the detector find rallies. Re-record or
89         re-process after enabling audio (see VIDEO_RECORDING_GUIDELINES.md).
90
91 # The motion baseline fabricates its per-segment metadata/events ? they are not
92 # measured. Warn so nobody trusts them as ground truth.
93 - code: synthetic_motion_metadata
94     severity: warn
95     when:

```

```

96         - { path: "stages.segmentation.details.segmenter_actual", op: eq, value: motion }
97     message: >-
98         The 'motion' segmenter was used: its per-segment metadata (intensity, speed,
99         events, server/receiver) are PLACEHOLDER values (unmeasured) ? fabricated by the
100         heuristic, NOT derived from the video ? so a PASS with segments here does not mean
101         the metadata is trustworthy. Use yolo_hybrid/gemini for real shot metadata.
102     faq_ref: "README#Q10"
103
104     # The segmenter that ran differs from config's default -> you may be looking at a
105     # different detector than expected (config/setup/CLI defaults are known to diverge).
106     - code: segmenter_divergence
107       severity: info
108       when:
109         - { path: "stages.segmentation.details.matches_config_default", op: eq, value:
false }
110         - { path: "stages.segmentation.details.segmenter_explicitly_requested", op: eq,
value: false }
111     message: >-
112         The segmenter that ran differs from config.json's default_segmenter, and no
113         explicit --segmenter was given ? so a fallback default kicked in. The indexer's
114         defaults are known to diverge across config.json / setup.py / the CLI; confirm
115         you ran the detector you meant.
116     faq_ref: "README#Q7"
117
118     # Re-processing an already-indexed video replaced its prior segments (INSERT OR
119     # REPLACE cascades). Note it so a changed result isn't mistaken for instability.
120     - code: reingest_replaced_prior
121       severity: info
122       when:
123         - { path: "stages.segmentation.details.was_reingest", op: eq, value: true }
124     message: >-
125         This video was already in the database; re-processing REPLACED its previous
126         segments (videos are keyed by file MD5; re-ingest cascades a delete). Expected
127         when you intend to re-run; just know the earlier results are gone.
128     faq_ref: "README#Q11"
129
130     # A validation check failed -> segmentation was halted. Point the reader at the
131     # report's Validation section + the per-check FAQ.
132     - code: validation_failed
133       severity: error
134       when:
135         - { path: "stages.validation.status", op: eq, value: error }
136     message: >-
137         One or more validation checks failed, so indexing was halted. See the
138         Processing stages > validation block above: each failed check states the
139         reason (relevance -> README#Q2, fps/keyframes -> README#Q5).
140     faq_ref: "README#Q13"
141     customer_message: >-
142         This video couldn't be processed because it didn't pass our quality checks
143         (for example: too blurry, too low resolution, or not a badminton court).
144

```

11. user (2026-06-10T11:18:00.940Z)

```

1     """Build an observability report for one indexer run.
2
3     Pure-ish harvest: given the validation results/context, the persisted segments,
4     and run metadata, populate an `obsreport.RunRecorder` with the raw signals. The
5     *footgun rules* (which turn signals into flags) live declaratively in spec.yaml,
6     so this module only records facts faithfully ? it does not decide warnings.
7
8     Imported only after `backend.reporting` confirms obsreport is installed, so it may
9     import obsreport freely.
10    """
11    from __future__ import annotations

```

```

12
13 import os
14 from pathlib import Path
15 from typing import Any, Dict, List, Optional
16
17 try:
18     import obsreport # soft dep; only needed for RunRecorder / load_spec at runtime
19 except ImportError:
20     obsreport = None # type: ignore[assignment]
21
22 _AI_SEGMENTERS = {"yolo_hybrid", "tracknet_hybrid", "wasb_hybrid", "gemini"}
23 _SPEC_PATH = Path(__file__).resolve().parent / "spec.yaml"
24
25
26 def load_spec() -> "obsreport.ReportSpec":
27     return obsreport.load_spec(_SPEC_PATH)
28
29
30 # --- media metadata -----
31 def video_meta_from_context(val_context: Optional[Dict[str, Any]], video_path:
Optional[str]) -> Dict[str, Any]:
32     """Derive VideoMeta fields from the validators' shared ffprobe_meta. fps_source
33     distinguishes a real probe from the absence of one (the fallback footgun)."""
34     meta: Dict[str, Any] = {"fps_source": "unknown"}
35     if video_path and os.path.exists(video_path):
36         try:
37             meta["size_mb"] = round(os.path.getsize(video_path) / (1024 * 1024), 1)
38         except OSError:
39             pass
40
41     ffmata = (val_context or {}).get("ffprobe_meta") or {}
42     streams = ffmata.get("streams") or []
43     fmt = ffmata.get("format") or {}
44     if streams:
45         s = streams[0]
46         w, h = s.get("width"), s.get("height")
47         meta["width"], meta["height"] = w, h
48         if w and h:
49             meta["resolution"] = f"{w}x{h}"
50         fps_str = s.get("r_frame_rate", "0/1")
51         try:
52             num, den = (int(x) for x in fps_str.split("/"))
53             if den:
54                 meta["fps"] = round(num / den, 2)
55                 meta["fps_source"] = "probed"
56             except (ValueError, ZeroDivisionError):
57                 pass
58         if s.get("codec_name"):
59             meta["container"] = fmt.get("format_name") or s.get("codec_name")
60     if fmt.get("duration"):
61         try:
62             meta["duration_s"] = round(float(fmt["duration"]), 1)
63         except (ValueError, TypeError):
64             pass
65     if fmt.get("format_name"):
66         meta["container"] = fmt.get("format_name")
67
68     # Audio readiness signal (for Input Quality UX extension per review item 2).
69     # Guidelines make "record with audio on, mic unobstructed" a first-class requirement
70     # (ADR-007) because shuttle thwacks are a strong rally cue for recall.
71     # We only record presence here (cheap, from ffprobe already in context); the actual
72     # thwack onset probe (pipeline/audio/hit_detector) remains a diagnostic /
73     calibration
74     # tool for now.
75     audio_streams = [s for s in streams if (s or {}).get("codec_type") == "audio"]

```

```

75     meta["audio_present"] = len(audio_streams) > 0
76     return meta
77
78
79 # --- stage builders -----
80 def _validation_stage(rec: "obsreport.RunRecorder", val_results: List[Any]) -> None:
81     with rec.stage("validation") as st:
82         total = len(val_results)
83         passed = sum(1 for r in val_results if getattr(r, "passed", False))
84         st.add_metric("checks_passed", passed)
85         st.add_metric("checks_total", total)
86         failures = [r for r in val_results if not getattr(r, "passed", True)]
87         for r in failures:
88             st.messages.append(f"FAILED {getattr(r, 'validator_name', '?')}: {getattr(r, 'message', '')}")
89         st.status = "error" if failures else ("ok" if total else "not_run")
90
91
92 def _segmentation_stage(
93     rec: "obsreport.RunRecorder",
94     play_segments: List[Dict[str, Any]],
95     candidates: List[Dict[str, Any]],
96     segmenter_requested: Optional[str],
97     segmenter_actual: Optional[str],
98     config_default: Optional[str],
99     was_reingest: bool,
100     ran: bool,
101 ) -> None:
102     with rec.stage("segmentation") as st:
103         if not ran:
104             st.status = "not_run"
105         seg_count = len(play_segments)
106         total_play = round(sum(float(s.get("duration", 0.0)) for s in play_segments), 1)
107         detector_tag = None
108         if play_segments:
109             detector_tag = (play_segments[0].get("metadata") or {}).get("detector")
110             st.add_metric("candidate_windows", len(candidates))
111             st.add_metric("segment_count", seg_count, audience="both")
112             st.add_metric("total_play_s", total_play, unit="s")
113             # "diverges" only when a segmenter actually RAN, the user did NOT explicitly
114             # pick it, and it differs from config's default (the genuine surprise case).
115             # An explicit --segmenter override or a halted run is not a divergence.
116             matches_default = True
117             if ran and segmenter_actual and config_default:
118                 matches_default = (segmenter_actual == config_default)
119             st.details.update({
120                 "segmenter_requested": segmenter_requested,
121                 "segmenter_actual": segmenter_actual,
122                 "config_default": config_default,
123                 "segmenter_explicitly_requested": segmenter_requested is not None,
124                 "matches_config_default": matches_default,
125                 "was_reingest": was_reingest,
126                 "detector": detector_tag,
127             })
128             # Surface metadata trustworthiness at the data level (not just in a warning):
129             # the motion baseline fabricates per-segment metadata; everything else measures
130             it.
131             if ran and segmenter_actual:
132                 st.details["metadata_source"] = (
133                     "synthetic/heuristic ? NOT measured" if segmenter_actual == "motion"
134                     else "measured (detector/AI)"
135                 )
136
137 def _ai_stage(rec: "obsreport.RunRecorder", config: Dict[str, Any], segmenter_actual:

```

```

Optional[str],
138         play_segments: List[Dict[str, Any]], ran: bool) -> None:
139     """Only meaningful for AI-based segmenters. Records whether the provider key was
140     present ? the signal the silent_provider_noop rule keys on."""
141     ai_based = segmenter_actual in _AI_SEGMENTERS
142     provider = (config.get("ai_provider") or "gemini")
143     key_env = f"{provider.upper()}_API_KEY"
144     with rec.stage("ai_handoff") as st:
145         if not ran or not ai_based:
146             st.status = "not_run" if not ran else "skipped"
147             st.details.update({
148                 "ai_based": ai_based,
149                 "provider": provider if ai_based else None,
150                 "model": config.get("ai_model") if ai_based else None,
151                 "api_key_present": bool(os.environ.get(key_env)) if ai_based else None,
152             })
153         if ai_based:
154             st.add_metric("confirmed_segments", len(play_segments))
155
156
157 def _highlights(rec: "obsreport.RunRecorder", play_segments: List[Dict[str, Any]],
158               video_meta: Dict[str, Any]) -> None:
159     total = round(sum(float(s.get("duration", 0.0)) for s in play_segments), 1)
160     coverage = None
161     dur = video_meta.get("duration_s")
162     if dur:
163         coverage = round(100.0 * total / dur, 1)
164     notes = []
165     if not play_segments:
166         notes.append("No highlights were produced for this video.")
167     rec.set_highlights(segment_count=len(play_segments), total_highlight_s=total,
168                       coverage_pct=coverage, notes=notes)
169
170
171 # --- top-level -----
172 def record_run(
173     *,
174     video_id: str,
175     video_path: Optional[str],
176     config: Dict[str, Any],
177     val_results: List[Any],
178     val_context: Optional[Dict[str, Any]],
179     play_segments: List[Dict[str, Any]],
180     candidates: List[Dict[str, Any]],
181     segmenter_requested: Optional[str] = None,
182     segmenter_actual: Optional[str] = None,
183     was_reingest: bool = False,
184     segmentation_ran: bool = True,
185     tool_version: Optional[str] = None,
186     spec: Optional["obsreport.ReportSpec"] = None,
187     **recorder_kwargs: Any,
188 ) -> "obsreport.RunRecorder":
189     """Assemble a finalized recorder for an indexer run (does NOT write)."""
190     rec = obsreport.RunRecorder(
191         "badminton-highlight-indexer",
192         tool_version=tool_version,
193         video_id=video_id,
194         video_id_kind="file_md5",
195         video_path=video_path,
196         spec=spec if spec is not None else load_spec(),
197         **recorder_kwargs,
198     )
199     vm = video_meta_from_context(val_context, video_path)
200     # If we never probed an fps but segmentation still ran, the segmenter must have
201     # used its internal fallback (30fps / 1280px) ? surface that as the footgun it is,

```



```

202      # AND show the assumed numeric value so the report reads "30 (fallback)" not "?"
(fallback)".
203      if vm.get("fps_source") == "unknown" and segmentation_ran:
204          vm["fps_source"] = "fallback"
205          if vm.get("fps") is None:
206              vm["fps"] = 30.0 # matches the runners' fallback (_probe_fps_width -> 30.0
/ 1280)
207      rec.set_video_meta(**vm)
208
209      _validation_stage(rec, val_results)
210      config_default = (config.get("indexing") or {}).get("default_segmenter")
211      _segmentation_stage(rec, play_segments, candidates, segmenter_requested,
212                          segmenter_actual, config_default, was_reingest,
segmentation_ran)
213      _ai_stage(rec, config, segmenter_actual, play_segments, segmentation_ran)
214      _highlights(rec, play_segments, vm)
215
216      rec.finalize()
217      return rec
218

```

12. user (2026-06-10T11:18:30.516Z)

```

backend\config\models.py:81:    ai_handoff_padding: float = 2.0
backend\reporting\__init__.py:15:    from . import harvest
backend\reporting\__init__.py:43:    # Normalize to a plain nested dict so harvest's dict access
works uniformly.
backend\reporting\__init__.py:54:        rec = harvest.record_run(
backend\reporting\harvest.py:3:Pure-ish harvest: given the validation results/context, the
persisted segments,
backend\reporting\harvest.py:144:    with rec.stage("ai_handoff") as st:
backend\reporting\harvest.py:172:def record_run(
backend\pipeline\detectors\base.py:58:harvest sees the outcome as "0 candidates" + detector
name. When we wire richer
backend\pipeline\segmenters\tracknet_hybrid.py:102:        handoff_padding =
idx_cfg.get("ai_handoff_padding", 2.0)
backend\pipeline\segmenters\wasb_hybrid.py:93:        handoff_padding =
idx_cfg.get("ai_handoff_padding", 2.0)
backend\pipeline\segmenters\yolo_hybrid.py:309:        handoff_padding =
idx_cfg.get("ai_handoff_padding", idx_cfg.get("gemini_handoff_padding", 2.0))

```

13. user (2026-06-10T11:18:31.555Z)

```

tests\test_gemini.py:39:@patch("backend.pipeline.segmenters.gemini.extract_video_chunk")
backend\eval\gemini_refine.py:11:sports.badminton prompt (sport_registry),
utils.video.extract_video_chunk,
backend\eval\gemini_refine.py:32:from backend.utils.video import get_video_duration,
extract_video_chunk
backend\eval\gemini_refine.py:83:    if not extract_video_chunk(video, cs, ce - cs, clip,
reencode=True, scale_width=clip_scale_width):
backend\eval\gemini_refine.py:171:        if not extract_video_chunk(video, cstart, cend -
cstart, clip, reencode=True, scale_width=clip_scale_width):
tests\test_provider_gemini.py:54:def
test_gemini_provider_rejects_oversize_before_upload(monkeypatch):
tests\test_provider_gemini.py:58:        # 600MB > MAX_UPLOAD_MB (500) -> refuse without ever
uploading
backend\utils\video.py:21:def extract_video_chunk(video_path: str, start_time: float, duration:
float,
backend\pipeline\segmenters\tracknet_hybrid.py:24:from backend.utils.video import
get_video_duration, extract_video_chunk
backend\pipeline\segmenters\tracknet_hybrid.py:185:        if not
extract_video_chunk(video_path, w_start, w_dur, w_path, reencode=True):
backend\pipeline\segmenters\gemini.py:10:from backend.utils.video import get_video_duration,
extract_video_chunk
backend\pipeline\segmenters\gemini.py:145:        success =

```

```

extract_video_chunk(video_to_upload, chunk_start, actual_chunk_dur, chunk_path)
backend\pipeline\segmenters\wasb_hybrid.py:22:from backend.utils.video import
get_video_duration, extract_video_chunk
backend\pipeline\segmenters\wasb_hybrid.py:165:            if not
extract_video_chunk(video_path, w_start, w_dur, w_path, reencode=True):
backend\pipeline\segmenters\yolo_hybrid.py:22:from backend.utils.video import
get_video_duration, extract_video_chunk
backend\pipeline\segmenters\yolo_hybrid.py:357:            success =
extract_video_chunk(video_path, cs, actual_dur, chunk_path)
backend\pipeline\segmenters\yolo_hybrid.py:397:            success =
extract_video_chunk(video_path, chunk_start, actual_dur, chunk_path)
backend\pipeline\segmenters\yolo_hybrid.py:492:            success =
extract_video_chunk(video_path, w_start, w_dur, w_path, reencode=True)
tests\test_tracknet_hybrid.py:31:    fn =
patch("backend.pipeline.segmenters.tracknet_hybrid.extract_video_chunk")(fn)
backend\providers\gemini.py:14:MAX_UPLOAD_MB = 500
backend\providers\gemini.py:39:    # 0. Refuse oversize uploads (Gemini File API limit is
2GB; we cap lower for safety/speed)
backend\providers\gemini.py:44:        if size_mb > MAX_UPLOAD_MB:
backend\providers\gemini.py:45:            logger.error(f"[{log_prefix}] {video_path} is
{size_mb:.0f}MB > {MAX_UPLOAD_MB}MB cap ? ")
tests\test_video.py:3:from backend.utils.video import get_video_duration, extract_video_chunk
tests\test_video.py:18:def test_extract_video_chunk_success(mock_run):
tests\test_video.py:19:    success = extract_video_chunk("in.mp4", 10.0, 5.0, "out.mp4")
tests\test_video.py:24:def test_extract_video_chunk_failure(mock_run):
tests\test_video.py:26:    success = extract_video_chunk("in.mp4", 10.0, 5.0, "out.mp4")
tests\test_yolo_hybrid.py:46:@patch("backend.pipeline.segmenters.yolo_hybrid.extract_video_chunk
")
tests\test_yolo_hybrid.py:92:@patch("backend.pipeline.segmenters.yolo_hybrid.extract_video_chunk
")
tests\test_yolo_hybrid.py:112:@patch("backend.pipeline.segmenters.yolo_hybrid.extract_video_chun
k")
tests\test_yolo_hybrid.py:182:@patch("backend.pipeline.segmenters.yolo_hybrid.extract_video_chun
k")
tests\test_yolo_hybrid.py:233:@patch("backend.pipeline.segmenters.yolo_hybrid.extract_video_chun
k")

```

14. user (2026-06-10T11:18:38.637Z)

```

1      """Per-video observability reports for the indexer.
2
3      `obsreport` is a SOFT dependency: if it is not installed, this module degrades to a
4      no-op and the pipeline runs exactly as before. The single entrypoint
5      `emit_indexer_report(...)` is designed to be called from a `finally` block and
6      NEVER raises ? emitting a report must not be able to break a processing run.
7      """
8      from __future__ import annotations
9
10     import os
11     from typing import Any, Dict, List, Optional
12
13     try: # soft dependency ? never hard-fail the pipeline on a reporting import
14         import obsreport # noqa: F401
15         from . import harvest
16
17         OBSREPORT_AVAILABLE = True
18     except Exception: # pragma: no cover - exercised only when obsreport is absent
19         OBSREPORT_AVAILABLE = False
20
21
22     def report_enabled(config: Optional[Dict[str, Any]]) -> bool:
23         return bool((config or {}).get("report", {}).get("enabled", True))
24
25
26     def emit_indexer_report(

```

```

27     *,
28     db,
29     video_id: str,
30     video_path: Optional[str],
31     config: Optional[Dict[str, Any]],
32     val_results: List[Any],
33     val_context: Optional[Dict[str, Any]] = None,
34     segmenter_requested: Optional[str] = None,
35     segmenter_actual: Optional[str] = None,
36     was_reingest: bool = False,
37     segmentation_ran: bool = True,
38 ) -> Optional[Dict[str, str]]:
39     """Harvest + write the report. Returns the written file paths, or None if
40     reporting is unavailable/disabled/failed. Never raises."""
41     config = config or {}
42     # Accept either a plain dict or a typed config object (e.g.
43     backend.config.AppConfig).
44     # Normalize to a plain nested dict so harvest's dict access works uniformly.
45     if hasattr(config, "model_dump"):
46         try:
47             config = config.model_dump(mode="json")
48         except Exception:
49             config = {}
50     if not OBSREPORT_AVAILABLE or not report_enabled(config):
51         return None
52     try:
53         play_segments = db.query_play_segments(video_id, {}) if db is not None else []
54         candidates = db.query_candidate_segments(video_id) if db is not None else []
55         rec = harvest.record_run(
56             video_id=video_id,
57             video_path=video_path,
58             config=config,
59             val_results=val_results,
60             val_context=val_context,
61             play_segments=play_segments,
62             candidates=candidates,
63             segmenter_requested=segmenter_requested,
64             segmenter_actual=segmenter_actual,
65             was_reingest=was_reingest,
66             segmentation_ran=segmentation_ran,
67         )
68         output_dir = (config.get("report") or {}).get("output_dir")
69         res = rec.write(output_dir=output_dir)
70         return res.paths or None
71     except Exception: # pragma: no cover - defensive; reporting must never break a run
72         return None

```

15. user (2026-06-10T11:18:38.916Z)

```

1     import pytest
2     from unittest.mock import MagicMock, patch
3     from backend.pipeline.segmenters.gemini import GeminiPlaySegmenter
4
5     @patch("backend.pipeline.segmenters.gemini.provider_registry")
6     @patch("backend.pipeline.segmenters.gemini.get_video_duration")
7     @patch("os.environ.get")
8     def test_gemini_segmenter_single_file(mock_env_get, mock_get_dur,
9     mock_provider_registry):
10         mock_env_get.return_value = "dummy_api_key"
11         mock_get_dur.return_value = 100.0 # 100s video
12
13         mock_provider = MagicMock()
14         mock_provider.analyze_video.return_value = ([
15             {"start_time": 10.0, "end_time": 20.0, "shuttle_exchange_count": 5}

```

```

15     ], {"upload_time": 1, "process_time": 1, "gen_time": 1})
16     mock_provider_registry.get.return_value = mock_provider
17
18     db_mock = MagicMock()
19     db_mock.add_play_segment.return_value = "segment_1"
20
21     config = {
22         "indexing": {
23             "chunk_duration": 300, # Large chunk duration, falls back to single file
24             "chunk_overlap": 30
25         }
26     }
27
28     segmenter = GeminiPlaySegmenter(db_mock, config)
29     res = segmenter.process_video("v1", "dummy.mp4")
30
31     assert len(res) == 1
32     assert res[0]["id"] == "segment_1"
33     assert res[0]["start_time"] == 10.0
34     assert res[0]["end_time"] == 20.0
35     assert res[0]["metadata"]["shot_count"] == 5
36
37     @patch("backend.pipeline.segmenters.gemini.provider_registry")
38     @patch("backend.pipeline.segmenters.gemini.get_video_duration")
39     @patch("backend.pipeline.segmenters.gemini.extract_video_chunk")
40     @patch("os.environ.get")
41     def test_gemini_segmenter_chunked(mock_env_get, mock_extract, mock_get_dur,
mock_provider_registry):
42         mock_env_get.return_value = "dummy_api_key"
43         mock_get_dur.return_value = 200.0 # 200s video
44         mock_extract.return_value = True
45
46         mock_provider = MagicMock()
47         # Chunk 1 returns 1 segment, Chunk 2 returns another
48         def analyze_video_side_effect(path, prompt, chunk_duration, label, **kwargs):
49             if "Chunk 1" in label:
50                 return [{"start_time": 10.0, "end_time": 20.0}], {}
51             else:
52                 return [{"start_time": 30.0, "end_time": 40.0}], {}
53
54         mock_provider.analyze_video.side_effect = analyze_video_side_effect
55         mock_provider_registry.get.return_value = mock_provider
56
57         db_mock = MagicMock()
58         db_mock.add_play_segment.side_effect = ["segment_1", "segment_2"]
59
60         config = {
61             "indexing": {
62                 "chunk_duration": 150, # Smaller than video, causes chunking
63                 "chunk_overlap": 10
64             }
65         }
66
67         segmenter = GeminiPlaySegmenter(db_mock, config)
68         res = segmenter.process_video("v1", "dummy.mp4")
69
70         # Should be 2 segments
71         assert len(res) == 2
72
73         # 1st chunk start is 0
74         assert res[0]["start_time"] == 10.0
75         # 2nd chunk start is 150-10 = 140
76         # So its local time 30 -> 140+30 = 170.0
77         # Let's just check length, exact timing requires checking array order (which may
vary due to concurrent futures)

```

```
78         times = [r["start_time"] for r in res]
79         assert 10.0 in times
80         assert 170.0 in times
81
```

16. assistant (2026-06-10T11:18:55.015Z)

The provider's oversize signal needs a path to the report. Let me see how main.py orchestrates segmentation ? report emission.

17. user (2026-06-10T11:18:56.278Z)

```
33:from backend.reporting import emit_indexer_report
108:def process_video(req: ProcessRequest):
128:    segmenter_actual = None
157:        segmenter_actual = seg_name
162:        result = segmenter.process_video(video_id, req.video_path, req.player_pool,
req.max_frames)
191:        paths = emit_indexer_report(
194:            segmenter_requested=req.segmenter, segmenter_actual=segmenter_actual,
407:        segmenter_actual = None
429:            segmenter_actual = seg_name
432:            result = segmenter.process_video(video_id, args.video_path,
max_frames=args.max_frames)
446:            paths = emit_indexer_report(
449:                segmenter_requested=args.segmenter, segmenter_actual=segmenter_actual,
```

18. user (2026-06-10T11:19:02.369Z)

```
100
101
102     # --- API Endpoints ---
103     @app.get("/api/config")
104     def get_config():
105         return global_config
106
107     @app.post("/api/process")
108     def process_video(req: ProcessRequest):
109         allowed_root = getattr(getattr(global_config, "security", None),
"allowed_video_root", None)
110         try:
111             validate_input_path(req.video_path, allowed_root=allowed_root)
112         except ValueError as e:
113             raise HTTPException(status_code=400, detail=str(e))
114         if not os.path.exists(req.video_path):
115             raise HTTPException(status_code=404, detail=f"Video file not found at
'{req.video_path}'")
116
117         video_id = compute_video_id(req.video_path)
118         was_reingest = db_instance.get_video(video_id) is not None
119
120         # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for the
report)
121         logger.info(f"Running validations for {req.video_path}...")
122         val_results, val_context = registry.run_all_with_context(req.video_path,
global_config)
123         failures = [r for r in val_results if not r.passed]
124
125         # typed AppConfig: attribute access for the default segmenter (with safe fallback)
126         default_seg = getattr(getattr(global_config, "indexing", None), "default_segmenter",
"motion")
127         seg_name = req.segmenter or default_seg
128         segmenter_actual = None
129         segmentation_ran = False
```

```

130     response: Optional[Dict[str, Any]] = None
131     try:
132         if failures:
133             # add_video is an UPSERT (no cascade) ? a failed re-validation only updates
134             # status; it no longer destroys the video's prior good segments.
135             db_instance.add_video(video_id, req.video_path, "failed", [r.dict() for r in
val_results])
136             response = {
137                 "video_id": video_id,
138                 "status": "failed",
139                 "message": "Video failed validation checks.",
140                 "results": [r.dict() for r in val_results],
141             }
142             return response
143
144         # 2. Run segmenter & indexer
145         print(f"[API] Video passed checks. Segmenting and indexing...")
146         db_instance.add_video(video_id, req.video_path, "processed", [r.dict() for r in
val_results])
147
148         segmenter_cls = segmenter_registry.get(seg_name)
149         if not segmenter_cls:
150             available = list(segmenter_registry.list_available().keys())
151             raise HTTPException(
152                 status_code=400,
153                 detail=f"Segmenter '{seg_name}' not found. Available segmenters:
{available}"
154             )
155
156         print(f"[API] Using segmenter: {seg_name}")
157         segmenter_actual = seg_name
158         # Destroy-AFTER-confirm: remember the pre-run rows, then keep new + drop old
159         only
160         # if the run yields segments; otherwise drop the new partial rows and keep the
old.
161         play_wm, cand_wm = db_instance.segment_watermarks(video_id)
162         segmenter = segmenter_cls(db_instance, global_config)
163         result = segmenter.process_video(video_id, req.video_path, req.player_pool,
req.max_frames)
164         segmentation_ran = True
165
166         if isinstance(result, Failure):
167             print(f"[API] Segmenter failed: {result.message}")
168             db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
169             response = {
170                 "video_id": video_id,
171                 "status": "failed",
172                 "message": f"Segmenter '{seg_name}' failed: {result.message}",
173                 "results": [r.dict() for r in val_results],
174                 "play_segments": [],
175             }
176             return response
177
178         segments = result.value if hasattr(result, "value") else result
179         _finalize_reingest(video_id, segments, play_wm, cand_wm)
180
181         response = {
182             "video_id": video_id,
183             "status": "success",
184             "message": f"Successfully indexed {len(segments)} play segments using
'{seg_name}' segmenter.",
185             "results": [r.dict() for r in val_results],
186             "play_segments": segments,
187         }
188         return response

```

```

188         finally:
189             # Always emit the per-video observability report (next to the video, or to
190             # report.output_dir). Never raises. Surface the paths in the response.
191             paths = emit_indexer_report(
192                 db=db_instance, video_id=video_id, video_path=req.video_path,
193                 config=global_config,
194                 val_results=val_results, val_context=val_context,
195                 segmenter_requested=req.segmenter, segmenter_actual=segmenter_actual,
196                 was_reingest=was_reingest, segmentation_ran=segmentation_ran,
197             )
198             if paths and isinstance(response, dict):
199                 response["reports"] = paths
200
201     @app.post("/api/query")
202     def query_play_segments(req: QueryRequest):
203         filters = {
204             "server": req.server,
205             "receiver": req.receiver,
206             "duration_min": req.duration_min,
207             "event_type": req.event_type
208         }
209         segments = db_instance.query_play_segments(req.video_id, filters)
210         return {"play_segments": segments}

```

19. user (2026-06-10T11:19:31.496Z)

C:\Users\avidu\Projects\sports-obsreport\src\obsreport__init__.py
Python 3.13.13

20. user (2026-06-10T11:19:34.273Z)

C:\Users\avidu\Projects\sports-obsreport\src\obsreport\render__init__.py
C:\Users\avidu\Projects\sports-obsreport\src\obsreport\render\json_render.py
C:\Users\avidu\Projects\sports-obsreport\src\obsreport\io.py
C:\Users\avidu\Projects\sports-obsreport\src\obsreport\diff.py
C:\Users\avidu\Projects\sports-obsreport\src\obsreport__init__.py
C:\Users\avidu\Projects\sports-obsreport\src\obsreport\envelope.py
C:\Users\avidu\Projects\sports-obsreport\src\obsreport\rules.py
C:\Users\avidu\Projects\sports-obsreport\src\obsreport\recorder.py
C:\Users\avidu\Projects\sports-obsreport\src\obsreport\spec.py
C:\Users\avidu\Projects\sports-obsreport\src\obsreport\render\markdown.py
C:\Users\avidu\Projects\sports-obsreport\src\obsreport\version.py

21. assistant (2026-06-10T11:19:42.144Z)

`obsreport` is installed (sibling repo, editable). Let me check its rule-op vocabulary.

22. user (2026-06-10T11:19:43.261Z)

```

1:"""The rule engine: evaluate a spec's rules against an envelope to derive flags.
4:for every operator except ``is_missing`` ? so a typo or an absent stage can never
11:from .envelope import Flag, ReportEnvelope
20:    def __repr__(self) -> str: # pragma: no cover - cosmetic
27:def to_rule_context(env: ReportEnvelope) -> Dict[str, Any]:
28:    """Project an envelope into an ergonomic dict for dotted-path lookups.
56:    """Navigate ``data`` by a dotted path; return MISSING if any hop is absent.
79:def _ordered(op: str, a: Any, b: Any) -> bool:
81:    if op == "gt":
83:    if op == "ge":
85:    if op == "lt":
87:    if op == "le":
96:    op = clause.op
98:    if op == "is_missing":
100:    if op == "is_present":

```

```

103:         # Every other operator is False against a missing path (no accidental fires).
106:     if op == "eq":
108:     if op == "ne":
110:     if op in ("gt", "ge", "lt", "le"):
111:         return _ordered(op, actual, clause.value)
112:     if op == "in":
117:     if op == "contains":
122:     if op in ("eq_path", "ne_path"):
126:         return actual == other if op == "eq_path" else actual != other
135: def _evidence(ctx: Dict[str, Any], rule: Rule) -> Dict[str, Any]:
145: def apply_rules(env: ReportEnvelope, spec: ReportSpec) -> List[Flag]:
146:     """Return the flags a spec's rules produce for an envelope (no mutation)."""
165: def run_spec(env: ReportEnvelope, spec: ReportSpec) -> ReportEnvelope:

```

23. user (2026-06-10T11:20:31.181Z)

```

1     from abc import ABC, abstractmethod
2     from typing import List, Dict, Any, Type, Optional
3     from backend.infrastructure.database import Database
4     from backend.results import Result # new standardized contract (Result[Value] ==
Success[Value] | Failure)
5
6     class BasePlaySegmenter(ABC):
7         def __init__(self, db: Database, config: Dict[str, Any]):
8             self.db = db
9             self.config = config
10
11         @property
12         @abstractmethod
13         def name(self) -> str:
14             """Unique identifier for this segmenter."""
15             pass
16
17         @property
18         @abstractmethod
19         def description(self) -> str:
20             """Human-readable explanation of this segmenter."""
21             pass
22
23         @abstractmethod
24         def process_video(self, video_id: str, video_path: str, player_pool: List[str] =
None, max_frames: int = None) -> "Result[List[Dict[str, Any]]]":
25             """
26             Processes a video to detect play segments, populates SQLite database,
27             and returns a Result.
28
29             Use isinstance(result, Failure) (or result.is_success()) to distinguish
30             "no segments found" from "something went wrong" (missing key, healthcheck
31             failure, etc.). This is the start of the standardized error/result contract.
32             """
33             pass
34
35     class SegmenterRegistry:
36         def __init__(self):
37             self._segmenters: Dict[str, Type[BasePlaySegmenter]] = {}
38
39         def register(self, segmenter_cls: Type[BasePlaySegmenter]):
40             """Registers a new segmenter class under its name property."""
41             try:
42                 temp_inst = segmenter_cls(None, {})
43                 name = temp_inst.name
44             except Exception as e:
45                 raise ValueError(f"Failed to resolve name property from segmenter class
{segmenter_cls.__name__}: {e}")
46             self._segmenters[name] = segmenter_cls

```



```

47
48     def get(self, name: str) -> Optional[Type[BasePlaySegmenter]]:
49         return self._segmenters.get(name)
50
51     def list_available(self) -> Dict[str, str]:
52         """Returns a dict of registered segmenter names and their descriptions."""
53         res = {}
54         for name, cls in self._segmenters.items():
55             try:
56                 temp_inst = cls(None, {})
57                 desc = temp_inst.description
58             except Exception:
59                 desc = "No description available."
60             res[name] = desc
61         return res
62
63     segmenter_registry = SegmenterRegistry()
64

```

24. user (2026-06-10T11:20:32.341Z)

```

tests\test_reporting_harvest.py
tests\test_reporting.py
tests\test_reporting_spec_faq.py

```

25. user (2026-06-10T11:20:33.356Z)

```

42: class Rule(BaseModel):
46:     severity: Severity
49:     faq_ref: Optional[str] = None
51:     customer_message: Optional[str] = None # if set, this finding also shows (gently) to
customers
55:     if self.severity in ("error", "warn") and not self.faq_ref:
57:         f"rule '{self.code}' has severity '{self.severity}' but no faq_ref "

```

26. user (2026-06-10T11:20:41.496Z)

```

backend\config\models.py:6:The models support graceful migration from old config.json files via
extra="allow".
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\models.py-7-"""
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\models.py-8-
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\models.py-9-from __future__
import annotations
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\models.py-10-
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\models.py-11-from typing
import Any, Literal, Optional
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\models.py-12-
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\models.py-13-from pydantic
import BaseModel, Field, field_validator, model_validator
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\models.py-14-
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\models.py-15-Sport =
Literal["badminton", "tennis", "pickleball", "table_tennis", "cricket"]
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\models.py-16-
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\models.py-17-
backend\config\models.py-18-class ValidationRelevanceConfig(BaseModel):
--
backend\config\models.py:51:     because nested config sections defaulted to extra='ignore'.
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\models.py-52-     """
backend\config\models.py-53-     wsl_distro: str = "Ubuntu"
backend\config\models.py-54-     repo_dir: str = "~/models/WASB-SBDT"
backend\config\models.py-55-     conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
backend\config\models.py-56-     conda_env: str = "wasb"
backend\config\models.py-57-     weights_path: str = "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"

```

```

backend\config\models.py-58- sport: str = "badminton"
backend\config\models.py-59- wsl_stage_dir: str = "~/clips"
backend\config\models.py-60- timeout_sec: int = 1800 # 30 min; kills a hung WSL/GPU call (0
= no timeout)
backend\config\models.py-61- velocity_threshold: float = 0.02
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\models.py-62-
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\models.py-63-
--
backend\config\models.py:130:class AppConfig(BaseModel):
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\models.py-131- """Root
configuration object.
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\models.py-132-
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\models.py-133- All runtime
configuration should be accessed via an instance of this class
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\models.py-134- (or the
loader) rather than raw dict lookups.
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\models.py-135- """
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\models.py-136-
backend\config\models.py-137- sport: Sport = "badminton"
backend\config\models.py-138- ai_provider: str = "gemini"
backend\config\models.py-139- # Auto-tracking alias: 2.5-flash is legacy now that 3.5-flash
is the stable flagship;
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\models.py-140- # the alias
never deprecates underneath us. Eval reports record the run date alongside.
backend\config\models.py-141- ai_model: str = "gemini-flash-latest"
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\models.py-142-
--
backend\config\models.py:158: # via model_dump, so legacy chained access keeps working ?
returning the Pydantic
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\models.py-159- # section
model instead breaks it (a model has no .get), which bit every validator on
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\models.py-160- # real
runs. Scalar leaves are returned as-is.
backend\config\models.py:161: def get(self, key: str, default: Any = None) -> Any:
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\models.py-162-
_MISSING = object()
backend\config\models.py-163- val: Any = _MISSING
backend\config\models.py-164- if key in type(self).model_fields or hasattr(self, key):
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\models.py-165- val
= getattr(self, key, _MISSING)
backend\config\models.py-166- if val is _MISSING:
backend\config\models.py-167- # Legacy convenience: find a leaf key inside any nested
section. Sections are
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\models.py-168- #
discovered dynamically from the model fields, so a future top-level section
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\models.py-169- #
(e.g. `audio`, `jobs`) is covered automatically without editing a hardcoded list.
backend\config\models.py-170- for fname in type(self).model_fields:
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\models.py-171-
section = getattr(self, fname, None)
backend\config\models.py-172- if isinstance(section, BaseModel) and
hasattr(section, key):
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\models.py-173-
val = getattr(section, key)
--
backend\config\models.py:178: return val.model_dump(mode="json")
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\models.py-179- return
val
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\models.py-180-
backend\config\models.py-181- def __getitem__(self, key: str) -> Any:
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\models.py-182- val =
self.get(key)
backend\config\models.py-183- if val is None and key not in type(self).model_fields:
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\models.py-184- #
Re-raise like a real dict for unknown top-level

```

```

C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\models.py-185-
raise KeyError(key)
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\models.py-186-           return
val
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\models.py-187-
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\models.py-188-
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\models.py-189-# Convenience
type alias for callers that want to annotate
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\models.py-190-Config =
AppConfig
--
backend\config\loader.py:28:     return AppConfig().model_dump(mode="json")
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\loader.py-29-
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\loader.py-30-
backend\config\loader.py-31-def load_config(path: str | Path | None = None) -> AppConfig:
backend\config\loader.py-32-    """Load configuration with the following precedence:
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\loader.py-33-
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\loader.py-34-    1. Built-in
defaults defined in the Pydantic models.
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\loader.py-35-    2. Values
from the JSON file (if present and readable).
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\loader.py-36-    3. (Future)
environment / CLI overrides.
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\loader.py-37-
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\loader.py-38-    Missing file
or unreadable file ? returns a fully defaulted AppConfig.
backend\config\loader.py:39:     Unknown keys in the file are tolerated (extra="allow" on the
model).
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\loader.py-40-    """
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\loader.py-41-    cfg_path =
Path(path) if path is not None else DEFAULT_CONFIG_PATH
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\loader.py-42-
backend\config\loader.py-43-    file_data: dict[str, Any] = {}
backend\config\loader.py-44-    if cfg_path.exists():
backend\config\loader.py-45-        try:
backend\config\loader.py-46-            with open(cfg_path, "r", encoding="utf-8") as f:
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\loader.py-47-
file_data = json.load(f)
backend\config\loader.py-48-        except Exception as exc:
backend\config\loader.py-49-            # Non-fatal: continue with defaults + whatever we could
parse.
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\loader.py-50-            # In
production we might log here.
backend\config\loader.py-51-            logger.warning(f"Failed to read {cfg_path}: {exc}. Using
defaults + partial data.")
--
backend\config\loader.py:68:     cfg_path.write_text(cfg.model_dump_json(indent=4),
encoding="utf-8")
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\loader.py-69-     return
cfg_path
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\loader.py-70-
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\loader.py-71-
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\loader.py-72-# Backwards-
compatible alias used by some older call sites during transition.
backend\config\loader.py:73:def get_default_config() -> dict[str, Any]:
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\loader.py-74-    """Return
defaults as a plain dict (for transitional use only)."""
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\loader.py-75-     return
_get_builtin_defaults()

```

27. user (2026-06-10T11:20:50.939Z)

```

1     """Tests for the indexer observability harvest + footgun rules.
2
3     These exercise the broken-run scenarios the report is meant to make debuggable.

```

```

4      obsreport is a soft dependency; skip cleanly if it isn't installed.
5      """
6      import os
7
8      import pytest
9
10     pytest.importorskip("obsreport")
11
12     from backend.reporting import emit_indexer_report, harvest # noqa: E402
13     from backend.pipeline.validators.base import ValidationResult # noqa: E402
14
15
16     class _FakeDB:
17         def __init__(self, play=None, candidates=None, existing=False):
18             self._play = play or []
19             self._candidates = candidates or []
20             self._existing = existing
21
22         def query_play_segments(self, video_id, filters):
23             return list(self._play)
24
25         def query_candidate_segments(self, video_id, detector=None, only_unlabeled=False):
26             return list(self._candidates)
27
28         def get_video(self, video_id):
29             return {"id": video_id} if self._existing else None
30
31
32     def _vr(name, passed, message="ok"):
33         return ValidationResult(validator_name=name, passed=passed, message=message)
34
35
36     _FFPROBE_OK = {
37         "streams": [{"width": 1920, "height": 1080, "r_frame_rate": "60000/1001",
38 "codec_name": "hevc"}],
39         "format": {"format_name": "mov,mp4,m4a", "duration": "676.0", "size": "55000000000"},
40     }
41
42     def _record(**kw):
43         base = dict(
44             video_id="md5abc",
45             video_path=None,
46             config={"ai_provider": "gemini", "ai_model": "gemini-flash", "indexing":
{"default_segmenter": "wasb_hybrid"}},
47             val_results=[_vr("format_check", True), _vr("relevance_check", True)],
48             val_context={"ffprobe_meta": _FFPROBE_OK},
49             play_segments=[{"duration": 5.0, "metadata": {"detector": "wasb-hybrid-
gemini"}}],
50             candidates=[{"id": 1}, {"id": 2}],
51             segmenter_requested="wasb_hybrid",
52             segmenter_actual="wasb_hybrid",
53             was_reingest=False,
54             segmentation_ran=True,
55         )
56         base.update(kw)
57         return harvest.record_run(**base).envelope
58
59
60     def _codes(env):
61         return {f.code for f in env.flags}
62
63
64     def test_healthy_run_is_clean(monkeypatch):
65         monkeypatch.setenv("GEMINI_API_KEY", "x")

```

```

66     env = _record()
67     assert env.verdict == "pass"
68     assert _codes(env) == set()
69     assert env.video_meta.fps == 59.94
70     assert env.video_meta.fps_source == "probed"
71     seg = env.get_stage("segmentation")
72     assert {m.key: m.value for m in seg.metrics}["segment_count"] == 1
73     assert {m.key: m.value for m in seg.metrics}["candidate_windows"] == 2
74     assert env.lint() == []
75
76
77 def test_silent_provider_noop_fires_when_no_key_and_zero_segments(monkeypatch):
78     monkeypatch.delenv("GEMINI_API_KEY", raising=False)
79     env = _record(play_segments=[])
80     assert "silent_provider_noop" in _codes(env)
81     assert env.verdict == "fail"
82     flag = next(f for f in env.flags if f.code == "silent_provider_noop")
83     assert flag.faq_ref == "README#Q7"
84
85
86 def test_ai_empty_warns_when_key_present_but_zero_segments(monkeypatch):
87     monkeypatch.setenv("GEMINI_API_KEY", "x")
88     env = _record(play_segments=[])
89     codes = _codes(env)
90     assert "ai_empty_vs_no_rallies" in codes
91     assert "silent_provider_noop" not in codes
92     assert env.verdict == "pass_with_warnings"
93
94
95 def test_motion_segmenter_flags_synthetic_metadata(monkeypatch):
96     monkeypatch.delenv("GEMINI_API_KEY", raising=False)
97     env = _record(segmenter_requested="motion", segmenter_actual="motion",
98                   config={"indexing": {"default_segmenter": "motion"}})
99     codes = _codes(env)
100    assert "synthetic_motion_metadata" in codes
101    # motion is not AI-based -> the provider-noop rule must NOT fire even with no key
102    assert "silent_provider_noop" not in codes
103
104
105 def test_segmenter_divergence_fires_only_without_explicit_override(monkeypatch):
106     monkeypatch.setenv("GEMINI_API_KEY", "x")
107     cfg = {"ai_provider": "gemini", "indexing": {"default_segmenter": "wasb_hybrid"}}
108     # No explicit request, but a different segmenter ran -> a fallback diverged: flag
109     it.
110     env = _record(segmenter_requested=None, segmenter_actual="gemini", config=cfg)
111     assert "segmenter_divergence" in _codes(env)
112     # Explicit --segmenter override that differs from config default is NOT a
113     divergence.
114     env2 = _record(segmenter_requested="gemini", segmenter_actual="gemini", config=cfg)
115     assert "segmenter_divergence" not in _codes(env2)
116
117
118 def test_reingest_info_flag(monkeypatch):
119     monkeypatch.setenv("GEMINI_API_KEY", "x")
120     env = _record(was_reingest=True)
121     assert "reingest_replaced_prior" in _codes(env)
122
123
124 def test_validation_failure_halts_and_flags(monkeypatch):
125     monkeypatch.setenv("GEMINI_API_KEY", "x")
126     env = _record(
127         val_results=[_vr("format_check", True), _vr("relevance_check", False, "court
color not found")],
128         play_segments=[],
129         segmenter_actual=None,

```

```

128         segmentation_ran=False,
129     )
130     assert env.get_stage("validation").status == "error"
131     assert env.get_stage("segmentation").status == "not_run"
132     assert "validation_failed" in _codes(env)
133     assert env.verdict == "fail"
134
135
136     def test_fps_fallback_flag_when_no_probe_but_segmented(monkeypatch):
137         monkeypatch.setenv("GEMINI_API_KEY", "x")
138         env = _record(val_context={}, segmentation_ran=True)
139         assert env.video_meta.fps_source == "fallback"
140         assert "fps_fallback_used" in _codes(env)
141
142
143     def test_emit_writes_files_next_to_video(tmp_path, monkeypatch):
144         monkeypatch.setenv("GEMINI_API_KEY", "x")
145         video = tmp_path / "match.mp4"
146         video.write_bytes(b"not really a video")
147         db = _FakeDB(play=[{"duration": 4.0, "metadata": {"detector": "wasb-hybrid"}}],
candidates=[{"id": 1}])
148         paths = emit_indexer_report(
149             db=db, video_id="md5abc", video_path=str(video),
150             config={"ai_provider": "gemini", "indexing": {"default_segmenter":
"wasb_hybrid"}},
151             val_results=[_vr("format_check", True)], val_context={"ffprobe_meta":
_FFPROBE_OK},
152             segmenter_requested="wasb_hybrid", segmenter_actual="wasb_hybrid",
153             was_reingest=False, segmentation_ran=True,
154         )
155         assert paths is not None
156         assert os.path.exists(paths["json"])
157         assert (tmp_path / "match.report.md").exists()
158         assert (tmp_path / "match.summary.md").exists()
159
160
161     def test_emit_accepts_typed_appconfig(tmp_path, monkeypatch):
162         """Reporting must work whether main passes a plain dict or the typed AppConfig
163         (backend.config) ? the reporting layer normalizes AppConfig -> dict."""
164         from backend.config import AppConfig
165
166         monkeypatch.setenv("GEMINI_API_KEY", "x")
167         video = tmp_path / "m.mp4"
168         video.write_bytes(b"x")
169         db = _FakeDB(play=[{"duration": 4.0, "metadata": {"detector": "wasb-hybrid"}}],
candidates=[{"id": 1}])
170         cfg = AppConfig() # typed config object, not a dict
171         paths = emit_indexer_report(
172             db=db, video_id="md5cfg", video_path=str(video), config=cfg,
173             val_results=[_vr("format_check", True)], val_context={"ffprobe_meta":
_FFPROBE_OK},
174             segmenter_requested="wasb_hybrid", segmenter_actual="wasb_hybrid",
175             was_reingest=False, segmentation_ran=True,
176         )
177         assert paths is not None
178         assert os.path.exists(paths["json"])
179
180
181     def test_emit_disabled_via_config(tmp_path, monkeypatch):
182         monkeypatch.setenv("GEMINI_API_KEY", "x")
183         db = _FakeDB()
184         paths = emit_indexer_report(
185             db=db, video_id="md5abc", video_path=str(tmp_path / "m.mp4"),
186             config={"report": {"enabled": False}},
187             val_results=[_vr("format_check", True)], val_context={},

```

```
188         segmenter_actual="motion", segmentation_ran=True,
189     )
190     assert paths is None
191
```

28. user (2026-06-10T11:20:51.189Z)

```
1     import pytest
2     from unittest.mock import MagicMock, patch
3     from backend.providers.gemini import GeminiProvider
4
5     def test_gemini_provider_success():
6         # Mock GenAI client
7         with patch("backend.providers.gemini.genai") as mock_genai:
8             client_mock = MagicMock()
9             mock_genai.Client.return_value = client_mock
10
11             # Mock file upload
12             file_mock = MagicMock()
13             file_mock.uri = "gs://dummy"
14             file_mock.name = "dummy_file"
15             file_mock.state.name = "ACTIVE"
16
17             client_mock.files.upload.return_value = file_mock
18             client_mock.files.get.return_value = file_mock
19
20             # Mock generate_content response
21             response_mock = MagicMock()
22             response_mock.text = '["start_time": 1.0, "end_time": 2.0]']
23             client_mock.models.generate_content.return_value = response_mock
24
25             provider = GeminiProvider(api_key="dummy_api_key",
model_name="gemini-2.5-flash")
26             segments, metrics = provider.analyze_video("dummy.mp4", "dummy prompt",
chunk_duration=10.0)
27
28             assert len(segments) == 1
29             assert segments[0]["start_time"] == 1.0
30             assert metrics["upload_time"] >= 0
31             assert metrics["process_time"] >= 0
32             assert metrics["gen_time"] >= 0
33
34             # Ensure delete is called
35             client_mock.files.delete.assert_called_with(name="dummy_file")
36
37     def test_gemini_provider_invalid_json():
38         with patch("backend.providers.gemini.genai") as mock_genai:
39             client_mock = MagicMock()
40             mock_genai.Client.return_value = client_mock
41             file_mock = MagicMock()
42             file_mock.state.name = "ACTIVE"
43             client_mock.files.upload.return_value = file_mock
44             client_mock.files.get.return_value = file_mock
45
46             response_mock = MagicMock()
47             response_mock.text = '{"not_a_list": true}'
48             client_mock.models.generate_content.return_value = response_mock
49
50             provider = GeminiProvider(api_key="dummy_api_key",
model_name="gemini-2.5-flash")
51             segments, metrics = provider.analyze_video("dummy.mp4", "dummy prompt")
52             assert len(segments) == 0
53
54     def test_gemini_provider_rejects_oversize_before_upload(monkeypatch):
55         with patch("backend.providers.gemini.genai") as mock_genai:
```

```

56         client_mock = MagicMock()
57         mock_genai.Client.return_value = client_mock
58         # 600MB > MAX_UPLOAD_MB (500) -> refuse without ever uploading
59         monkeypatch.setattr("backend.providers.gemini.os.path.getsize",
60                             lambda p: 600 * 1024 * 1024)
61         provider = GeminiProvider(api_key="k", model_name="gemini-flash-latest")
62         segments, metrics = provider.analyze_video("big.mp4", "p")
63         assert segments == []
64         client_mock.files.upload.assert_not_called()
65
66
67     def test_gemini_provider_upload_fail():
68         with patch("backend.providers.gemini.genai") as mock_genai:
69             client_mock = MagicMock()
70             mock_genai.Client.return_value = client_mock
71             client_mock.files.upload.side_effect = Exception("Upload err")
72
73             provider = GeminiProvider(api_key="dummy_api_key",
model_name="gemini-2.5-flash")
74             segments, metrics = provider.analyze_video("dummy.mp4", "dummy prompt")
75             assert len(segments) == 0
76

```

29. assistant (2026-06-10T11:21:48.437Z)

I now have the full design. Let me check where the docs record this gotcha (CODE_MAP / README FAQ) so the docs stay truthful, then implement.

30. user (2026-06-10T11:21:50.483Z)

```

docs\EVALUATION.md:38:  silently skipped ? so labeling mistakes surface immediately.
docs\CODE_MAP.md:132:- `@backend/eval/calibrate_wasb.py` ? WASB tuning CLI.
`::BASELINE=(0.02,2.0,2.0)`, `::default_grid` (45 cfigs), `::make_predict_fn` (parses trajectory
once, closes over `trajectory_to_action_windows`), `::load_golden_gts` ($`start_time,end_time`
cols; ? SILENTLY skips bad rows ? opposite of annotations.load_annotations). ?
`{load_golden_gts, make_predict_fn, BASELINE}` imported by 4 downstream drivers
(calibrate_local, export_wasb_segments, gemini_refine, audio/e1_probe).
docs\CODE_MAP.md:152:- `@backend/tools/rally_slicer.py` ? standalone CLI.
`::compute_clip_windows` (pure, unit-tested), `::load_rally_labels` (golden schema; ? silently
skips degenerate rows), `::slice_rallies`, `::ClipWindow`(rally:str),
`::BEGIN_PADDING=0.5`/`::END_PADDING=1.0` (? duplicate compiler's by copy). `_read_time` ->
backend/eval/annotations.parse_timestamp`.
docs\CODE_MAP.md:168:[Omitted long matching line]
docs\CODE_MAP.md:175:[Omitted long matching line]
docs\CODE_MAP.md:204:- golden/slicer (`calibrate_wasb.load_golden_gts`,
`rally_slicer.load_rally_labels`, `export.SEG_FIELDS`):
`rally_number,start_time,end_time[,duration,start_mmss,end_mmss]`; SILENTLY skips bad rows.
docs\CODE_MAP.md:328:| `test_providers.py` / `test_provider_gemini.py` | `providers` registry +
GeminiProvider lifecycle (`MAX_UPLOAD_MB` guard, non-list JSON -> [], cleanup delete) |

```

31. user (2026-06-10T11:21:51.223Z)

```

backend\eval\gemini_refine.py:73:                                     clip_scale_width: Optional[int] = 1280) ->
Tuple[int, Interval, List[Interval]]:
backend\eval\gemini_refine.py:83:     if not extract_video_chunk(video, cs, ce - cs, clip,
reencode=True, scale_width=clip_scale_width):
backend\eval\gemini_refine.py:104:         min_dur: float = 2.0, limit: int = 0,
clip_scale_width: Optional[int] = 1280) -> dict:
backend\eval\gemini_refine.py:123:                                     tmpdir, i, clip_scale_width)
backend\eval\gemini_refine.py:150:                                     out_csv: str =
"output\gemini_fullvideo.csv", clip_scale_width: Optional[int] = 1280,
backend\eval\gemini_refine.py:171:         if not extract_video_chunk(video, cstart, cend -
cstart, clip, reencode=True, scale_width=clip_scale_width):
backend\eval\gemini_refine.py:184:     print(f"[gemini_full] {len(starts)} chunk(s) of {video}

```



```
(<= {chunk_sec}s, width {clip_scale_width}) "
backend\eval\gemini_refine.py:213:    ap.add_argument("--clip-scale-width", type=int,
default=1280,
backend\eval\gemini_refine.py:219:    scale = args.clip_scale_width or None
backend\eval\gemini_refine.py:222:                                clip_scale_width=scale,
min_dur=args.min_dur, max_workers=args.max_workers)
backend\eval\gemini_refine.py:229:    min_dur=args.min_dur, limit=args.limit,
clip_scale_width=scale)
```

32. user (2026-06-10T11:22:00.062Z)

```
160     - `@backend/pipeline/validators/resolution_fps.py::ResolutionFPSValidator`
(`resolution_fps_check`) ? CONSUMES `ffprobe_meta` (? hard-fails "Run format_check first" if
absent). `min_fps` 29.9; min res 1280x720.
161     - `@backend/pipeline/validators/pts_continuity.py::PTSContinuityValidator`
(`pts_continuity_check`) ? re-probes (no context). ? 10.0s keyframe-gap threshold hardcoded
(comment says 8.0s ? drift, NOT config-driven); any backward jump fails; `<2` keyframes PASSES
(too short).
162     - `@backend/pipeline/validators/scene_cut.py::SceneCutDensityValidator`
(`scene_cut_check`) ? OpenCV. ? `cut_threshold=0.6` hardcoded (NOT config); samples ~2fps, caps
at 100 SAMPLED frames (~first 50s); `max_scene_cuts_per_minute` (0.5) is the config fail
threshold.
163     - `@backend/pipeline/validators/blur.py::BlurValidator` (`blur_check`) ? Laplacian var <
`min_blur_threshold` (default 100.0 in-file, 20.0 in models/config) = fail; samples 15 frames. ?
magic absolute, not resolution-normalized.
164     - `@backend/pipeline/validators/relevance.py::RelevanceValidator` (`relevance_check`) ?
HSV court-green ratio (? purely color, no geometry despite name); lazy `import backend.sports`;
3-tier config precedence (`validation.relevance` -> sport profile -> hardcoded); ? unknown sport
-> empty cfg, falls back to defaults, does NOT fail.
165
166     ### 2.6 Sports / Providers / Annotations ?
167     - `@backend/sports/base.py::SportProfile` ABC
(`name`, `get_prompt`, `get_relevance_config`);
`@backend/sports/badminton.py::BadmintonSportProfile` (the Gemini prompt + HSV relevance cfg);
`@backend/sports/__init__.py::sport_registry` (+`::SportRegistry`, auto-registers badminton; ?
`register` instantiates profile to read `name`). ? **add a sport**: subclass `SportProfile`,
`sport_registry.register(MyProfile)`. $ prompt emits JSON array `{start_time(float DECIMAL SEC),
end_time, shuttle_exchange_count(int), shot_count_confidence('Low'|'Medium'|'High')}`. ? prompt
actively defends against MM:SS; `get()` returns a fresh instance each call.
168     - `@backend/providers/base.py::AIVideoProvider` ABC (`__init__(api_key, model_name)`);
`@backend/providers/gemini.py::GeminiProvider` (lazy `::client` property; `::MAX_UPLOAD_MB=500`;
upload->poll->generate JSON temp 0->delete); `@backend/providers/__init__.py::provider_registry`
(+`::ProviderRegistry`, auto-registers `gemini`). ? **add a provider**: subclass,
`provider_registry.register('name', MyCls)` (key passed EXPLICITLY); `get(name, api_key,
model_name)`. $ `analyze_video(path, prompt, chunk_duration=0.0, label='')` -> (segments:list,
metrics:dict{upload_time,process_time,gen_time})`. ? blocking 10s poll; returns `([], metrics)`
on ANY failure (empty != error); oversize clip silently skipped; orphaned remote file possible
if process dies mid-run.
169     - `@backend/annotations/base.py::AnnotationProvider` ABC (`submit/poll/fetch/annotate`,
`::Interval` (= metrics.Interval), `::QUEUED/RUNNING/DONE/FAILED`/`::STATUSES`,
`::AnnotationProviderRegistry` (decorator-friendly `register` returns the class; ? keys off
`provider_cls().name`, falls back to `__name__` if no-arg ctor throws; `list_available()` ->
LIST not dict), `::annotation_registry`. ? **add a tier**: subclass (define `name`),
`annotation_registry.register(MyCls)` at module bottom + import in `__init__.py`. $ `fetch` ->
List[Interval] sorted; contract says RAISE (not `[]`) on unavailable.
170     - `@backend/annotations/file_provider.py::FileProvider` (`file`; job_id IS the
resolved CSV path; `_resolve` -> `

```

```

173
174     ### 2.7 Infrastructure (DB) ?
175     - `@backend/infrastructure/database.py::Database` ? SQLite, 4 tables
    (`videos/play_segments/events/candidate_segments`), `PRAGMA user_version` migration ladder in
    `::init_db` (currently =1, `version<1` creates `candidate_segments`). `::get_connection` (?
    re-issues `PRAGMA foreign_keys=ON` per connection ? else CASCADE/SET NULL silently skipped).
    Methods: `add_video`, `add_play_segment`, `add_event`, `add_candidate_segment`, `link_candidate_to_s
    egment`, `query_candidate_segments(detector=,only_unlabeled=)`, `query_play_segments`, `get_video`,
    `clear_video_data`. ? candidate_segments is the detector-agnostic fine-tuning extension point
    (common fields columns, detector-specifics in `stats`/`detection_params` JSON; index
    `idx_candidates_video_detector`). ? `add_video` is `INSERT OR REPLACE` -> cascade-DELETES
    dependent segments on re-ingest; events column is `type` not `event_type`; `query_play_segments`
    filters use truthiness so `duration_min=0`/empty-string = "no filter"; write helpers swallow
    exceptions.
176
177     ### 2.8 Reporting / observability ? (per-video, soft-dep `obsreport`)
178     - `@backend/reporting/__init__.py` ? `::emit_indexer_report(*, db, video_id, video_path,
    config, val_results, val_context=None, segmenter_requested=, segmenter_actual=,
    was_reingest=False, segmentation_ran=True)` ? single endpoint, keyword-only. Normalizes typed
    config (`config.model_dump(...)` if `hasattr model_dump`); short-circuits to None if
    unavailable/disabled; pulls play_segments/candidates from `db` (read-only
    `query_play_segments(vid,{})`/`query_candidate_segments(vid)`); delegates to
    `harvest.record_run`; writes via `rec.write(output_dir=config["report"]["output_dir"])`. ? wraps
    everything in bare `except Exception: return None` ? **NEVER raises** (designed for `main.py`'s
    `finally:`); writes `

```

33. user (2026-06-10T11:22:16.211Z)

```

?? artifacts/
?? scratch/HIGH_RISK_REMEDIATION_PLAN.md
Switched to a new branch 'fix/gemini-4k-oversize-chunks'

```

34. user (2026-06-10T11:22:24.995Z)

<tool_use_error>File has not been read yet. Read it first before writing to it.</tool_use_error>

35. assistant (2026-06-10T11:22:21.107Z)

Now the config knob:

36. user (2026-06-10T11:22:31.082Z)

```
60         timeout_sec: int = 1800 # 30 min; kills a hung WSL/GPU call (0 = no timeout)
61         velocity_threshold: float = 0.02
62
63
64     class IndexingConfig(BaseModel):
65         default_segmenter: str = "yolo_hybrid"
66         use_gpu: bool = True
67         motion_threshold: float = 0.08
68         min_segment_duration: float = 3.0
69         dead_time_merge_gap: float = 2.0
70         chunk_duration: float = 90.0
71         chunk_overlap: float = 10.0
72         detector_model: str = "fasterrcnn_resnet50_fpn"
73         detector_score_threshold: float = 0.5
74         yolo_velocity_threshold: float = 0.15
75         yolo_frame_skip: int = 3
76         yolo_tracker: str = "bytetrack.yaml"
77         yolo_prefetch_workers: int = 2
78         min_action_window_duration: float = 2.0
79         log_yolo_candidates: bool = True
80         store_yolo_candidates: bool = True
81         ai_handoff_padding: float = 2.0
82         ai_max_workers: int = 5
83         # Optional debug knob: trim every video to the first N seconds before processing.
84         debug_duration: Optional[float] = None
85
86         tracknet: TrackNetConfig = Field(default_factory=TrackNetConfig)
87         wasb: WasbIndexConfig = Field(default_factory=WasbIndexConfig)
88
89         @field_validator("default_segmenter")
```

37. user (2026-06-10T11:22:40.612Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\models.py has been updated successfully. (file state is current in your context ? no need to Read it back)

38. user (2026-06-10T11:22:51.339Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\backend\providers\gemini.py has been updated successfully. (file state is current in your context ? no need to Read it back)

39. assistant (2026-06-10T11:22:56.235Z)

Now the new run-signals channel:

40. assistant (2026-06-10T11:23:01.993Z)

""In-process signal channel: pipeline internals -> the per-video report.

Some report-worthy facts (e.g. the provider refusing an oversize chunk) surface deep inside a segmenter's worker threads, with no return path to the harvest ? segmenters return only their segment list. Layers record those facts here keyed by video_id; `emit\_indexer\_report` pops them when the run's report is built.

Process-local and best-effort BY DESIGN: recording must never break a run, the store is consumed (popped) once per report, and signals for a video are simply

```

dropped if no report is emitted. No obsreport dependency ? importable everywhere.
"""
from __future__ import annotations

import threading
from typing import Any, Dict

_lock = threading.Lock()
_signals: Dict[str, Dict[str, Any]] = {}

```

```

def incr(video_id: str, key: str, by: int = 1) -> None:
    """Add `by` to the counter `key` for this video's in-flight run."""
    if not video_id or by == 0:
        return
    with _lock:
        sig = _signals.setdefault(video_id, {})
        sig[key] = int(sig.get(key, 0)) + by

def pop(video_id: str) -> Dict[str, Any]:
    """Consume (remove and return) all signals recorded for this video's run."""
    with _lock:
        return _signals.pop(video_id, {})

```

41. user (2026-06-10T11:23:02.044Z)

File created successfully at: C:\Users\avidu\Projects\badminton-highlight-indexer\backend\reporting\run_signals.py (file state is current in your context ? no need to Read it back)

42. user (2026-06-10T11:23:12.231Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\segmenters\gemini.py has been updated successfully. (file state is current in your context ? no need to Read it back)

43. user (2026-06-10T11:23:22.969Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\segmenters\gemini.py has been updated successfully. (file state is current in your context ? no need to Read it back)

44. user (2026-06-10T11:23:31.381Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\segmenters\gemini.py has been updated successfully. (file state is current in your context ? no need to Read it back)

45. user (2026-06-10T11:23:47.217Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\segmenters\gemini.py has been updated successfully. (file state is current in your context ? no need to Read it back)

46. user (2026-06-10T11:23:55.675Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\segmenters\gemini.py has been updated successfully. (file state is current in your context ? no need to Read it back)

47. assistant (2026-06-10T11:24:01.991Z)

Now plumb the signals through `emit\_indexer\_report` and the harvest:

48. user (2026-06-10T11:24:04.957Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\backend\reporting__init__.py has been updated successfully. (file state is current in your context ? no need to Read it back)

49. user (2026-06-10T11:24:13.491Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\backend\reporting__init__.py has been updated successfully. (file state is current in your context ? no need to Read it back)

50. user (2026-06-10T11:24:19.782Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\backend\reporting__init__.py has been updated successfully. (file state is current in your context ? no need to Read it back)

51. assistant (2026-06-10T11:24:26.034Z)

Now the harvest ? record the count in the `ai\_handoff` stage details:

52. user (2026-06-10T11:24:37.769Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\backend\reporting\harvest.py has been updated successfully. (file state is current in your context ? no need to Read it back)

53. user (2026-06-10T11:24:46.177Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\backend\reporting\harvest.py has been updated successfully. (file state is current in your context ? no need to Read it back)

54. user (2026-06-10T11:24:52.873Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\backend\reporting\harvest.py has been updated successfully. (file state is current in your context ? no need to Read it back)

55. assistant (2026-06-10T11:24:58.126Z)

Now the spec rule:

56. user (2026-06-10T11:25:05.329Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\backend\reporting\spec.yaml has been updated successfully. (file state is current in your context ? no need to Read it back)

57. assistant (2026-06-10T11:25:15.077Z)

Now the tests. First, let me check what the spec-FAQ test validates so the new rule passes it:

58. user (2026-06-10T11:25:15.230Z)

```
1      """Golden guard: every footgun rule's faq_ref must resolve to a real README heading.
2
3      This is the committed backbone of report debuggability ? if a rule cites README#Qn,
4      that section must exist, so a reader who follows the pointer always lands somewhere.
5      (A fresh-reader eval found a rule citing the wrong-but-existing section; an existence
6      test can't catch *semantic* mismatch, but it locks the floor: no dangling anchors.)
7      """
8      import re
9      from pathlib import Path
10
11     import pytest
12
13     pytest.importorskip("obsreport")
14     from obsreport import load_spec # noqa: E402
```

```

15
16     ROOT = Path(__file__).resolve().parent.parent
17     SPEC = ROOT / "backend" / "reporting" / "spec.yaml"
18     README = ROOT / "README.md"
19
20
21     def _headings(md: str):
22         return [ln.lstrip("#").strip() for ln in md.splitlines() if
ln.lstrip().startswith("#")]
23
24
25     def _slug(h: str) -> str:
26         s = re.sub(r"^[^w\s-]", "", h.lower())
27         return re.sub(r"\s+", "-", s.strip())
28
29
30     def _anchor_resolves(anchor: str, headings) -> bool:
31         a = anchor.lower()
32         for h in headings:
33             hl = h.lower()
34             if hl == a or hl.startswith(a + ":") or hl.startswith(a + " ") or
hl.startswith(a + "."):
35                 return True # Q-number style: "Q7" -> "### Q7: ..."
36             if _slug(h) == a:
37                 return True # full GitHub slug match
38         return False
39
40
41     def test_every_rule_faq_ref_resolves_to_a_readme_heading():
42         spec = load_spec(SPEC)
43         headings = _headings(README.read_text(encoding="utf-8"))
44         unresolved = []
45         for rule in spec.rules:
46             if not rule.faq_ref:
47                 continue
48             file_part, _, anchor = rule.faq_ref.partition("#")
49             if file_part in ("README", "README.md"):
50                 if not _anchor_resolves(anchor, headings):
51                     unresolved.append((rule.code, rule.faq_ref))
52             elif file_part in ("VIDEO_RECORDING_GUIDELINES",
"VIDEO_RECORDING_GUIDELINES.md",
53                               "AUDIO_RALLY_DETECTION_PLAN",
"AUDIO_RALLY_DETECTION_PLAN.md"):
54                 # These are dedicated guidance docs surfaced from report rules (Input
Quality
55                 # extension). The whole document is the destination; no #Q anchor required.
56                 pass
57             else:
58                 unresolved.append((rule.code, rule.faq_ref))
59         assert not unresolved, f"faq_ref anchors with no matching README heading:
{unresolved}"
60
61
62     def test_actionable_rules_have_faq_refs():
63         """error/warn findings must cite a FAQ (info may omit). Belt-and-suspenders on top
64         of obsreport's own spec-load validation."""
65         spec = load_spec(SPEC)
66         missing = [r.code for r in spec.rules if r.severity in ("error", "warn") and not
r.faq_ref]
67         assert not missing, f"actionable rules missing faq_ref: {missing}"
68

```

59. assistant (2026-06-10T11:25:32.704Z)

`README#Q7` already resolves (existing rules cite it). Now the tests ? first check the config

test file and how `run\_all\_tests.py` invokes tests:

60. user (2026-06-10T11:25:33.737Z)

```
tests\test_config.py
tests\test_config_wasb.py
```

61. user (2026-06-10T11:25:34.102Z)

```
1      #!/usr/bin/env python
2      """Run the FULL test suite for the badminton-highlight-indexer.
3
4      A single, discoverable entry point so you (or a reviewer / CI) can verify the
5      whole suite before opening a PR:
6
7          python run_all_tests.py          # full suite, quiet
8          python run_all_tests.py -v       # verbose (passthrough to pytest)
9          python run_all_tests.py -k config # filter by keyword (passthrough)
10         python run_all_tests.py --cov=backend --cov-report=term-missing # coverage
11
12     Anything after the script name is passed straight through to pytest.
13
14     Notes
15     -----
16     - The suite is fully mocked (no GPU, no network, no WSL) and runs offline.
17     - The observability **reporting** tests skip cleanly if the optional ``obsreport``
18       package isn't installed (they use ``pytest.importorskip``). To exercise them too:
19         pip install -e ../sports-obsreport --no-build-isolation
20     - Exit code is pytest's own, so this is CI / pre-PR friendly (0 = all green).
21     """
22     from __future__ import annotations
23
24     import importlib.util
25     import subprocess
26     import sys
27
28
29     def _installed(module: str) -> bool:
30         try:
31             return importlib.util.find_spec(module) is not None
32         except (ImportError, ValueError):
33             return False
34
35
36     def main() -> int:
37         args = list(sys.argv[1:])
38         # Default to quiet unless the caller chose a verbosity.
39         if not any(a in ("-q", "-v", "-vv", "-vvv", "--quiet", "--verbose") for a in args):
40             args = ["-q", *args]
```

62. user (2026-06-10T11:25:42.865Z)

```
1: """indexing.wasb.* and the other live indexing knobs must actually take effect.
4:nested sections silently dropped unknown keys and the whole indexing.wasb block (the live
7:from backend.config.models import AppConfig, IndexingConfig
8:from backend.pipeline.detectors.wasb_runner import WasbConfig
11:def test_wasb_override_reaches_runner():
12:     cfg = AppConfig(indexing={"wasb": {
16:         assert idx["wasb"]["weights_path"] == "/custom/weights.pth"
17:         assert idx["wasb"]["velocity_threshold"] == 0.09
25:     fields = IndexingConfig.model_fields
28:     assert "wasb" in fields
36:     fields = IndexingConfig.model_fields
```

63. user (2026-06-10T11:25:49.082Z)

```
1      """indexing.wasb.* and the other live indexing knobs must actually take effect.
2
3      Regression for the nested-config-severance finding: only AppConfig had extra='allow', so
4      nested sections silently dropped unknown keys and the whole indexing.wasb block (the
live
5      shuttle substrate's config) was discarded ? config.json overrides did nothing.
6      """
7      from backend.config.models import AppConfig, IndexingConfig
8      from backend.pipeline.detectors.wasb_runner import WasbConfig
9
10
11     def test_wasb_override_reaches_runner():
12         cfg = AppConfig(indexing={"wasb": {
13             "weights_path": "/custom/weights.pth", "velocity_threshold": 0.09, "conda_env":
"myenv",
14         }})
15         idx = cfg.get("indexing")
16         assert idx["wasb"]["weights_path"] == "/custom/weights.pth"
17         assert idx["wasb"]["velocity_threshold"] == 0.09
18         # And the runner's config builder actually picks it up (previously always
defaulted).
19         wcfg = WasbConfig.from_indexing_cfg(idx)
20         assert wcfg.weights_path == "/custom/weights.pth"
21         assert wcfg.conda_env == "myenv"
22
23
24     def test_live_indexing_knobs_declared():
25         fields = IndexingConfig.model_fields
26         assert "yolo_prefetch_workers" in fields
27         assert "debug_duration" in fields
28         assert "wasb" in fields
29         cfg = AppConfig(indexing={"yolo_prefetch_workers": 7, "debug_duration": 30})
30         idx = cfg.get("indexing")
31         assert idx["yolo_prefetch_workers"] == 7
32         assert idx["debug_duration"] == 30
33
34
35     def test_dead_knobs_removed():
36         fields = IndexingConfig.model_fields
37         assert "yolo_parallel_chunks" not in fields
38         assert "yolo_chunk_workers" not in fields
39
```

64. assistant (2026-06-10T11:25:57.845Z)

Now I'll write the tests. First, extend the config knob guard:

65. user (2026-06-10T11:26:04.490Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_config_wasb.py has been updated successfully. (file state is current in your context ? no need to Read it back)

66. assistant (2026-06-10T11:26:10.182Z)

Now the segmenter tests ? re-encode args and the oversize signal path:

67. user (2026-06-10T11:26:12.095Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_gemini.py has been updated successfully. (file state is current in your context ? no need to Read it back)

68. user (2026-06-10T11:26:38.983Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_gemini.py has been updated successfully. (file state is current in your context ? no need to Read it back)

69. assistant (2026-06-10T11:26:56.697Z)

Now extend the provider oversize test:

70. user (2026-06-10T11:27:03.153Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_provider_gemini.py has been updated successfully. (file state is current in your context ? no need to Read it back)

71. user (2026-06-10T11:27:11.571Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_provider_gemini.py has been updated successfully. (file state is current in your context ? no need to Read it back)

72. user (2026-06-10T11:27:19.555Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_reporting_harvest.py has been updated successfully. (file state is current in your context ? no need to Read it back)

73. user (2026-06-10T11:27:35.471Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_reporting_harvest.py has been updated successfully. (file state is current in your context ? no need to Read it back)

74. assistant (2026-06-10T11:27:47.712Z)

Quick check that the envelope's stage exposes `details` as a dict:

75. user (2026-06-10T11:27:49.075Z)

```
42:class Stage(BaseModel):
50:     details: Dict[str, Any] = Field(default_factory=dict)
```

76. user (2026-06-10T11:27:56.086Z)

```
115:- `@backend/pipeline/segmenters/gemini.py::GeminiPlaySegmenter` (name `gemini`) ? pure-AI,
no P2. ? chunks only if `duration>chunk_duration`; `max_workers=5` hardcoded;
`max_segment_duration=90.0` hardcoded; `debug_duration` silently truncates; `parse_time` accepts
colon timestamps (warns); does NOT persist candidate segments (only hybrids feed the fine-tuning
table).
301:| `test_gemini.py` | `pipeline/segmenters/gemini.GeminiPlaySegmenter` (single vs chunked
path, per-chunk offset, `shot_count`<=`shuttle_exchange_count`) |
```

77. user (2026-06-10T11:28:12.899Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\docs\CODE_MAP.md has been updated successfully. (file state is current in your context ? no need to Read it back)

78. user (2026-06-10T11:28:20.556Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\docs\CODE_MAP.md has been updated successfully. (file state is current in your context ? no need to Read it back)

79. user (2026-06-10T11:28:29.521Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\docs\CODE_MAP.md has been updated successfully. (file state is current in your context ? no need to Read it back)

80. user (2026-06-10T11:28:41.509Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\docs\CODE_MAP.md has been updated successfully. (file state is current in your context ? no need to Read it back)

81. user (2026-06-10T11:28:49.998Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\docs\CODE_MAP.md has been updated successfully. (file state is current in your context ? no need to Read it back)

82. user (2026-06-10T11:29:02.086Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\docs\CODE_MAP.md has been updated successfully. (file state is current in your context ? no need to Read it back)

83. user (2026-06-10T11:29:39.423Z)

```
=====
Full test suite: C:\Users\avidu\AppData\Local\Programs\Python\Python313\python.exe -m pytest
-q
obsreport (optional reporting dep): available
=====
..... [ 21%]
..... [ 43%]
..... [ 64%]
..... [ 86%]
..... [100%]
===== warnings summary =====
tests/test_api_endpoints.py::test_process_success_with_mocked_segmenter
  C:\Users\avidu\Projects\badminton-highlight-indexer\backend\main.py:146:
PydanticDeprecatedSince20: The `dict` method is deprecated; use `model_dump` instead. Deprecated
in Pydantic V2.0 to be removed in V3.0. See Pydantic V2 Migration Guide at
https://errors.pydantic.dev/2.13/migration/
    db_instance.add_video(video_id, req.video_path, "processed", [r.dict() for r in
val_results])

tests/test_api_endpoints.py::test_process_success_with_mocked_segmenter
  C:\Users\avidu\Projects\badminton-highlight-indexer\backend\main.py:184:
PydanticDeprecatedSince20: The `dict` method is deprecated; use `model_dump` instead. Deprecated
in Pydantic V2.0 to be removed in V3.0. See Pydantic V2 Migration Guide at
https://errors.pydantic.dev/2.13/migration/
    "results": [r.dict() for r in val_results],

tests/test_api_endpoints.py::test_process_validation_failure_preserves_prior_segments
  C:\Users\avidu\Projects\badminton-highlight-indexer\backend\main.py:135:
PydanticDeprecatedSince20: The `dict` method is deprecated; use `model_dump` instead. Deprecated
in Pydantic V2.0 to be removed in V3.0. See Pydantic V2 Migration Guide at
https://errors.pydantic.dev/2.13/migration/
    db_instance.add_video(video_id, req.video_path, "failed", [r.dict() for r in val_results])

tests/test_api_endpoints.py::test_process_validation_failure_preserves_prior_segments
  C:\Users\avidu\Projects\badminton-highlight-indexer\backend\main.py:140:
PydanticDeprecatedSince20: The `dict` method is deprecated; use `model_dump` instead. Deprecated
in Pydantic V2.0 to be removed in V3.0. See Pydantic V2 Migration Guide at
https://errors.pydantic.dev/2.13/migration/
    "results": [r.dict() for r in val_results],

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
333 passed, 4 warnings in 23.53s
=====
RESULT: PASS - all tests green
=====
```

84. assistant (2026-06-10T11:29:47.196Z)

All 333 tests green (8 new). Committing and opening the PR:

85. user (2026-06-10T11:29:50.254Z)

```
warning: in the working copy of 'backend/reporting/run_signals.py', LF will be replaced by CRLF
the next time Git touches it
M backend/config/models.py
M backend/pipeline/segmenters/gemini.py
M backend/providers/gemini.py
M backend/reporting/__init__.py
M backend/reporting/harvest.py
A backend/reporting/run_signals.py
M backend/reporting/spec.yaml
M docs/CODE_MAP.md
M tests/test_config_wasb.py
M tests/test_gemini.py
M tests/test_provider_gemini.py
M tests/test_reporting_harvest.py
?? artifacts/
?? scratch/HIGH_RISK_REMEDIATION_PLAN.md
```

86. user (2026-06-10T11:30:06.322Z)

```
[fix/gemini-4k-oversize-chunks 64265aa] fix(gemini): re-encode AI chunks (4K oversize skip =
silent 0-segments) + loud report flag
12 files changed, 271 insertions(+), 14 deletions(-)
create mode 100644 backend/reporting/run_signals.py
```

87. user (2026-06-10T11:30:15.559Z)

```
remote:
remote: Create a pull request for 'fix/gemini-4k-oversize-chunks' on GitHub by visiting:
remote:      https://github.com/avidullu/badminton-highlight-
indexer/pull/new/fix/gemini-4k-oversize-chunks
remote:
branch 'fix/gemini-4k-oversize-chunks' set up to track 'origin/fix/gemini-4k-oversize-chunks'.
To https://github.com/avidullu/badminton-highlight-indexer.git
* [new branch]      fix/gemini-4k-oversize-chunks -> fix/gemini-4k-oversize-chunks
```

88. user (2026-06-10T11:30:42.609Z)

```
https://github.com/avidullu/badminton-highlight-indexer/pull/80
```

89. assistant (2026-06-10T11:30:51.126Z)

```
[PR #80](https://github.com/avidullu/badminton-highlight-indexer/pull/80) is open. Updating the
current-state checkpoint memory per the working agreement:
```

90. user (2026-06-10T11:31:00.439Z)

```
<tool_use_error>File has not been read yet. Read it first before writing to it.</tool_use_error>
```

91. user (2026-06-10T11:31:09.987Z)

```
1      ---
2      name: current-state
3      description: "Live micro-checkpoint state for Claude ? updated every checkpoint (not
only on request). What just landed, what's next, open decisions."
4      metadata:
5        node_type: memory
6        type: project
7        originSessionId: b13673cc-f087-43b1-9527-35dc0b67b301
8      ---
9
10     # Current state ? updated continuously (see [[working-agreement-checkpoints]])
11
```

```
12      **Checkpoint:** 2026-06-10 (M0 GREENLIT SEQUENCE EXECUTED ? session wrap). Owner
ratified ADR-008 ("split driven by
13      productionization, not isolation") + greenlit M0.4+M0.1. **All 5 steps shipped, every PR
merged on verified-green CI:**
14      - **#73 ADR-008** (DECISIONS.md): training in-repo behind an executable wall; artifact-
bundle boundary; ship-gate
```

92. user (2026-06-10T11:31:25.152Z)

The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\current-state.md has been updated successfully. (file state is current in your context ? no need to Read it back)

93. user (2026-06-10T11:32:22.740Z)

```
Full test suite (offline, mocked)      pending 0      https://github.com/avidullu/badminton-
highlight-indexer/actions/runs/27273171484/job/80547559365
Syntax sweep + import smoke (no GPU stack)      pass      20s
https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27273171484/job/80547559257
Refreshing checks status every 20 seconds. Press Ctrl+C to quit.
```

```
Full test suite (offline, mocked)      pending 0      https://github.com/avidullu/badminton-
highlight-indexer/actions/runs/27273171484/job/80547559365
Syntax sweep + import smoke (no GPU stack)      pass      20s
https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27273171484/job/80547559257
Full test suite (offline, mocked)      pass      1m28s      https://github.com/avidullu/badminton-
highlight-indexer/actions/runs/27273171484/job/80547559365
Syntax sweep + import smoke (no GPU stack)      pass      20s
https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27273171484/job/80547559257
Full test suite (offline, mocked)      pass      1m28s      https://github.com/avidullu/badminton-
highlight-indexer/actions/runs/27273171484/job/80547559365
Syntax sweep + import smoke (no GPU stack)      pass      20s
https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27273171484/job/80547559257
```

94. assistant (2026-06-10T11:32:40.127Z)

****Done ? [PR #80](https://github.com/avidullu/badminton-highlight-indexer/pull/80) is open with both CI jobs green**** (import-smoke 20s, full suite 1m28s). Suite went 325 ? 333 green locally. Not merging ? that's your call per convention.

What shipped, in three layers:

****1. The actual fix ? chunks are re-encoded, not stream-copied.****
[gemini.py](backend/pipeline/segmenters/gemini.py) now calls `extract_video_chunk(..., reencode=True, scale_width=?)` driven by a new knob `indexing.ai_chunk_scale_width`, ****declared**** in [models.py](backend/config/models.py) (so it survives the nested-config key-dropping footgun). Default 1280, matching `gemini_refine`'s `--clip-scale-width`; `0` restores legacy stream-copy. I went with "always re-encode for AI handoff" rather than a bitrate heuristic ? it's simpler, the upload size becomes predictable, and it fixes the keyframe-drift gotcha at chunk boundaries for free. A 4K 90s chunk drops from ~1.3GB to tens of MB.

****2. The skip can never be silent again.**** The provider keeps its refusal guard (defense in depth) but now sets `metrics["oversize_skipped"] = True` alongside a clearer error. The segmenter aggregates the count on both the chunked and single-file paths into a `logger.error` ("N/M chunks SKIPPED WITHOUT ANALYSIS?") and records it in a new thread-safe channel, [run_signals.py](backend/reporting/run_signals.py) ? needed because the fact originates in worker threads with no return path to the report harvest.

****3. The per-video report flags it.**** `emit_indexer_report` pops the signals (unconditionally, so nothing accumulates), [harvest.py](backend/reporting/harvest.py) records `ai_handoff.details.oversize_clips_skipped`, and a new ERROR rule `oversize_clip_skipped` in [spec.yaml](backend/reporting/spec.yaml) tells the reader segments are *missing* for those time ranges ? with the config fix inline. I confirmed obsreport's rule engine supports `op: gt` before writing the rule.

Tests cover each layer plus end-to-end emit consumption, and CODE_MAP is truth-synced (I also corrected its stale "max_workers=5 hardcoded" claim ? that was fixed by #79).

One thing for your machine-side verification once merged: re-run the 4K video ? chunk sizes in the GEMINI CHUNKING SUMMARY table should read ~20?60MB, and the actual rally battery is still gated on the Gemini billing top-up from task 17.

95. user (2026-06-10T11:32:42.732Z)

Please make a PR and merge the fix ensuring test coverage and CI enhancement if needed

96. assistant (2026-06-10T11:33:09.524Z)

I'll verify the CI setup first to judge whether an enhancement is needed, then merge on a verified-green rollup.

97. user (2026-06-10T11:33:10.303Z)

```
1      name: CI
2
3      on:
4        push:
5          branches: [master]
6        pull_request:
7
8      permissions:
9        contents: read
10
11     concurrency:
12       group: ci-${{ github.ref }}
13       cancel-in-progress: true
14
15     jobs:
16       # Fast guard against the 7fbbeb0 class of breakage: a SyntaxError or
17       # module-level import error shipping while the suite stays green.
18       # Deliberately does NOT install cv2/torch/numpy ? the smoke tests must hold
19       # on machines without the GPU stack (test_import_smoke stubs what's absent).
20       import-smoke:
21         name: Syntax sweep + import smoke (no GPU stack)
22         runs-on: ubuntu-latest
23         steps:
24           - uses: actions/checkout@v4
25           - uses: actions/setup-python@v5
26             with:
27               python-version: "3.13"
28           - name: Install minimal deps
29             run: python -m pip install pytest fastapi uvicorn pydantic python-dotenv google-
genai
30           - name: Run import smoke tests
31             run: python -m pytest tests/test_import_smoke.py -v
32
33       full-suite:
34         name: Full test suite (offline, mocked)
35         runs-on: ubuntu-latest
36         steps:
37           - uses: actions/checkout@v4
38           - uses: actions/setup-python@v5
39             with:
40               python-version: "3.13"
41               cache: pip
42           - name: System libs for opencv-python
43             run: |
44               sudo apt-get update
45               sudo apt-get install -y libgl1 libglu1-mesa-dev || sudo apt-get install -y
```

```

libgl1 libgl2.0-0
46         - name: Install CPU-only torch (no CUDA wheels on a CPU runner)
47         run: python -m pip install torch torchvision --index-url
https://download.pytorch.org/whl/cpu
48         - name: Install requirements
49         run: python -m pip install -r requirements.txt pytest httpx # httpx: fastapi
TestClient
50         # obsreport (private, soft dep) is absent here: reporting tests skip via
51         # pytest.importorskip ? run_all_tests.py prints which.
52         - name: Run full suite
53         run: python run_all_tests.py
54

```

98. assistant (2026-06-10T11:33:53.383Z)

```

"""run_signals + its consumption contract must hold WITHOUT obsreport.

```

The reporting rule tests (test_reporting_harvest.py) skip when the soft dep is absent ? as it is in CI (obsreport is a private sibling repo). These tests pin the CI-visible half of the oversize-skip channel: the collector itself and emit_indexer_report's UNCONDITIONAL consumption (no cross-run accumulation), neither of which needs obsreport.

```

from backend.reporting import emit_indexer_report, run_signals

```

```

def test_incr_and_pop_are_consuming_and_isolated():
    run_signals.pop("vid_a")
    run_signals.pop("vid_b")
    run_signals.incr("vid_a", "oversize_clips_skipped")
    run_signals.incr("vid_a", "oversize_clips_skipped", 6)
    run_signals.incr("vid_b", "oversize_clips_skipped", 2)
    assert run_signals.pop("vid_a") == {"oversize_clips_skipped": 7}
    assert run_signals.pop("vid_a") == {} # pop consumes
    assert run_signals.pop("vid_b") == {"oversize_clips_skipped": 2} # per-video isolation

```

```

def test_incr_ignores_empty_video_id_and_zero_increment():
    run_signals.incr("", "k")
    run_signals.incr("vid_c", "k", 0)
    assert run_signals.pop("") == {}
    assert run_signals.pop("vid_c") == {}

```

```

def test_emit_consumes_signals_even_when_reporting_is_off(tmp_path):
    """Reporting disabled (or obsreport absent, as in CI) must still consume the
run's signals ? the process-local store must never accumulate finished runs."""
    run_signals.incr("vid_d", "oversize_clips_skipped", 3)
    res = emit_indexer_report(
        db=None, video_id="vid_d", video_path=str(tmp_path / "v.mp4"),
        config={"report": {"enabled": False}},
        val_results=[],
    )
    assert res is None
    assert run_signals.pop("vid_d") == {}

```

99. user (2026-06-10T11:33:53.395Z)

File created successfully at: C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_run_signals.py (file state is current in your context ? no need to Read it back)

100. user (2026-06-10T11:34:00.507Z)

```
325:| `test_reporting_harvest.py` | `reporting` (`emit_indexer_report`, `harvest.record_run`); ?
`importorskip("obsreport")`; all 7 footgun rules + plain-dict-OR-AppConfig +
`report.enabled=False` |
326:| `test_reporting_spec_faq.py` | `reporting/spec.yaml` <-> `README.md` cross-ref integrity
(every `faq_ref` resolves; actionable rules cite a `faq_ref`); ? `importorskip("obsreport")` |
```

101. user (2026-06-10T11:34:09.786Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\docs\CODE_MAP.md has been updated successfully. (file state is current in your context ? no need to Read it back)

102. user (2026-06-10T11:34:42.564Z)

```
-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
336 passed, 4 warnings in 23.27s
=====
RESULT: PASS - all tests green
=====
```

103. user (2026-06-10T11:34:56.139Z)

```
warning: in the working copy of 'tests/test_run_signals.py', LF will be replaced by CRLF the
next time Git touches it
[fix/gemini-4k-oversize-chunks d5b90ad] test(reporting): pin the CI-visible half of the oversize
channel (no obsreport needed)
 2 files changed, 42 insertions(+), 1 deletion(-)
 create mode 100644 tests/test_run_signals.py
remote:
remote: Create a pull request for 'fix/gemini-4k-oversize-chunks' on GitHub by visiting:
remote:      https://github.com/avidullu/badminton-highlight-
indexer/pull/new/fix/gemini-4k-oversize-chunks
remote:
To https://github.com/avidullu/badminton-highlight-indexer.git
 * [new branch]      fix/gemini-4k-oversize-chunks -> fix/gemini-4k-oversize-chunks
```

104. assistant (2026-06-10T11:35:06.502Z)

The push said "new branch" ? the remote branch may have been deleted (PR possibly merged or closed in the meantime). Checking PR state:

105. user (2026-06-10T11:35:10.246Z)

```
{"headRefName":"fix/gemini-4k-oversize-chunks","mergeCommit":{"oid":"a2e65d737e5206b91a97633e5db
14921af9437dd"},"mergedAt":"2026-06-10T11:34:36Z","state":"MERGED","title":"fix(gemini): re-
encode AI chunks so 4K sources don't silently lose every chunk to the upload cap"}
[]
```

106. user (2026-06-10T11:35:33.959Z)

Your branch is behind 'origin/master' by 1 commit, and can be fast-forwarded.

```
(use "git pull" to update your local branch)
Switched to branch 'master'
Updating 6353bfb..a2e65d7
Fast-forward
 backend/config/models.py          |  7 +++
 backend/pipeline/segmenters/gemini.py | 29 ++++++++
 backend/providers/gemini.py        |  4 +-
 backend/reporting/__init__.py      |  6 +++
 backend/reporting/harvest.py        | 12 +++--
 backend/reporting/run_signals.py    | 33 ++++++++
 backend/reporting/spec.yaml         | 21 ++++++
 docs/CODE_MAP.md                   | 13 +---
 tests/test_config_wasb.py           | 10 ++++
 tests/test_gemini.py                | 98 ++++++
```

```
tests/test_provider_gemini.py          | 11 +++-
tests/test_reporting_harvest.py        | 41 ++++++++-----
12 files changed, 271 insertions(+), 14 deletions(-)
create mode 100644 backend/reporting/run_signals.py
a2e65d7 fix(gemini): re-encode AI chunks (4K oversize skip = silent 0-segments) + loud report
flag (#80)
6353bfb fix(gemini): honor indexing.ai_max_workers (was hardcoded 5); ship serial default (#79)
4ca7526 chore(config): ai_model -> gemini-flash-latest (auto-tracking alias) (#78)
Switched to a new branch 'test/oversize-channel-ci-coverage'
[test/oversize-channel-ci-coverage 210d50c] test(reporting): pin the CI-visible half of the
oversize channel (no obsreport needed)
Date: Wed Jun 10 17:04:53 2026 +0530
2 files changed, 42 insertions(+), 1 deletion(-)
create mode 100644 tests/test_run_signals.py
```

107. user (2026-06-10T11:36:11.979Z)

336 passed, 4 warnings in 23.56s

=====

RESULT: PASS - all tests green

=====

remote:

remote: Create a pull request for 'test/oversize-channel-ci-coverage' on GitHub by visiting:

remote: <https://github.com/avidullu/badminton-highlight-indexer/pull/new/test/oversize-channel-ci-coverage>

remote:

branch 'test/oversize-channel-ci-coverage' set up to track 'origin/test/oversize-channel-ci-coverage'.

To <https://github.com/avidullu/badminton-highlight-indexer.git>

* [new branch] test/oversize-channel-ci-coverage -> test/oversize-channel-ci-coverage

To <https://github.com/avidullu/badminton-highlight-indexer.git>

- [deleted] fix/gemini-4k-oversize-chunks

108. user (2026-06-10T11:36:31.451Z)

<https://github.com/avidullu/badminton-highlight-indexer/pull/82>

109. user (2026-06-10T11:38:22.874Z)

Full test suite (offline, mocked) pass 1m30s <https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27273471191/job/80548594780>

Syntax sweep + import smoke (no GPU stack) pass 21s

<https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27273471191/job/80548594765>

Full test suite (offline, mocked) pass 1m30s <https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27273471191/job/80548594780>

Syntax sweep + import smoke (no GPU stack) pass 21s

<https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27273471191/job/80548594765>

110. assistant (2026-06-10T11:38:30.559Z)

Both jobs green. Verifying the rollup state and merging:

111. user (2026-06-10T11:38:38.147Z)

```
{"conclusion":"SUCCESS","name":"Syntax sweep + import smoke (no GPU stack)"}
```

```
{"conclusion":"SUCCESS","name":"Full test suite (offline, mocked)"}
```

112. user (2026-06-10T11:38:49.252Z)

```
{"mergedAt":"2026-06-10T11:38:37Z","state":"MERGED"}
```

M backend/providers/gemini.py

Your branch is up to date with 'origin/master'.

Already on 'master'

From <https://github.com/avidullu/badminton-highlight-indexer>


```
3377486..f01387e master -> origin/master
Updating 3377486..f01387e
Fast-forward
f01387e test(reporting): pin the CI-visible half of the oversize channel (no obsreport needed)
(#82)
3377486 test(reporting): pin the CI-visible half of the oversize channel (no obsreport needed)
(#81)
Deleted branch fix/gemini-4k-oversize-chunks (was d5b90ad).
error: branch 'test/oversize-channel-ci-coverage' not found
M backend/providers/gemini.py
?? artifacts/
?? scratch/HIGH_RISK_REMEDIATION_PLAN.md
```

113. user (2026-06-10T11:39:23.445Z)

```
{"author":"avidullu","headRefName":"test/oversize-channel-
pin","mergedAt":"2026-06-10T11:37:58Z","state":"MERGED","title":"Tests: pin the CI-visible half
of the oversize-signal channel (rescued from #80's branch)"}
commit f01387e70dde7cb4c47f1e0b79d913b6c0391675
Author: Avi <avi.dullu@gmail.com>
Date: Wed Jun 10 17:08:36 2026 +0530
```

```
test(reporting): pin the CI-visible half of the oversize channel (no obsreport needed) (#82)
```

```
CI's full-suite lane lacks obsreport (private soft dep), so the rule-firing
tests skip there. run_signals itself + emit_indexer_report's unconditional
commit 3377486f5869d088829532840ff3e8fe1f7d5913
Author: Avi <avi.dullu@gmail.com>
Date: Wed Jun 10 17:07:58 2026 +0530
```

```
test(reporting): pin the CI-visible half of the oversize channel (no obsreport needed) (#81)
```

```
CI's full-suite lane lacks obsreport (private soft dep), so the rule-firing
tests skip there. run_signals itself + emit_indexer_report's unconditional
```

114. user (2026-06-10T11:39:24.658Z)

```
warning: in the working copy of 'backend/providers/gemini.py', LF will be replaced by CRLF the
next time Git touches it
diff --git a/backend/providers/gemini.py b/backend/providers/gemini.py
index 9fcb0f0..ab9e3c9 100644
--- a/backend/providers/gemini.py
+++ b/backend/providers/gemini.py
@@ -1,5 +1,6 @@
import os
import time
+import random
import json
import logging
from typing import Tuple
@@ -104,7 +105,11 @@ class GeminiProvider(AIVideoProvider):
    logger.info(f"[{log_prefix}] Submitting multimodal request to {self.model_name}...")
    t_gen_start = time.time()
    raw_text = None
-    for attempt in range(3):
+    # 6 attempts with exponential backoff + jitter (~3s..96s, worst case ~3min total):
+    # capacity 503s on the flagship flash model come in waves that outlast the old
+    # 3x(3..9s) schedule ? observed live: 5/7 chunks of a run lost to three-strike 503s.
+    gen_attempts = 6
+    for attempt in range(gen_attempts):
        try:
            response = self.client.models.generate_content(
                model=self.model_name,
@@ -119,9 +124,9 @@ class GeminiProvider(AIVideoProvider):
            logger.info(f"[{log_prefix}] Response received in
```

```

{metrics['gen_time']:.1f}s:\n{raw_text}")
        break
    except Exception as e:
        logger.warning(f"[{log_prefix}] generate attempt {attempt + 1}/3 failed: {e}")
        if attempt < 2:
            time.sleep(3 * (attempt + 1))
        logger.warning(f"[{log_prefix}] generate attempt {attempt + 1}/{gen_attempts}
failed: {e}")
        if attempt < gen_attempts - 1:
            time.sleep(3 * (2 ** attempt) + random.uniform(0, 2))
    try:
        self.client.files.delete(name=video_file.name)
    except Exception:

```

115. user (2026-06-10T11:39:53.588Z)

```

fatal: invalid --pretty format: files:
{"conclusion":"","displayTitle":"test(reporting): pin the CI-visible half of the oversize
channel (no ?","status":"in_progress"}
{"conclusion":"cancelled","displayTitle":"test(reporting): pin the CI-visible half of the
oversize channel (no ?","status":"completed"}

```

116. user (2026-06-10T11:40:01.490Z)

117. user (2026-06-10T11:40:28.237Z)

ANNOTATIONS

! Node.js 20 actions are deprecated. The following actions are running on Node.js 20 and may not work as expected: actions/checkout@v4, actions/setup-python@v5. Actions will be forced to run with Node.js 24 by default starting June 16th, 2026. Node.js 20 will be removed from the runner on September 16th, 2026. Please check if updated versions of these actions are available that support Node.js 24. To opt into Node.js 24 now, set the FORCE_JAVASCRIPT_ACTIONS_TO_NODE24=true environment variable on the runner or in your workflow file. Once Node.js 24 becomes the default, you can temporarily opt out by setting ACTIONS_ALLOW_USE_UNSECURE_NODE_VERSION=true. For more information see: <https://github.blog/changelog/2025-09-19-deprecation-of-node-20-on-github-actions-runners/>
Syntax sweep + import smoke (no GPU stack): .github#2

118. user (2026-06-10T11:40:38.434Z)

```

{"conclusion":"success","headSha":"f01387e70dde7cb4c47f1e0b79d913b6c0391675","status":"completed"}

```

119. user (2026-06-10T11:41:03.142Z)

<tool_use_error>File has not been read yet. Read it first before writing to it.</tool_use_error>

120. user (2026-06-10T11:41:14.275Z)

```

10      # Current state ? updated continuously (see [[working-agreement-checkpoints]])
11
12      **Checkpoint:** 2026-06-10 (GEMINI BASELINE LANDED ? owner's "gradual ramp-up" theory
CORRECT). New key in gitignored
13      `.env` (rotate later ? pasted in chat). The "credits depleted" 429 was a BURST-THROTTLE
artifact: text ping + serial
14      upload worked ? **PR #79 merged**: gemini segmenter now honors `indexing.ai_max_workers`
(was hardcoded 5; config ships
15      1=serial ? at 5 workers the same video 429'd, at 1 it worked).
16      - ? **WRAPPER BASELINE (gemini-flash-latest, ~$0.2-0.5 total): Mahadevpura F1 0.93** (P
1.00 R 0.87, mIoU 0.80, ±0.7s;
17      3 of 4 misses = short rallies Gemini FOUND but our min_segment_duration=3.0 rejected)
vs Gen-0 learned 0.744 vs

```

18 heuristic 0.657. **Clean footage = commoditized.**
19 - ?? **MOAT TEST (Kushagra multi-court, windowed n=12 GT): wrapper F1 0.56** (P .54 R .58 mIoU .69) ? Gen-0 learned
20 0.561, both >> heuristic 0.157. **The multi-court confound hurts Gemini too ? the wedge survives on the target
21 footage class.** Caveats: only chunks 2+4 answered (5/7 lost to GENUINE gemini-flash-latest capacity 503s ? retry
22 off-peak for full coverage); visual-only (audio stripped in downscale); 1280p re-encode (4K chunks were 1.3GB >
23 500MB upload cap ? the audit's silent-oversize-skip fired live; fix = chip task_44e72c41 ? see PR #80 checkpoint
24 below). Kushagra preds keyed under downscaled-file MD5 (`output/kushagra\_1280.mp4`).
25 - ? **DEMO ARTIFACT: `output/MahadevpuraSingles\_highlights.mp4`** ? real 27-rally reel compiled by VideoCompiler

121. user (2026-06-10T11:41:27.317Z)

31:**Checkpoint:** 2026-06-10 (4K gemini oversize fix ? **PR #80 OPEN**, awaiting review/CI).
Root cause of the live

122. user (2026-06-10T11:41:33.558Z)

25 - ? **DEMO ARTIFACT: `output/MahadevpuraSingles\_highlights.mp4`** ? real 27-rally reel compiled by VideoCompiler
26 (the once-broken module) from Gemini segments. The academy-pitch object exists.
27 - **STRATEGY READ:** product path = Gemini for clean footage TODAY (cheap, F1 0.93) + owned model's job = multi-court
28 (0.56 is the bar to beat there, not 0.93); 503 flakiness = real reliability cost of the wrapper (provider-fallback
29 story strengthens). PENDING: gemini_refine two-stage (503-gated, run off-peak), full Kushagra re-run, PR #80 merge.
30
31 **Checkpoint:** 2026-06-10 (4K gemini oversize fix ? **PR #80 OPEN**, awaiting review/CI). Root cause of the live
32 "0 segments / success" on KushagraYashAviFirstVideo.MP4 (4K): gemini segmenter stream-copies 90s chunks ? all 7 at
33 832?1299MB > provider `MAX\_UPLOAD\_MB=500` ? every chunk silently skipped. Fix (3 layers): (1) chunks now RE-ENCODED +
34 downscaled to new declared knob `indexing.ai\_chunk\_scale\_width` (default 1280 = gemini_refine's `--clip-scale-width`;
35 0 = legacy stream-copy; also fixes keyframe drift at chunk boundaries); (2) skip is LOUD ? provider
36 `metrics["oversize\_skipped"]` + clearer error; segmenter aggregates (chunked + single-file) ? `logger.error` + NEW
37 thread-safe `backend/reporting/run\_signals` channel (incr/pop by video_id; popped unconditionally in
38 `emit\_indexer\_report`); (3) report flags it ?
39 `ai\_handoff.details.oversize\_clips\_skipped` + spec.yaml ERROR rule
40 `oversize\_clip\_skipped` (op: gt ? obsreport supports ordered ops). Suite 325? **333 green** ; CODE_MAP truth-synced
41 (incl. the stale "max_workers=5 hardcoded" claim, actually fixed by #79). ?? owner: review/merge #80, then machine-side
42 re-run of the 4K video (chunks should be ~tens of MB; full battery in task 17 still pending Gemini billing top-up).
43
44 **Checkpoint:** 2026-06-10 (M0 GREENLIT SEQUENCE EXECUTED ? session wrap). Owner ratified ADR-008 ("split driven by
45 productionization, not isolation") + greenlit M0.4+M0.1. **All 5 steps shipped, every PR merged on verified-green CI:**
46 - **#73 ADR-008** (DECISIONS.md): training in-repo behind an executable wall; artifact-bundle boundary; ship-gate
47 product-side; pre-committed split trigger ? `sports-temporal-models` at Gen-2/second-consumer.

123. user (2026-06-10T11:41:55.074Z)

The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\current-state.md has been updated successfully. (file state is current in your context ? no need to Read it back)

124. assistant (2026-06-10T11:42:14.324Z)

Done ? **the fix is fully merged to master (`f01387e`) with CI green**, though the merge itself had a twist worth knowing about.

What's on master now:

- **[#80](https://github.com/avidullu/badminton-highlight-indexer/pull/80)** ? the 4K oversize fix (re-encoded chunks via `indexing.ai\_chunk\_scale\_width`, loud provider skip, `run\_signals` channel, `oversize\_clip\_skipped` report rule). You (or a sibling session) had already merged this one before I got to it.
- **[#81](https://github.com/avidullu/badminton-highlight-indexer/pull/81)/[#82](https://github.com/avidullu/badminton-highlight-indexer/pull/82)** ? the test-coverage addition, which landed twice by race: when #80 merged, GitHub deleted the PR branch right as I was pushing the new tests to it; my push re-created the branch orphaned, you rescued it as #81, and my cherry-picked #82 merged 39 seconds later as a **confirmed-empty commit** (zero file changes ? harmless duplicate in history). Both stale branches are cleaned up, and the final master CI run is SUCCESS on both jobs.

On "test coverage and CI enhancement if needed": the genuine gap was that CI's full-suite lane has no `obsreport` (it's the private sibling repo ? can't be pip-installed there), so #80's reporting-rule tests **skip** in CI. The rule-firing half can't run there, period, but I added [test_run_signals.py](tests/test_run_signals.py) with **no importorskip** so CI now verifies the stdlib half: `incr`/`pop` consume-once semantics, per-video isolation, and the no-leak guarantee that `emit_indexer_report` consumes signals even with reporting disabled ? exactly the CI condition. No `ci.yml` change was needed beyond that. Suite is 336 green locally, 333 + skips in CI.

One heads-up: there's an **uncommitted WIP** in the working tree ? `backend/providers/gemini.py` extending generate retries 3?6 with exponential backoff + jitter for the 503 capacity waves. That's not mine (presumably yours or the baseline session's); I left it strictly untouched. It'll want its own PR ? and note it touches the same file as #80, so rebase/diff against fresh master before committing.

Memory checkpoint is updated with the merged state, the branch-race process note, and the WIP warning.